RAJALAKSHMI ENGINEERING COLLEGE
RAJALAKSHMI NAGAR, THANDALAM – 602 105



CS23334 Fundamentals of Data Science
Laboratory Record Note Book

Name :	Deepika P.
Year / Branch / Section :	II / CSE / F
University Register No. :	2116250701133
College Roll No. :	250701133
Semester :	3
Academic Year :	2025-2026

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CS23334 Fundamentals of Data Science

NAME	Deepika P.
ROLL NO	250701133
YEAR	II
SEC	F

RAJALAKSHMI ENGINEERING COLLEGE
An Autonomous Institution

BONAFIDE CERTIFICATE

Name: Deepika P.

Academic Year: 2025-2026 Semester: 3 Branch: CSE

Register No. 2116250701133

Certified that this is the bonafide record of work done by the above student in the CS23334

Fundamentals of Data Science Laboratory during the academic year 2025-2026.

Signature of Faculty in-charge

Submitted for the Practical Examination held on 05/10/2026

Internal Examiner External Examiner

INDEX

SlNO	Experiment	Date	GitHub
1	Installation of Open CV & To perform the basic image handling processing operation on the image	03/01/2026	
2	Implement Edge Detection, Line Detection and Corner Detection	10/01/2026	
3	Demonstrate Camera Calibration using python	24/01/2026	
4	Implement Image Histogram and Histogram Equalization	31/01/2026	
5	Develop a python program for Skin color Detection	07/02/2026	
6	Create a python program for Warping and Estimation	14/02/2026	
7	Develop a python program for Motion Tracking	21/02/2026	
8	Design a program for Object Detection using YOLO	07/03/2026	
9	Develop a python program for Stereo Vision and Depth Estimation	14/03/2026	
10	Demonstrate Augmented reality using feature matching	04/04/2026	
11	Mini Project <project title>	11/04/2026, 18/04/2026, 25/04/2026, 02/05/2026	

Experiment-1

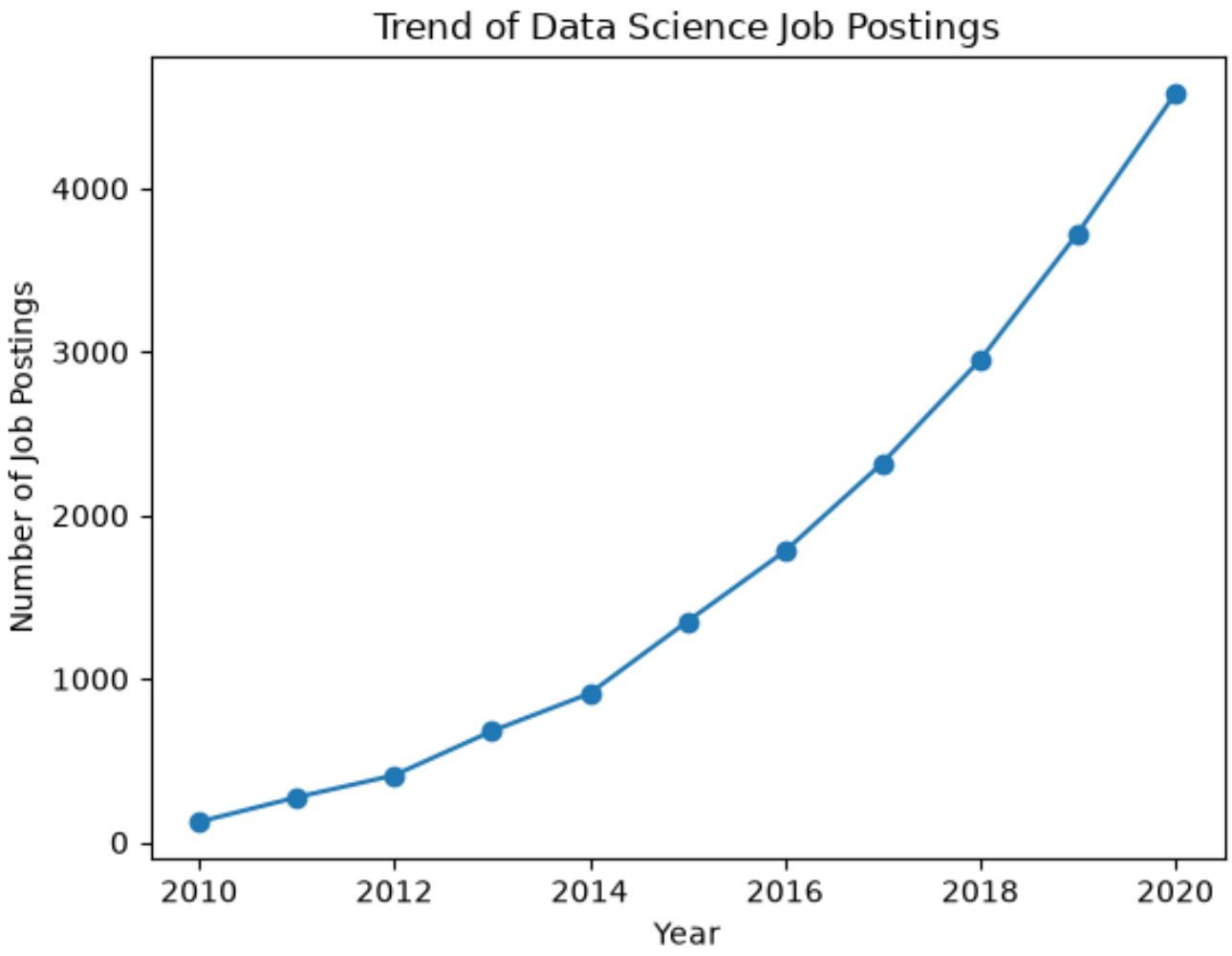
Code and complete execution output

Code Cell 1

```
# 1.a: Analyze the trend of data science job postings over the last decade
import pandas as pd
import matplotlib.pyplot as plt

data = {
    "Year": list(range(2010, 2021)),
    "Job Postings": [125, 275, 410, 680, 910, 1350, 1780, 2320, 2950, 3725, 4580],
}
df = pd.DataFrame(data)
plt.plot(df["Year"], df["Job Postings"], marker="o")
plt.title("Trend of Data Science Job Postings")
plt.xlabel("Year")
plt.ylabel("Number of Job Postings")
plt.show()
```

Output



Code Cell 2

```
# 1.b: Analyze and visualize the distribution of data science roles
import matplotlib.pyplot as plt

roles = ["Data Analyst", "Data Engineer", "Data Scientist", "ML Engineer", "Business Analyst"]
counts = [325, 465, 520, 245, 180]
plt.bar(roles, counts)
plt.title("Distribution of Data Science Roles")
plt.xlabel("Role")
plt.ylabel("Count")
plt.xticks(rotation=25, ha="right")
```

```
plt.tight_layout()
plt.show()
```

Output



Code Cell 3

```
# 1.c: Differentiate structured, unstructured, and semi-structured data
import pandas as pd

structured_data = pd.DataFrame({
    "ID": [11, 12, 13],
    "Name": ["Maya", "Noah", "Iris"],
    "Age": [28, 34, 41],
})
print("Structured Data:\n", structured_data)

unstructured_data = "A field note: strong winds moved the clouds east before noon."
print("\nUnstructured Data:\n", unstructured_data)

semi_structured_data = {
    "ID": 11,
    "Name": "Maya",
    "Attributes": {"Height": 172, "Weight": 64},
}
print("\nSemi-structured Data:\n", semi_structured_data)
```

Output

```
Structured Data:
    ID  Name  Age
0  11  Maya   28
1  12  Noah   34
2  13  Iris   41

Unstructured Data:
 A field note: strong winds moved the clouds east before noon.

Semi-structured Data:
 {'ID': 11, 'Name': 'Maya', 'Attributes': {'Height': 172, 'Weight': 64}}
```

Code Cell 4

```
# 1.d: Encrypt and decrypt sensitive data
from cryptography.fernet import Fernet

key = Fernet.generate_key()
cipher_suite = Fernet(key)
plain_text = b"Confidential field observation."
cipher_text = cipher_suite.encrypt(plain_text)
decrypted_text = cipher_suite.decrypt(cipher_text)
print("Original Data:", plain_text)
print("Encrypted Data:", cipher_text)
print("Decrypted Data:", decrypted_text)
```

Output

```
Original Data: b'Confidential field observation.'
Encrypted Data:
    b'gAAAAABq5Ot1dcAdNQNQ04MIar03bu93Y6RcLUvRz4pBvKhi0Jq5FV1X57cF_ptoM37e9iCCV1kUFk6Yce-yTpMpyQl1maJKbhEDaNuybpzAjG
    dPHnoNq4o='
Decrypted Data: b'Confidential field observation.'
```

Experiment-2

Code and complete execution output

Code Cell 1

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from pathlib import Path

# Load the alternate sales dataset
file_path = Path.cwd() / "data" / "sales-data.csv"
df = pd.read_csv(file_path)

# Display the first few rows of the DataFrame
print(df.head())

# Check for missing values
print(df.isnull().sum())

# Fill or drop missing values if necessary
df["Sales"] = df["Sales"].fillna(df["Sales"].mean())
df = df.dropna(subset=["Product", "Quantity", "Region"])

# Summary statistics
print(df.describe())

# Group by product and calculate the total sales and quantity
product_summary = df.groupby("Product").agg({
    "Sales": "sum",
    "Quantity": "sum",
}).reset_index()
print(product_summary)

# Bar plot of total sales by product
plt.figure(figsize=(10, 6))
plt.bar(product_summary["Product"], product_summary["Sales"])
plt.xlabel("Product")
plt.ylabel("Total Sales")
plt.title("Total Sales by Product")
plt.show()

# Line plot of sales over time
df["Date"] = pd.to_datetime(df["Date"])
sales_over_time = df.groupby("Date").agg({"Sales": "sum"}).reset_index()
plt.figure(figsize=(10, 6))
plt.plot(sales_over_time["Date"], sales_over_time["Sales"])
plt.xlabel("Date")
plt.ylabel("Total Sales")
plt.title("Sales Over Time")
plt.show()

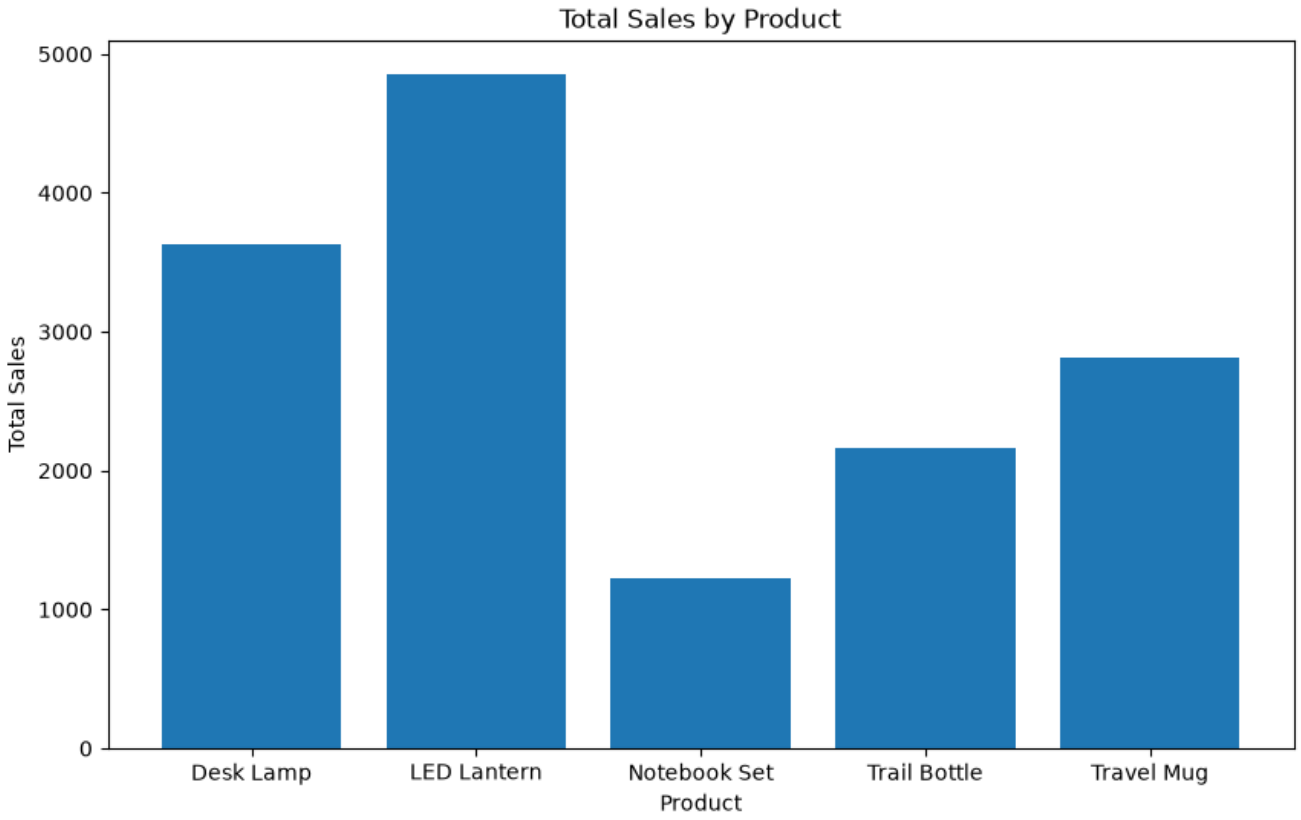
# Pivot table to analyze sales by region and product
pivot_table = df.pivot_table(
    values="Sales", index="Region", columns="Product", aggfunc=np.sum, fill_value=0
)
print(pivot_table)

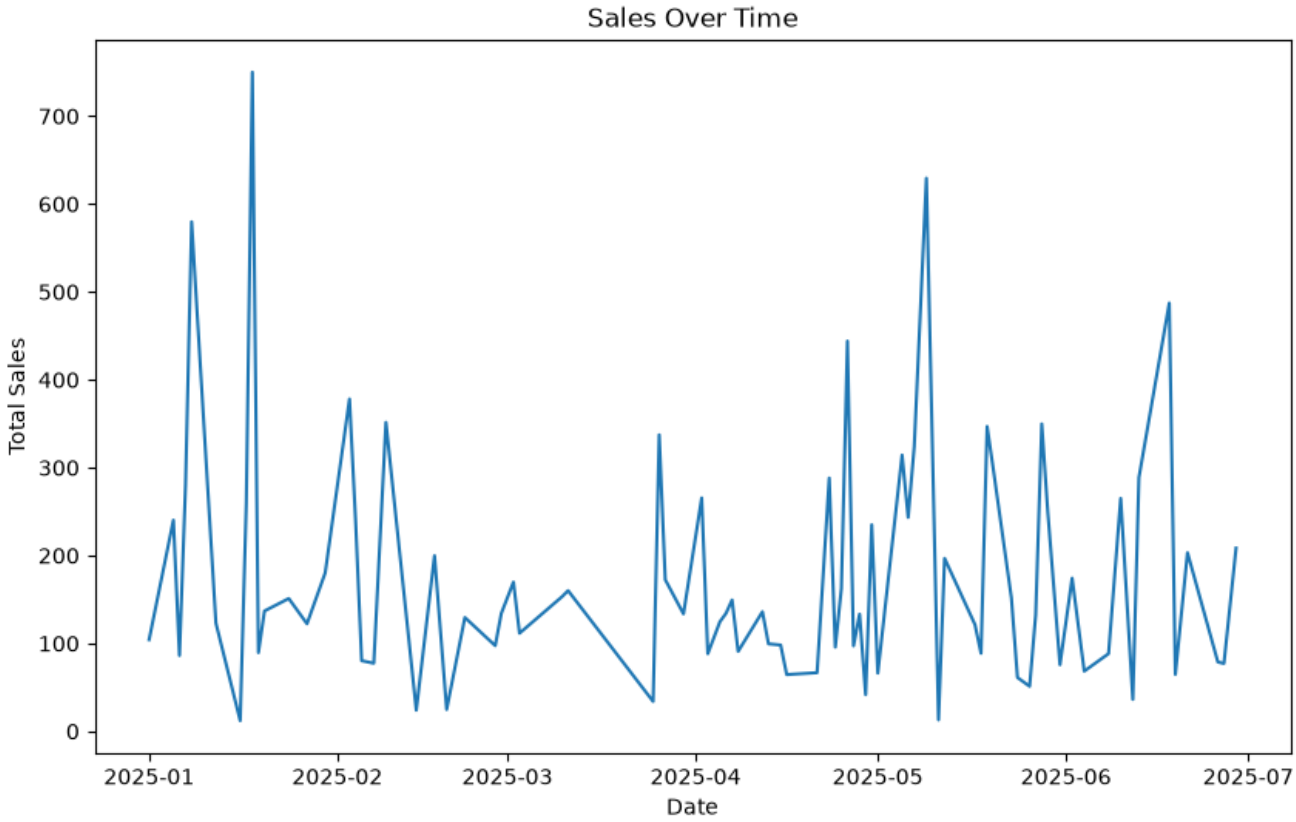
# Correlation matrix of numeric columns only
correlation_matrix = df.corr(numeric_only=True)
print(correlation_matrix)

# Heatmap of the correlation matrix
plt.figure(figsize=(8, 6))
sns.heatmap(correlation_matrix, annot=True, cmap="coolwarm")
plt.title("Correlation Matrix")
plt.show()
```

Output

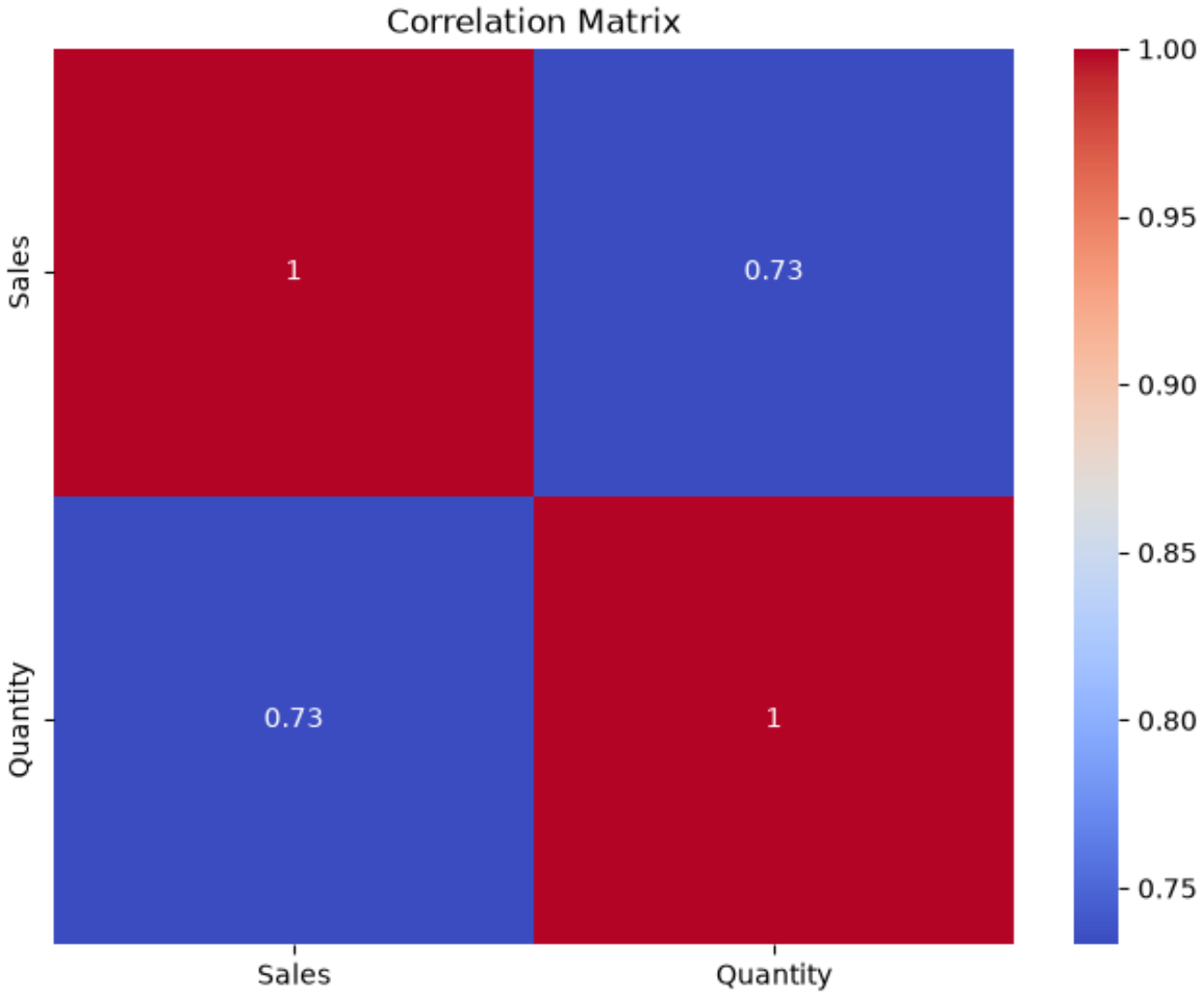
```
         Date       Product   Sales  Quantity    Region
0  2025-04-13  Trail Bottle   19.73         1  Northern
1  2025-01-05     Desk Lamp  240.26         7  Northern
2  2025-04-27    Travel Mug   97.16         4  Southern
3  2025-06-04  Notebook Set   68.39         6   Coastal
4  2025-05-12   LED Lantern  196.78         5  Northern
Date        0
Product     0
Sales       2
Quantity    0
Region      1
dtype: int64
            Sales    Quantity
count  119.000000  119.000000
mean   123.470832    4.747899
std     78.830139    2.159796
min     12.040000    1.000000
25%     68.060000    3.000000
50%     97.390000    5.000000
75%    163.975000    7.000000
max    351.570000    8.000000
        Product        Sales  Quantity
0     Desk Lamp  3634.669492       113
1   LED Lantern  4854.010000       118
2  Notebook Set  1229.410000       102
3  Trail Bottle  2164.340000       115
4    Travel Mug  2810.599492       117
```





Product	Desk Lamp	LED Lantern	Notebook Set	Trail Bottle	Travel Mug
Region					
Central	1146.389492	1952.81	468.48	425.17	546.450000
Coastal	915.980000	206.60	384.21	655.66	781.059492
Northern	1248.840000	804.99	159.11	719.89	368.740000
Southern	323.460000	1889.61	217.61	363.62	1114.350000

	Sales	Quantity
Sales	1.000000	0.733335
Quantity	0.733335	1.000000


Correlation Matrix
Sales
Quantity
1
0.73
0.73
1
Sales
Quantity
1.00
0.95
0.90
0.85
0.80
0.75

Experiment-3

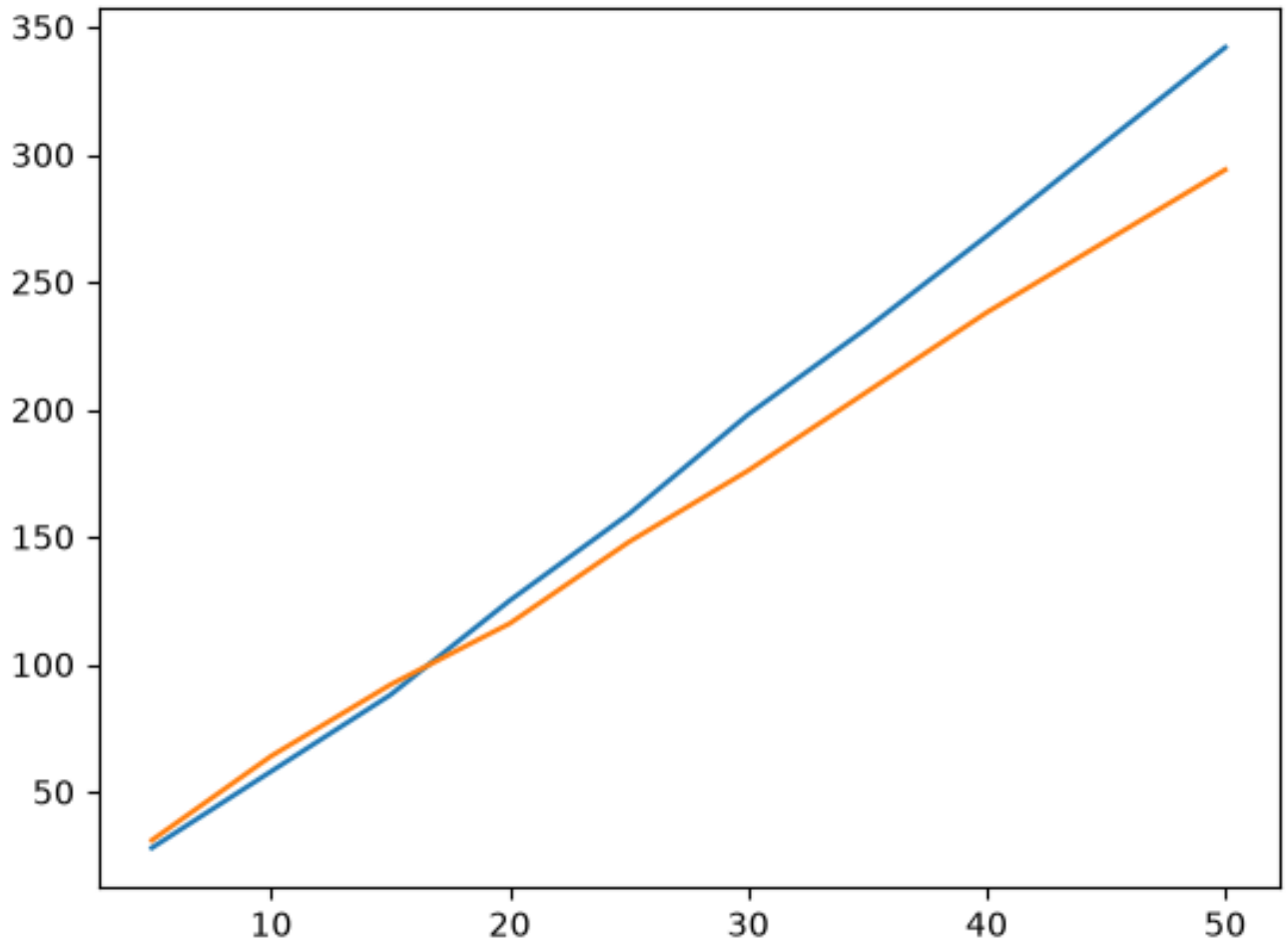
Code and complete execution output

Code Cell 1

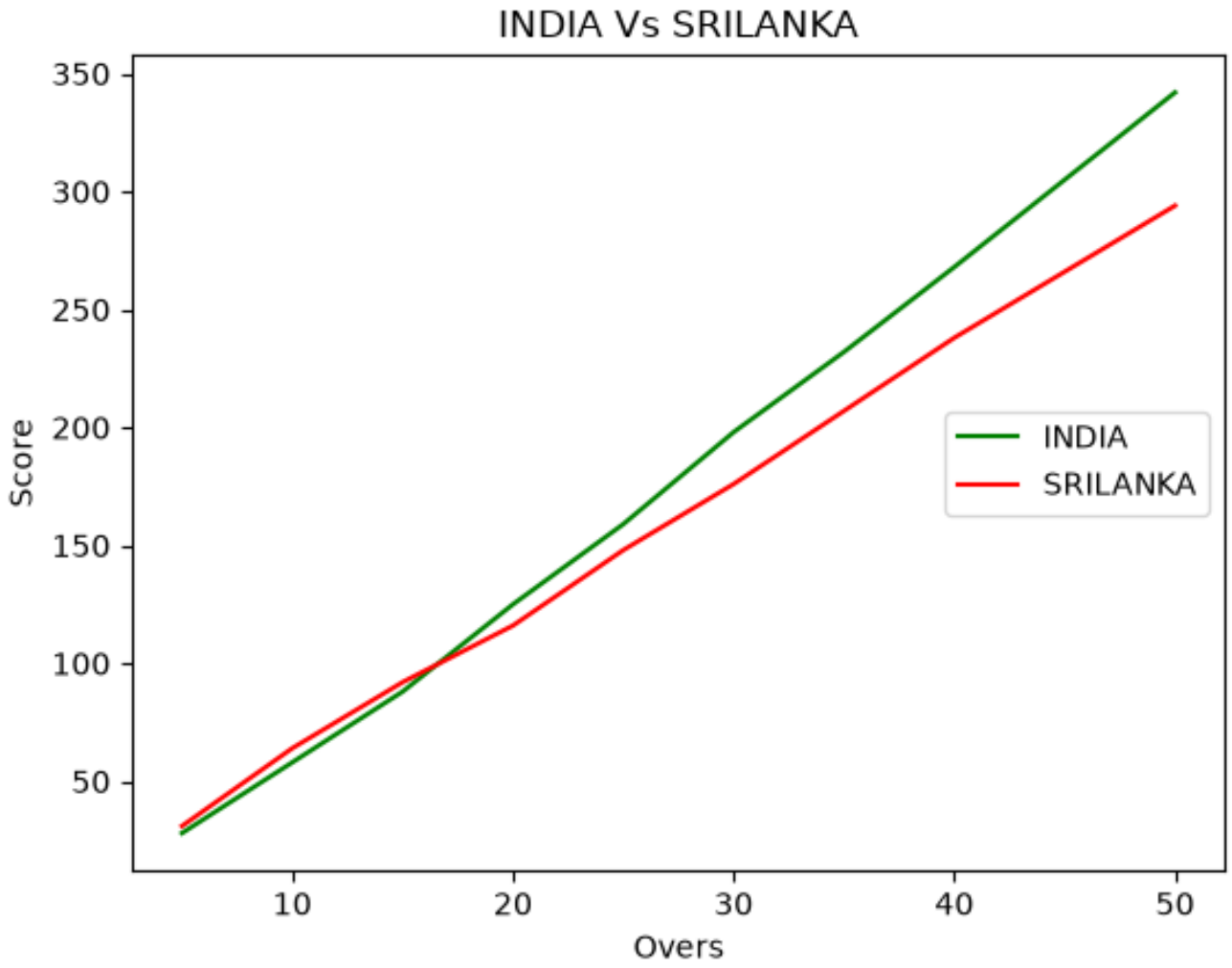
```
# 3.a: Data visualization using a line plot
import matplotlib.pyplot as cricket

Overs = list(range(5, 51, 5))
Indian_Score = [28, 58, 88, 125, 159, 198, 232, 268, 305, 342]
Srilankan_Score = [31, 64, 92, 116, 148, 176, 207, 238, 266, 294]
cricket.plot(Overs, Indian_Score)
cricket.plot(Overs, Srilankan_Score)
cricket.show()
cricket.title("INDIA Vs SRILANKA")
cricket.xlabel("Overs")
cricket.ylabel("Score")
cricket.legend()
cricket.plot(Overs, Indian_Score, color="green", label="INDIA")
cricket.plot(Overs, Srilankan_Score, color="red", label="SRILANKA")
cricket.legend(loc="center right")
cricket.show()
```

Output




```
C:\Users\Myself\AppData\Local\Temp\ipykernel_8104\2087024106.py:13: UserWarning: No artists with labels found to
    put in legend.  Note that artists whose label start with an underscore are ignored when legend() is called with
    no argument.
  cricket.legend()
```

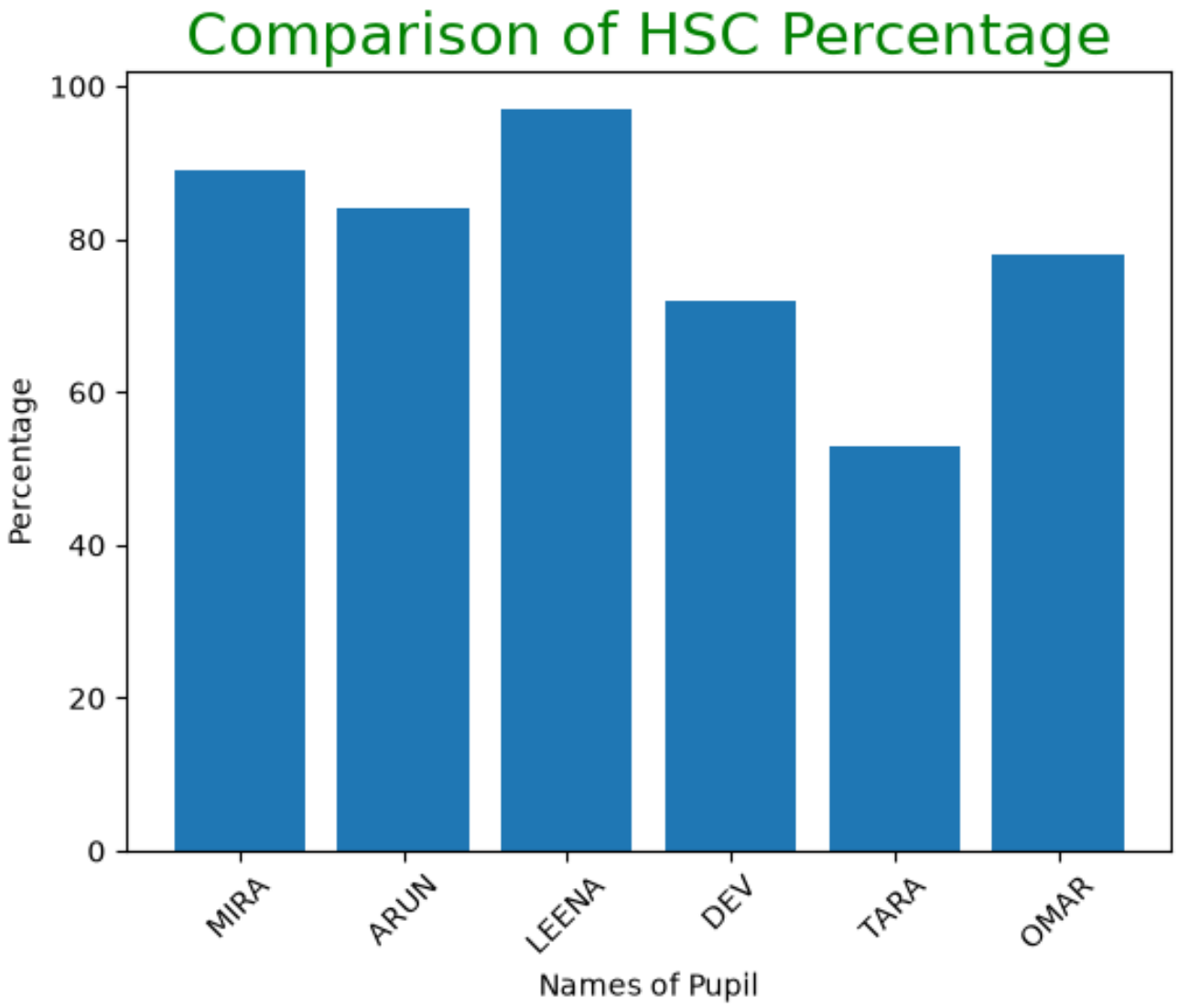


Code Cell 2

```
# 3.b: Data visualization using a bar chart
import matplotlib.pyplot as hscmark
import numpy as np

Names = ["MIRA", "ARUN", "LEENA", "DEV", "TARA", "OMAR"]
xaxis = np.arange(len(Names))
Percentage_hsc = [89, 84, 97, 72, 53, 78]
hscmark.bar(Names, Percentage_hsc)
hscmark.xticks(xaxis, Names, rotation=45)
hscmark.xlabel("Names of Pupil")
hscmark.ylabel("Percentage")
hscmark.title("Comparison of HSC Percentage", fontsize=20, color="green")
hscmark.show()
```

Output

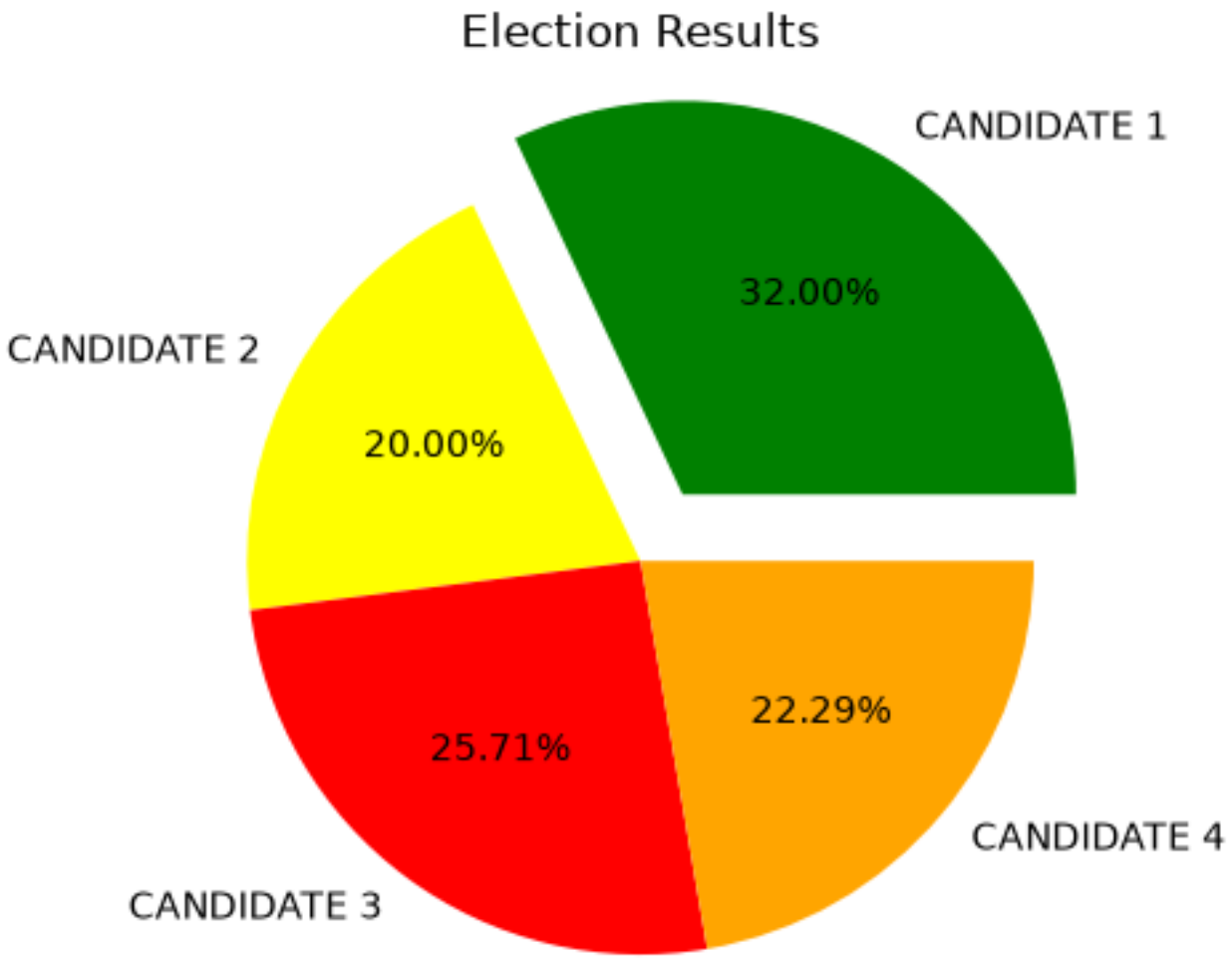


Code Cell 3

```
# 3.c: Data visualization using a pie chart
import matplotlib.pyplot as election

labels = ["CANDIDATE 1", "CANDIDATE 2", "CANDIDATE 3", "CANDIDATE 4"]
Votes = [280, 175, 225, 195]
colors = ["green", "yellow", "red", "orange"]
explode = (0.2, 0, 0, 0)
election.pie(Votes, labels=labels, colors=colors, explode=explode, autopct="%0.2f%%")
election.title("Election Results")
election.show()
```

Output


Election Results
CANDIDATE 1
32.00%
CANDIDATE 2
20.00%
22.29%
25.71%
CANDIDATE 4
CANDIDATE 3

Experiment-4

Code and complete execution output

Code Cell 1

```
import nltk
from nltk.tokenize import word_tokenize
from nltk.corpus import gutenberg

# Download the Gutenberg corpus and tokenizer resources if needed
# nltk.download("gutenberg")
# nltk.download("punkt")
# nltk.download("punkt_tab")

# Use a different Gutenberg text from the one in the manual
sample = gutenberg.raw("carroll-alice.txt")
token = word_tokenize(sample)
wlist = []
for i in range(50):
    wlist.append(token[i])
wordfreq = [wlist.count(word) for word in wlist]
print("Pairs\n" + str(list(zip(wlist, wordfreq))))
```

Output

```
Pairs
[('[', 1), ('Alice', 2), ("'s", 1), ('Adventures', 1), ('in', 1), ('Wonderland', 1), ('by', 2), ('Lewis', 1),
    ('Carroll', 1), ('1865', 1), (']', 1), ('CHAPTER', 1), ('I', 1), ('.', 1), ('Down', 1), ('the', 3),
    ('Rabbit-Hole', 1), ('Alice', 2), ('was', 1), ('beginning', 1), ('to', 2), ('get', 1), ('very', 1), ('tired',
    1), ('of', 2), ('sitting', 1), ('by', 2), ('her', 2), ('sister', 1), ('on', 1), ('the', 3), ('bank', 1), (',',
    1), ('and', 1), ('of', 2), ('having', 1), ('nothing', 1), ('to', 2), ('do', 1), (':', 1), ('once', 1), ('or',
    1), ('twice', 1), ('she', 1), ('had', 1), ('peeped', 1), ('into', 1), ('the', 3), ('book', 1), ('her', 2)]
```

Experiment-5

Code and complete execution output

Code Cell 1

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from pathlib import Path

diabetes_df = pd.read_csv(Path.cwd() / "data" / "diabetes.csv")
print(diabetes_df.head())

# Dataset structure and summary statistics
print(diabetes_df.info())
print(diabetes_df.describe())

# Visualize the distribution of each feature
diabetes_df.hist(bins=50, figsize=(20, 15))
plt.show()

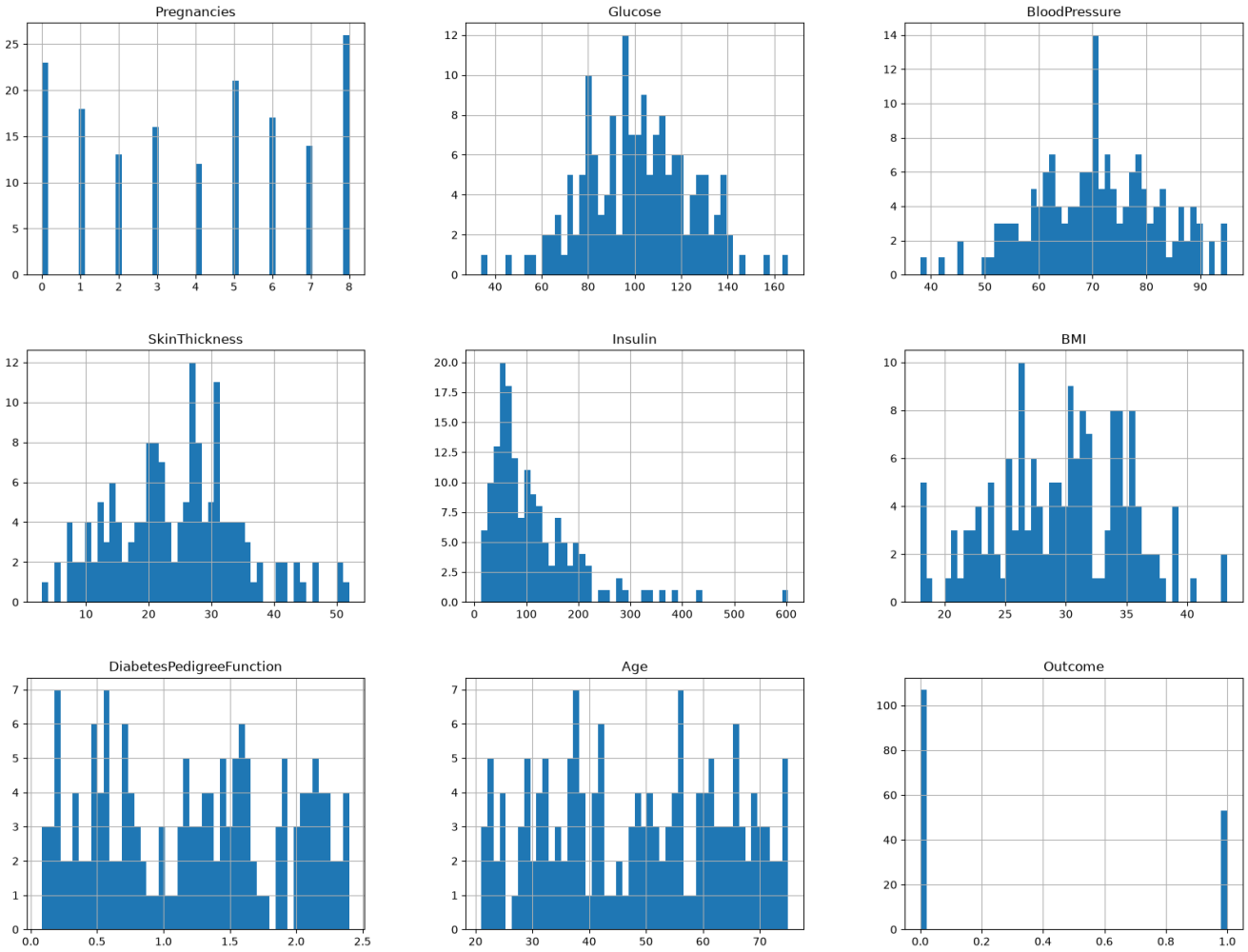
# Pairplot to visualize relationships
sns.pairplot(diabetes_df)
plt.show()
```

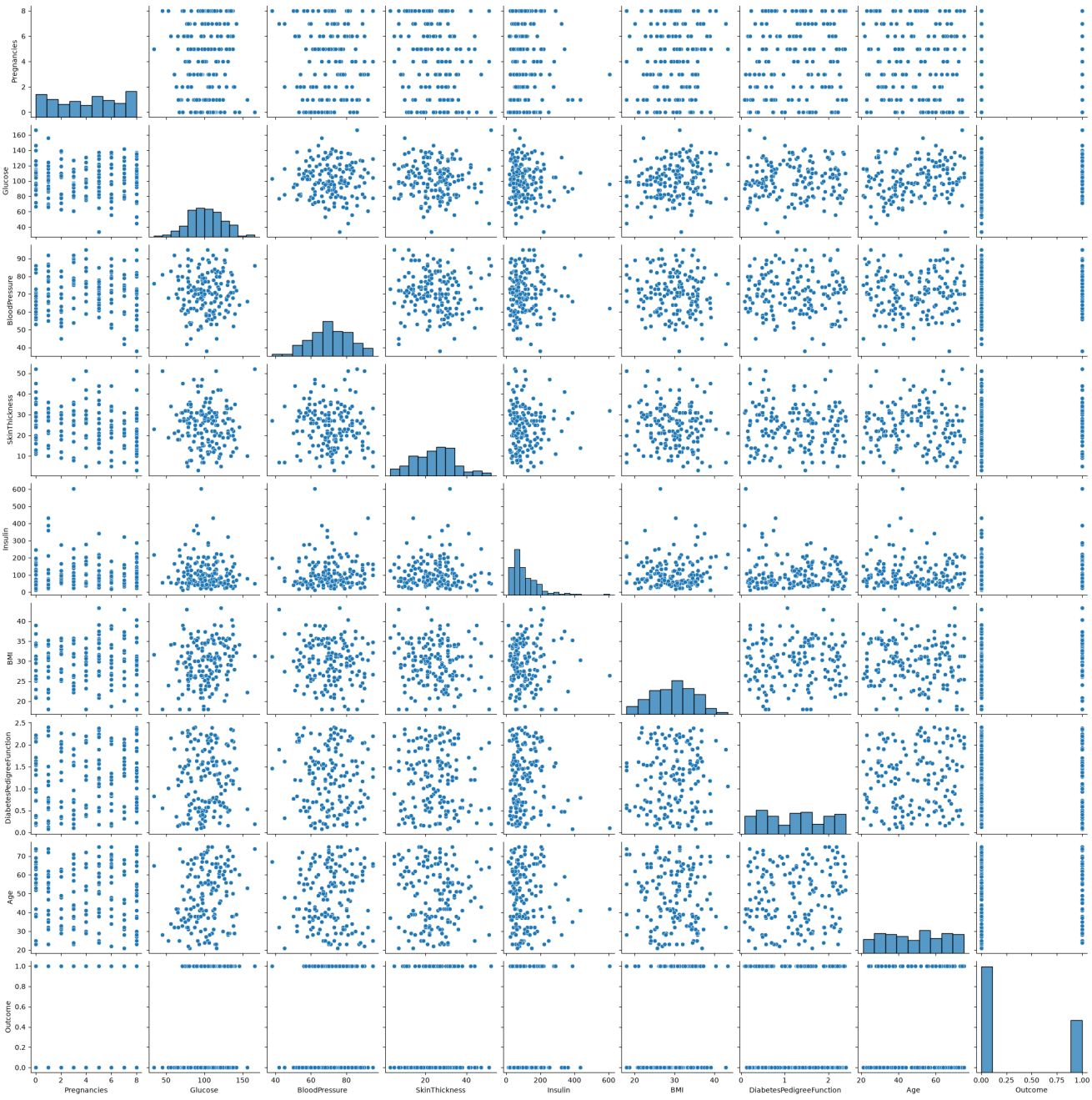
Output

```
   Pregnancies  Glucose  BloodPressure  SkinThickness  Insulin   BMI  \
0            6      135             82             33      122  30.3
1            4      124             85             19      180  35.8
2            0      108             56             38       60  31.1
3            5       98             63             26      161  20.9
4            1      116             70             31      130  32.0

   DiabetesPedigreeFunction  Age  Outcome
0                     1.188   56        0
1                     1.637   30        1
2                     0.743   55        1
3                     1.606   41        1
4                     1.325   23        0
<class 'pandas.DataFrame'>
RangeIndex: 160 entries, 0 to 159
Data columns (total 9 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   Pregnancies               160 non-null    int64
 1   Glucose                   160 non-null    int64
 2   BloodPressure             160 non-null    int64
 3   SkinThickness             160 non-null    int64
 4   Insulin                   160 non-null    int64
 5   BMI                       160 non-null    float64
 6   DiabetesPedigreeFunction  160 non-null    float64
 7   Age                       160 non-null    int64
 8   Outcome                   160 non-null    int64
dtypes: float64(2), int64(7)
memory usage: 11.4 KB
None
       Pregnancies     Glucose  BloodPressure  SkinThickness     Insulin  \
count   160.000000  160.000000     160.000000     160.000000  160.000000
mean      4.081250  101.631250      70.706250      24.512500  111.868750
std       2.779014   22.683943      11.521703      10.162818   86.053305
min       0.000000   34.000000      38.000000       3.000000   13.000000
25%       1.000000   85.250000      62.000000      17.750000   56.000000
50%       4.000000  101.000000      71.000000      25.000000   87.000000
75%       6.250000  116.000000      79.000000      31.000000  149.250000
max       8.000000  166.000000      95.000000      52.000000  604.000000

              BMI  DiabetesPedigreeFunction         Age     Outcome
count  160.000000                160.000000  160.000000  160.000000
mean    29.667500                  1.220919   48.443750    0.331250
std      5.392746                  0.684733   15.549372    0.472141
min     18.000000                  0.086000   21.000000    0.000000
25%     26.050000                  0.574250   36.000000    0.000000
50%     30.300000                  1.255000   49.500000    0.000000
75%     33.950000                  1.788750   62.000000    1.000000
max     43.300000                  2.399000   75.000000    1.000000
```


Pregnancies
Glucose
BloodPressure
SkinThickness
Insulin
BMI
DiabetesPedigreeFunction
Age
Outcome


Pregnancies
Glucose
BloodPressure
SkinThickness
Insulin
BMI
DiabetesPedigreeFunction
Age
Outcome

Experiment-6

Code and complete execution output

Code Cell 1

```
import pandas as pd
import matplotlib.pyplot as plt
from statsmodels.tsa.seasonal import seasonal_decompose
from pathlib import Path

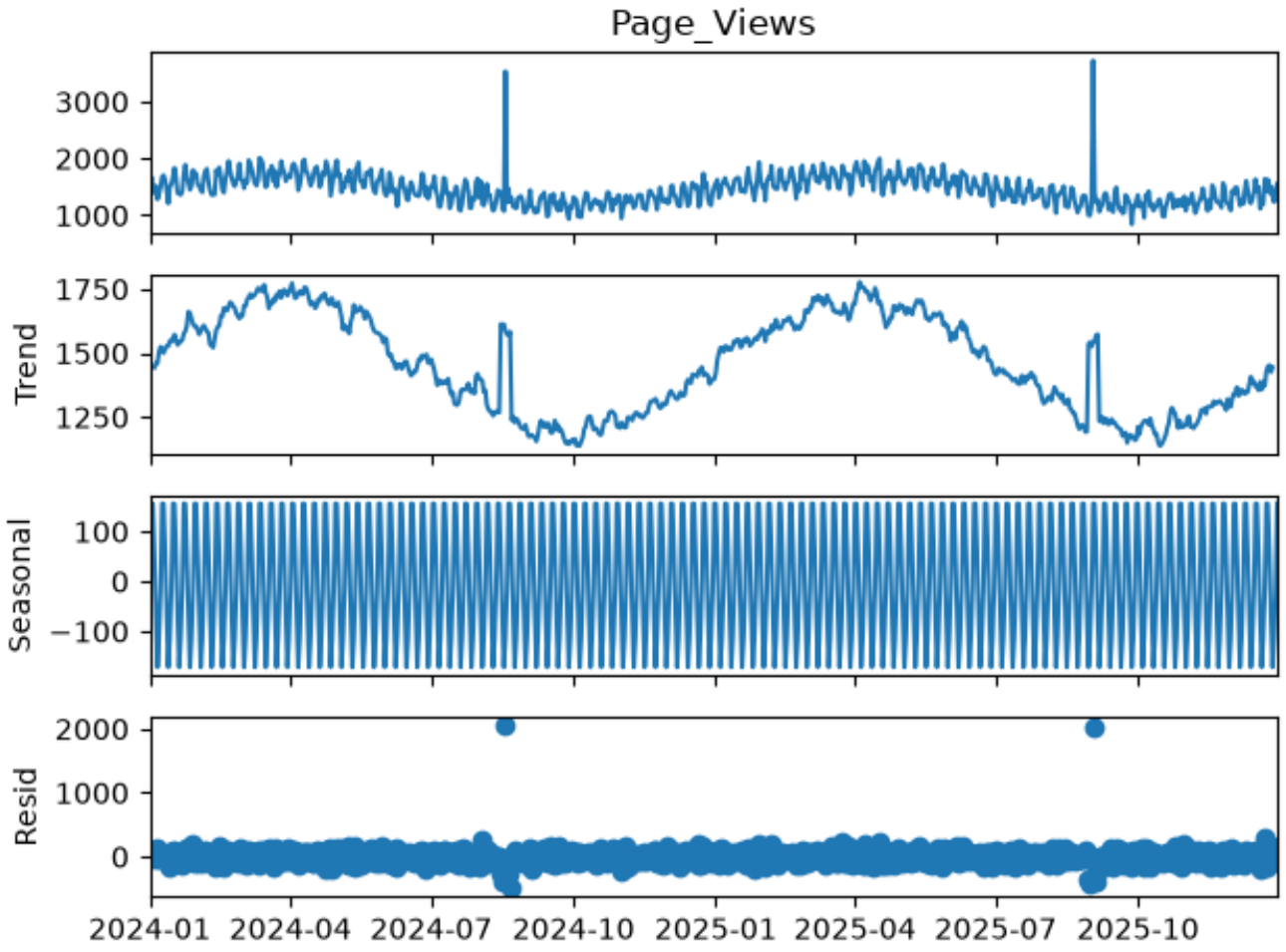
# Load the alternate website traffic data
traffic_path = Path.cwd() / "data" / "website_traffic.csv"
data = pd.read_csv(traffic_path, parse_dates=["Date"], index_col="Date")

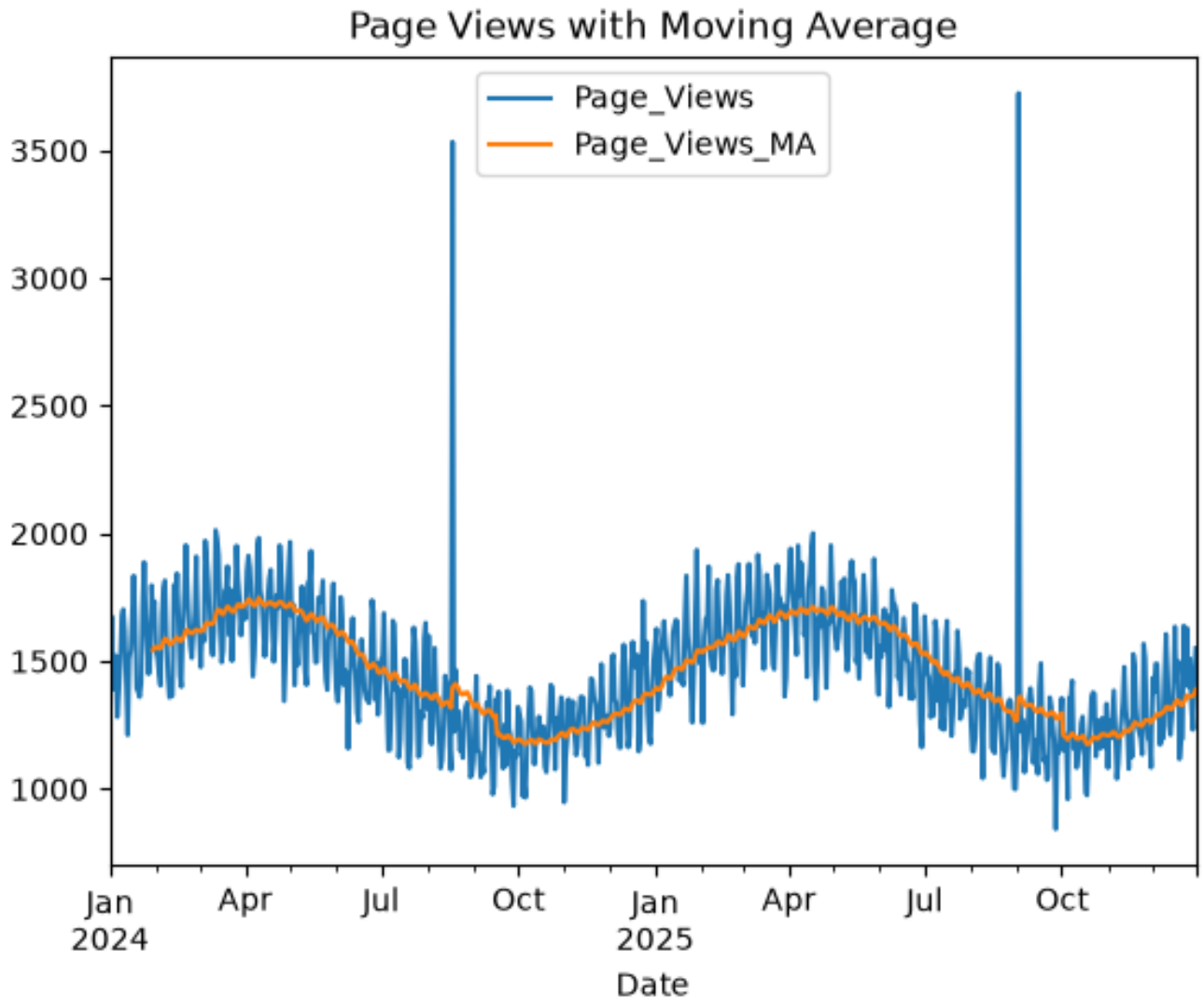
# Time Series Decomposition
decomposition = seasonal_decompose(data["Page_Views"], model="additive")
decomposition.plot()
plt.show()

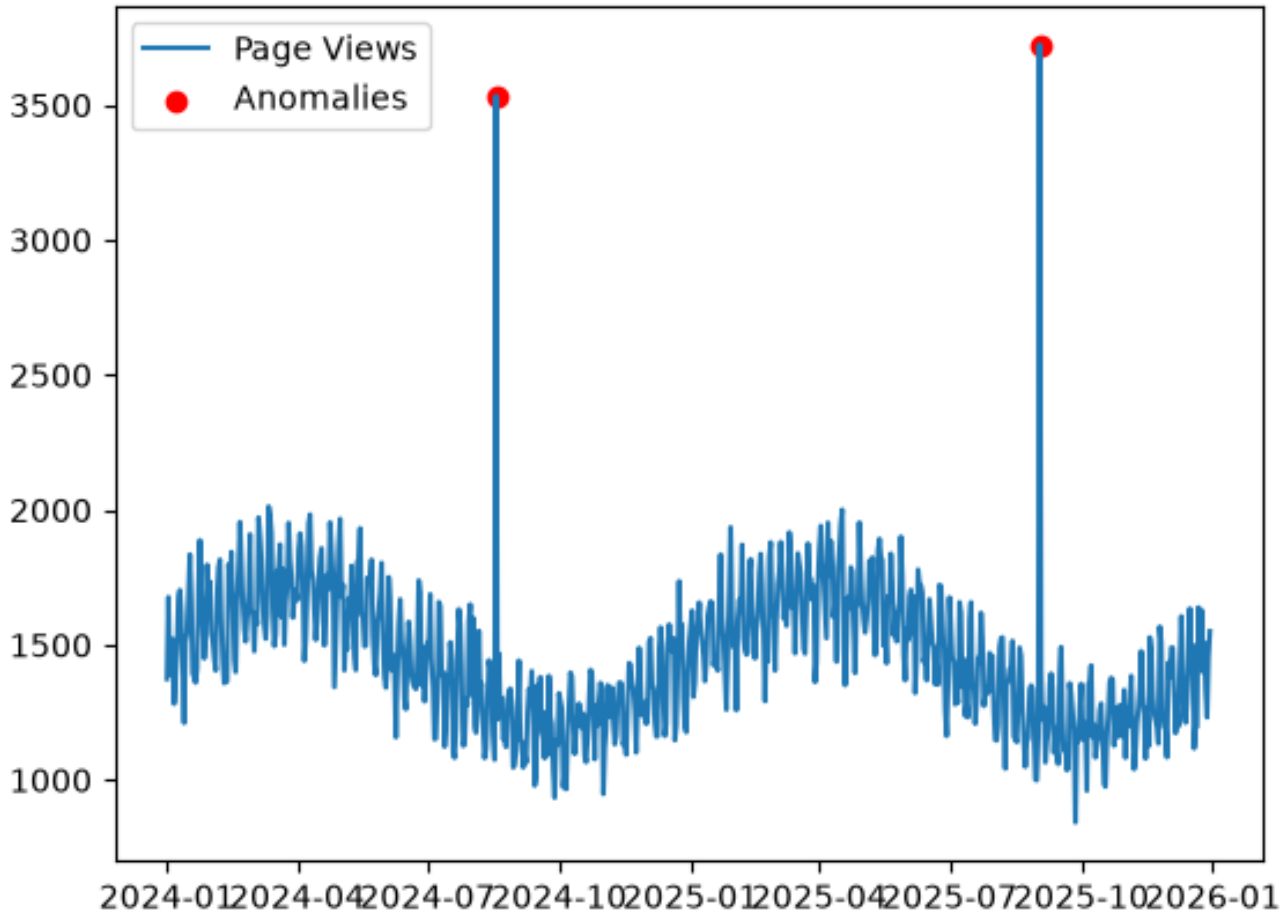
# Moving Average
data["Page_Views_MA"] = data["Page_Views"].rolling(window=30).mean()
data[["Page_Views", "Page_Views_MA"]].plot()
plt.title("Page Views with Moving Average")
plt.show()

# Anomaly Detection
data["Page_Views_Z"] = (
    data["Page_Views"] - data["Page_Views"].mean()
) / data["Page_Views"].std()
anomalies = data[data["Page_Views_Z"].abs() > 3]
plt.plot(data.index, data["Page_Views"], label="Page Views")
plt.scatter(anomalies.index, anomalies["Page_Views"], color="red", label="Anomalies")
plt.legend()
plt.show()
```

Output




Page Views with Moving Average
Page_Views
Page_Views_MA
3500
3000
2500
2000
1500
1000
Jan
2024
Apr
Jul
Oct
Jan
2025
Apr
Jul
Oct
Date


Page Views
Anomalies
3500
3000
2500
2000
1500
1000
2024-01
2024-04
2024-07
2024-10
2025-01
2025-04
2025-07
2025-10
2026-01

Experiment-7

Code and complete execution output

Code Cell 1

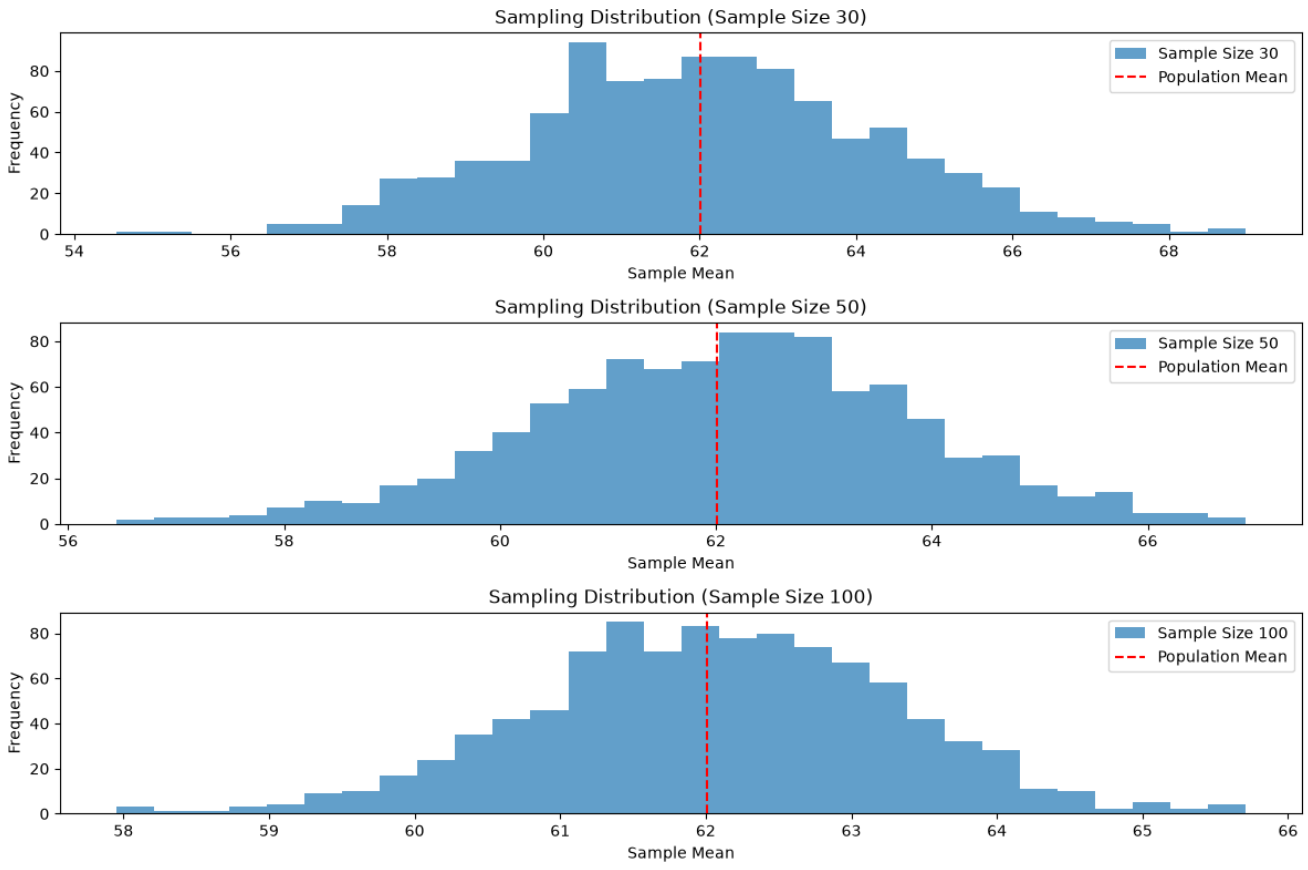
```
import numpy as np
import matplotlib.pyplot as plt

# Generate a normal population with different parameters from the example
population_mean = 62
population_std = 12
population_size = 100000
population = np.random.normal(population_mean, population_std, population_size)

# Random sampling
sample_sizes = [30, 50, 100]
num_samples = 1000
sample_means = {}
for size in sample_sizes:
    sample_means[size] = []
    for _ in range(num_samples):
        sample = np.random.choice(population, size=size, replace=False)
        sample_means[size].append(np.mean(sample))

# Plot sampling distributions
plt.figure(figsize=(12, 8))
for i, size in enumerate(sample_sizes):
    plt.subplot(len(sample_sizes), 1, i + 1)
    plt.hist(sample_means[size], bins=30, alpha=0.7, label=f"Sample Size {size}")
    plt.axvline(np.mean(population), color="red", linestyle="dashed", linewidth=1.5,
                label="Population Mean")
    plt.title(f"Sampling Distribution (Sample Size {size})")
    plt.xlabel("Sample Mean")
    plt.ylabel("Frequency")
    plt.legend()
plt.tight_layout()
plt.show()
```

Output



Experiment-8

Code and complete execution output

Code Cell 1

```
import numpy as np
import scipy.stats as stats

# Alternate hypothetical weights in grams
sample_data = np.array([
    154, 151, 149, 153, 146, 155, 150, 152, 148, 154,
    151, 147, 156, 150, 153, 149, 152, 145, 154, 151,
    148, 155, 150, 152, 147, 153, 149, 156, 151, 148,
])

# Population mean under the null hypothesis
population_mean = 150
sample_mean = np.mean(sample_data)
sample_std = np.std(sample_data, ddof=1)
n = len(sample_data)
z_statistic = (sample_mean - population_mean) / (sample_std / np.sqrt(n))
p_value = 2 * (1 - stats.norm.cdf(np.abs(z_statistic)))
print(f"Sample Mean: {sample_mean:.2f}")
print(f"Z-Statistic: {z_statistic:.4f}")
print(f"P-Value: {p_value:.4f}")
alpha = 0.05
if p_value < alpha:
    print("Reject the null hypothesis: The average weight is significantly different from 150 grams.")
else:
    print("Fail to reject the null hypothesis: There is no significant difference in average weight from 150
    grams.")
```

Output

```
Sample Mean: 150.97
Z-Statistic: 1.7718
P-Value: 0.0764
Fail to reject the null hypothesis: There is no significant difference in average weight from 150 grams.
```

Experiment-9

Code and complete execution output

Code Cell 1

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from pathlib import Path

data = pd.read_csv(Path.cwd() / "data" / "iris_visualization.csv")
data.info()
print(data.describe())
print(data.value_counts("variety"))
sns.countplot(x="variety", data=data)
plt.show()

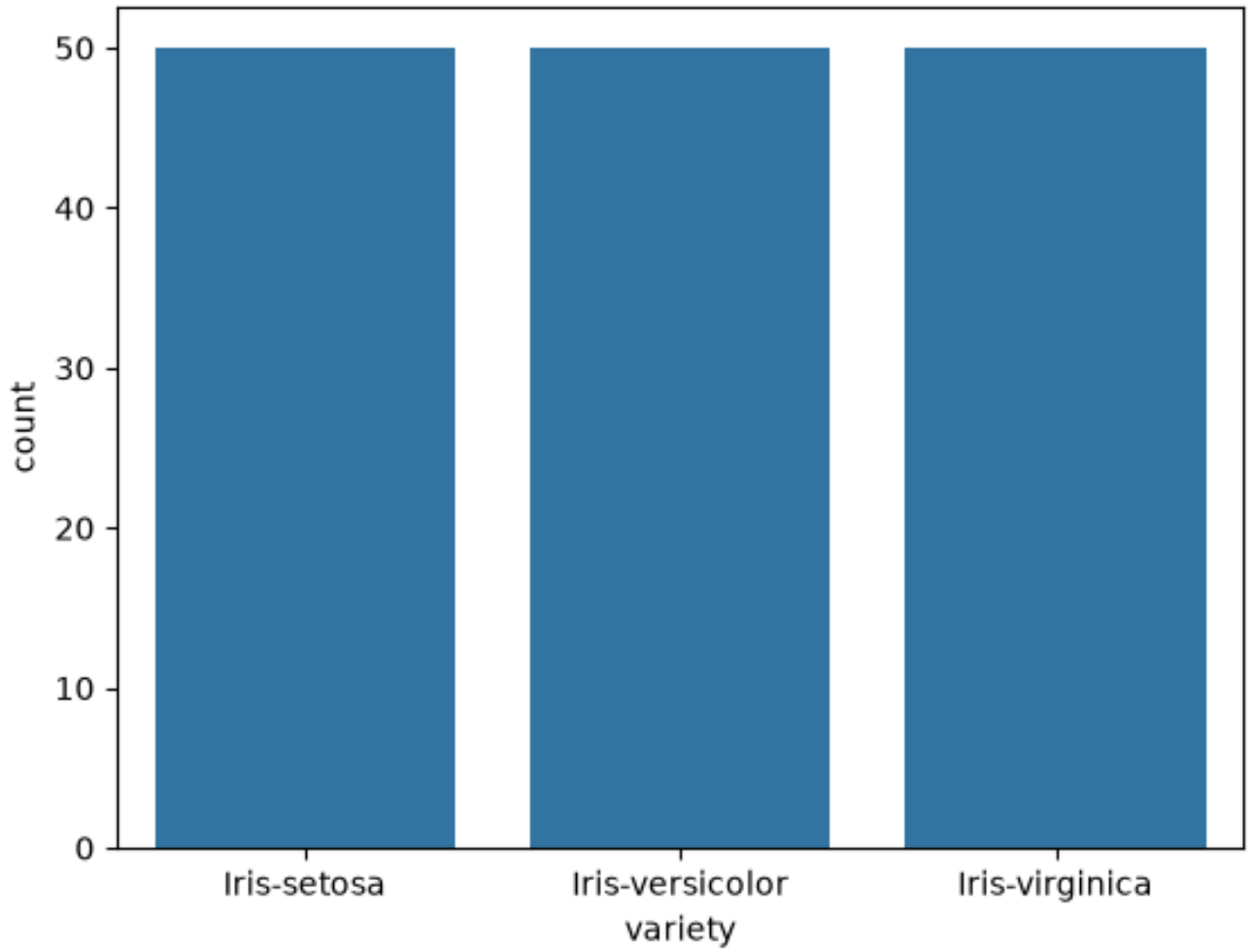
dummies = pd.get_dummies(data["variety"])
FinalDataset = pd.concat([dummies, data.iloc[:, [0, 1, 2, 3]]], axis=1)
print(FinalDataset.head())

sns.scatterplot(x="SepalLengthCm", y="SepalWidthCm", hue="variety", data=data)
plt.show()
sns.scatterplot(x="PetalLengthCm", y="PetalWidthCm", hue="variety", data=data)
plt.show()
sns.pairplot(data, hue="variety", height=3)
plt.show()

for feature in ["PetalLengthCm", "PetalWidthCm", "SepalLengthCm", "SepalWidthCm"]:
    sns.FacetGrid(data, hue="variety", height=5).map(sns.histplot, feature).add_legend()
    plt.show()
```

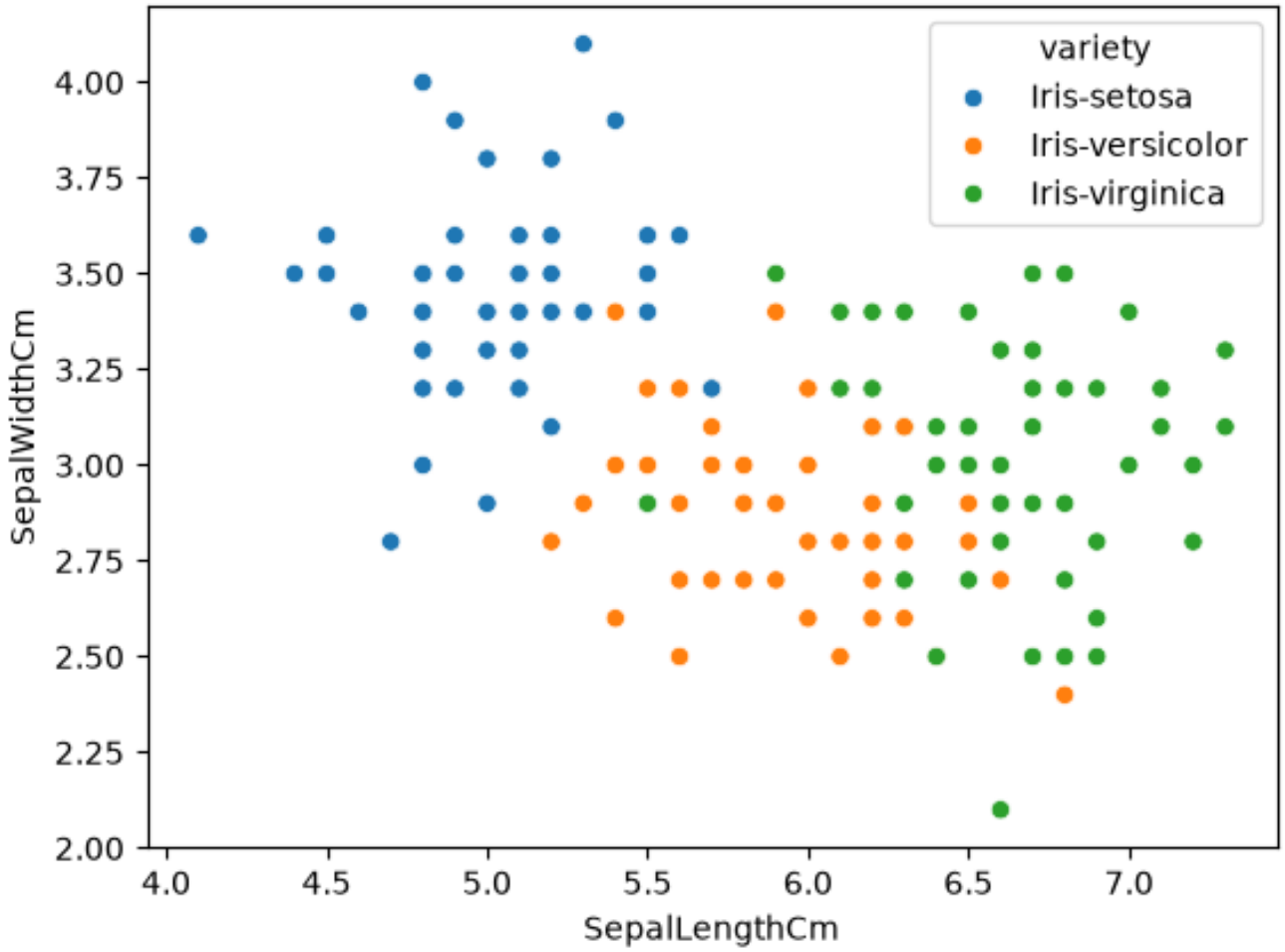
Output

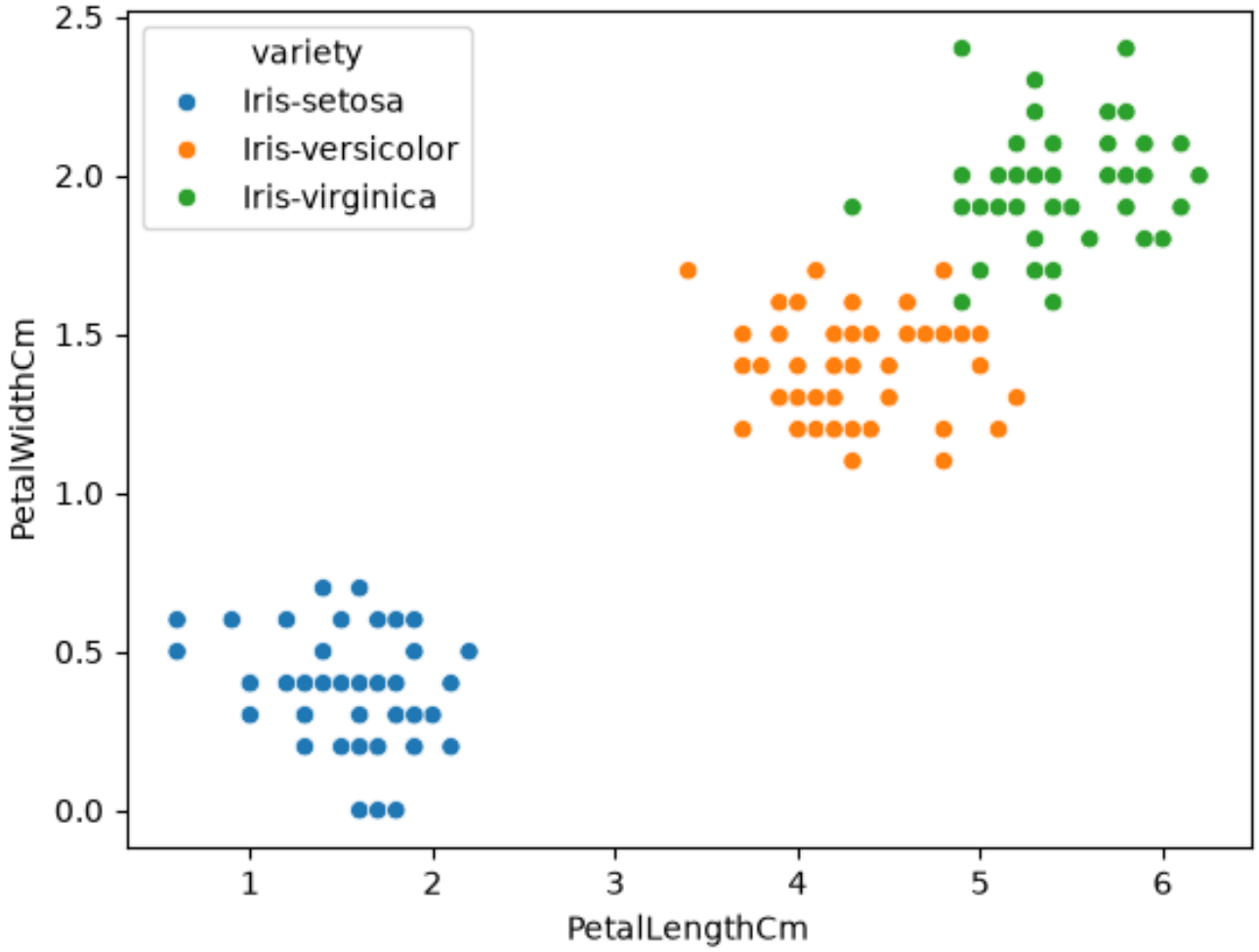
```
<class 'pandas.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 6 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   Id             150 non-null    int64
 1   SepalLengthCm  150 non-null    float64
 2   SepalWidthCm   150 non-null    float64
 3   PetalLengthCm  150 non-null    float64
 4   PetalWidthCm   150 non-null    float64
 5   variety        150 non-null    str
dtypes: float64(4), int64(1), str(1)
memory usage: 7.2 KB
               Id  SepalLengthCm  SepalWidthCm  PetalLengthCm  PetalWidthCm
count  150.000000     150.000000    150.000000     150.000000    150.000000
mean   275.500000       5.854000      3.106667       3.776667      1.248667
std     43.445368       0.735726      0.370138       1.686879      0.685197
min    201.000000       4.100000      2.100000       0.600000      0.000000
25%    238.250000       5.200000      2.825000       1.800000      0.500000
50%    275.500000       5.900000      3.100000       4.200000      1.400000
75%    312.750000       6.500000      3.400000       5.200000      1.900000
max    350.000000       7.300000      4.100000       6.200000      2.400000
variety
Iris-setosa        50
Iris-versicolor    50
Iris-virginica     50
Name: count, dtype: int64
```

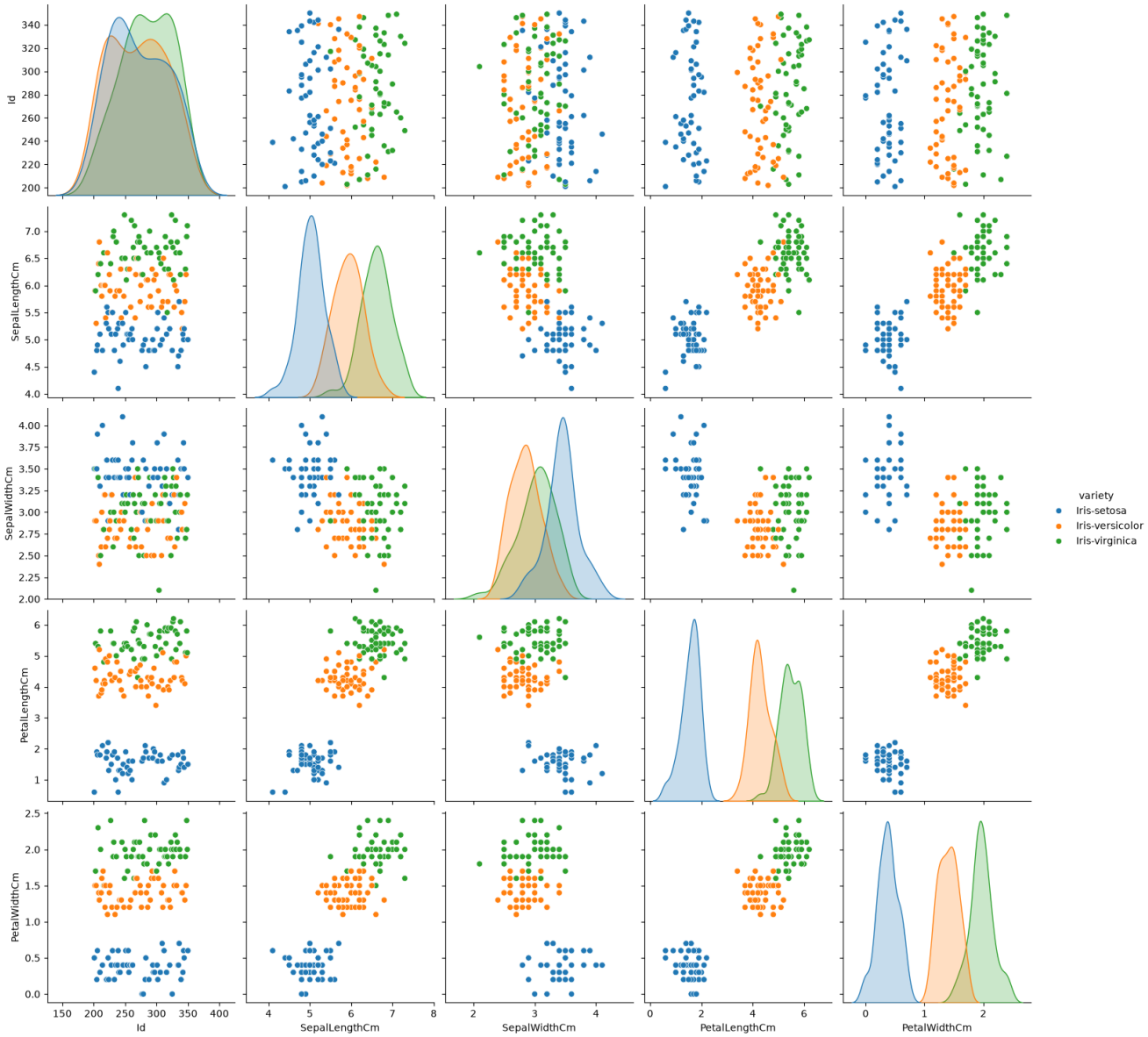


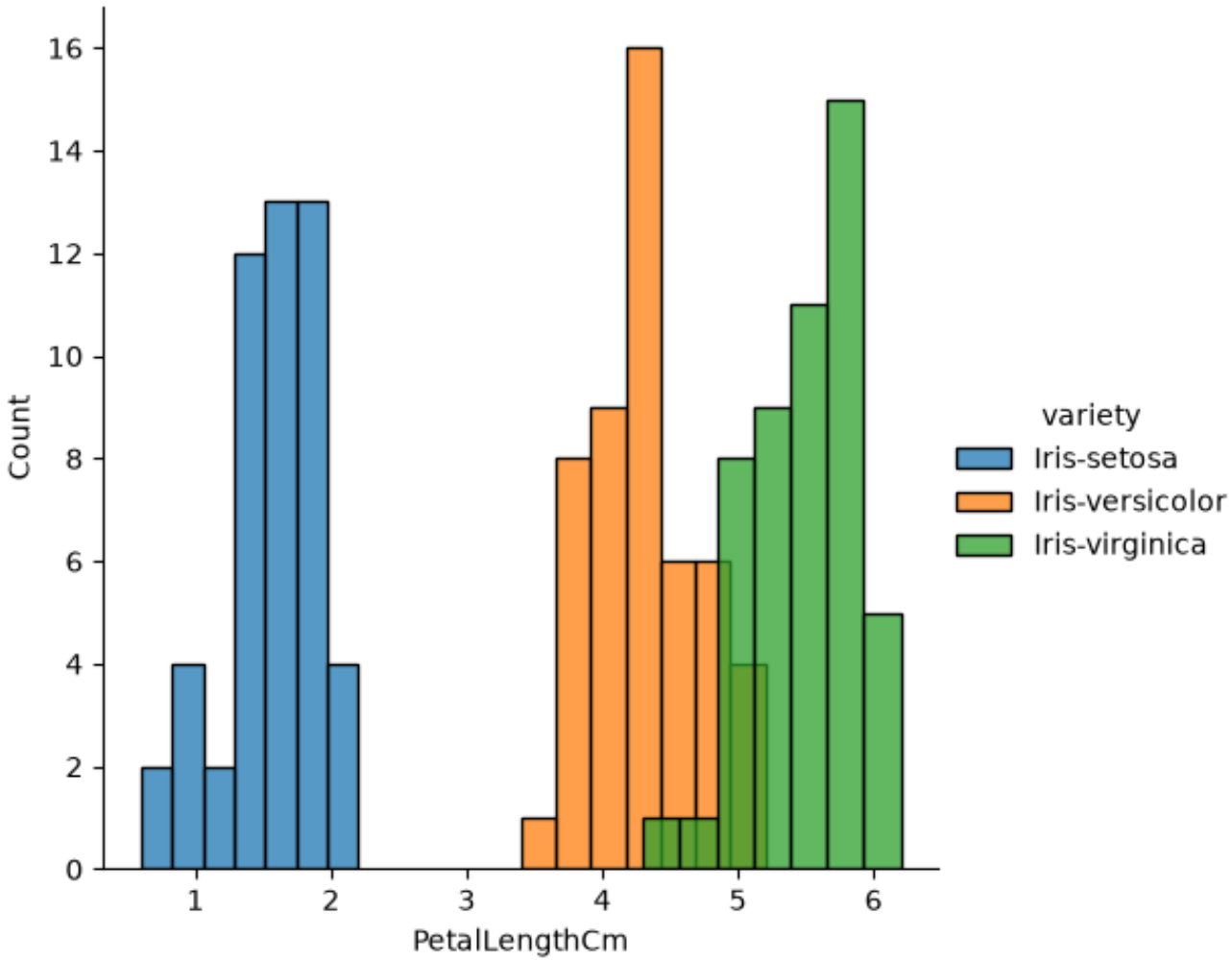
	Iris-setosa	Iris-versicolor	Iris-virginica	Id	SepalLengthCm	\
0	True	False	False	201	4.4	
1	False	True	False	202	5.9	
2	False	False	True	203	5.9	
3	False	True	False	204	5.3	
4	True	False	False	205	4.8	

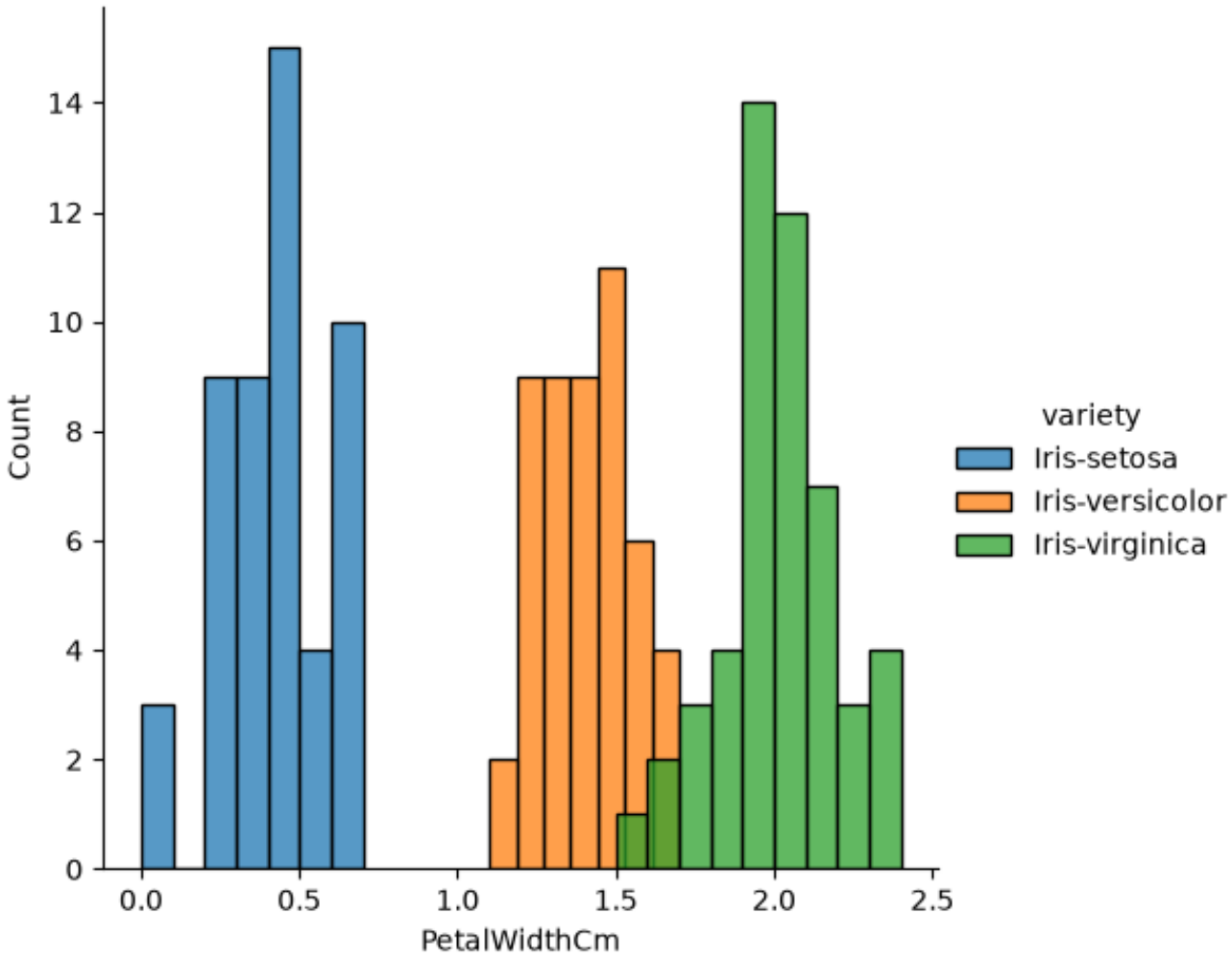
	SepalWidthCm	PetalLengthCm
0	3.5	0.6
1	2.9	4.6
2	3.5	5.4
3	2.9	4.2
4	3.5	1.9

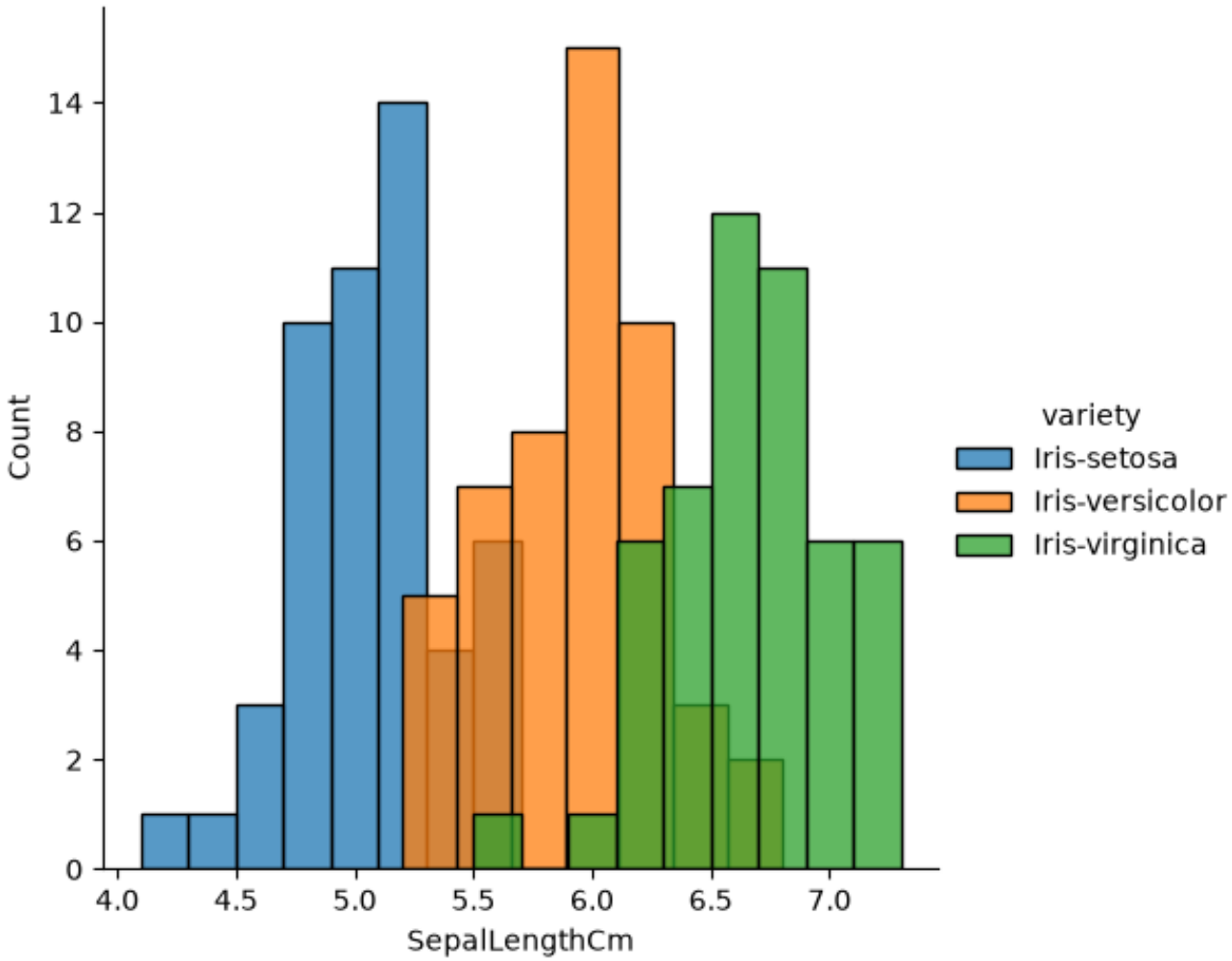

variety
Iris-setosa
Iris-versicolor
Iris-virginica
SepalWidthCm
SepalLengthCm
4.00
3.75
3.50
3.25
3.00
2.75
2.50
2.25
2.00
4.0
4.5
5.0
5.5
6.0
6.5
7.0

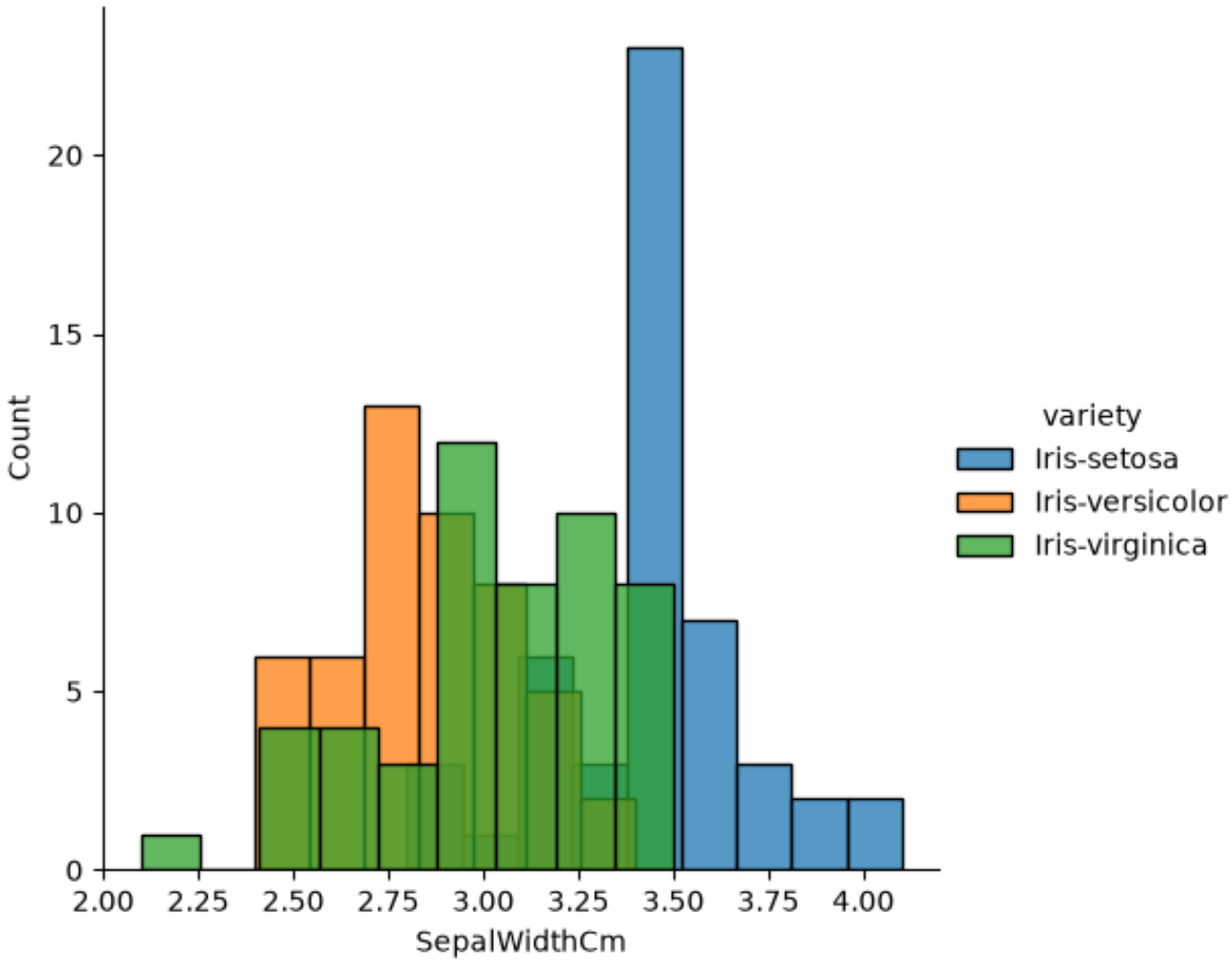

variety
Iris-setosa
Iris-versicolor
Iris-virginica
PetalWidthCm
PetalLengthCm
2.5
2.0
1.5
1.0
0.5
0.0
1
2
3
4
5
6


Id
SepalLengthCm
SepalWidthCm
PetalLengthCm
PetalWidthCm
variety
Iris-setosa
Iris-versicolor
Iris-virginica


16
14
12
10
8
6
4
2
0
Count
1
2
3
4
5
6
PetalLengthCm
variety
Iris-setosa
Iris-versicolor
Iris-virginica


Count
PetalWidthCm
0
2
4
6
8
10
12
14
0.0
0.5
1.0
1.5
2.0
2.5
variety
Iris-setosa
Iris-versicolor
Iris-virginica


Count
14
12
10
8
6
4
2
0
4.0
4.5
5.0
5.5
6.0
6.5
7.0
SepalLengthCm
variety
Iris-setosa
Iris-versicolor
Iris-virginica


Count
20
15
10
5
0
2.00
2.25
2.50
2.75
3.00
3.25
3.50
3.75
4.00
SepalWidthCm
variety
Iris-setosa
Iris-versicolor
Iris-virginica

Experiment-10

Code and complete execution output

Code Cell 1

```
import numpy as np

array = np.random.randint(1, 100, 9)
print(array)
print(np.sqrt(array))
print(array.ndim)
new_array = array.reshape(3, 3)
print(new_array)
print(new_array.ndim)
print(new_array.ravel())
newm = new_array.reshape(3, 3)
print(newm)
print(newm[2, 1:3])
print(newm[1:2, 1:3])
print(new_array[0:3, 0:0])
print(new_array[0:2, 0:1])
print(new_array[0:3, 0:1])
print(new_array[1:3])
```

Output

```
[17 74 93 78 42  2  7 96 14]
[4.12310563 8.60232527 9.64365076 8.83176087 6.4807407  1.41421356
 2.64575131 9.79795897 3.74165739]
1
[[17 74 93]
 [78 42  2]
 [ 7 96 14]]
2
[17 74 93 78 42  2  7 96 14]
[[17 74 93]
 [78 42  2]
 [ 7 96 14]]
[96 14]
[[42  2]]
[]
[[17]
 [78]]
[[17]
 [78]
 [ 7]]
[[78 42  2]
 [ 7 96 14]]
```

Experiment-11

Code and complete execution output

Code Cell 1

```
import numpy as np
import pandas as pd
from pathlib import Path

# Create a small DataFrame from changed hardcoded employee values
employee_rows = [[11, "Mira", 54000], [12, "Arun", 62500]]
df = pd.DataFrame(employee_rows)
print(df)
df.columns = ["Empd", "Name", "Salary"]
print(df)
df.info()

# Load an alternate startup dataset and inspect it
startup_df = pd.read_csv(Path.cwd() / "data" / "startup_funding.csv")
startup_df.info()
print(startup_df.head())
print(startup_df.tail())

# Load an alternate employee dataset and explore pandas Series/statistics
employee_df = pd.read_csv(Path.cwd() / "data" / "employee_data.csv")
print(employee_df.head())
print(employee_df.tail())
employee_df.info()
print(employee_df["salary"])
print(type(employee_df["salary"]))
print("Mean:", employee_df["salary"].mean())
print("Median:", employee_df["salary"].median())
print("Mode:\n", employee_df["salary"].mode())
print("Variance:", employee_df["salary"].var())
print("Standard deviation:", employee_df["salary"].std())
print(employee_df.describe())
print(employee_df.describe(include="all"))
emp_columns = employee_df.columns
emp_array = employee_df.values
employee_frame = pd.DataFrame(emp_array, columns=emp_columns)
print(employee_frame)
```

Output

```
    0     1      2
0  11  Mira  54000
1  12  Arun  62500
   Empd  Name  Salary
0    11  Mira   54000
1    12  Arun   62500
<class 'pandas.DataFrame'>
RangeIndex: 2 entries, 0 to 1
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   Empd    2 non-null      int64
 1   Name    2 non-null      str
 2   Salary  2 non-null      int64
dtypes: int64(2), str(1)
memory usage: 180.0 bytes
<class 'pandas.DataFrame'>
RangeIndex: 50 entries, 0 to 49
Data columns (total 5 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   R&D Spend        50 non-null     float64
 1   Administration   50 non-null     float64
 2   Marketing Spend  50 non-null     float64
 3   State            50 non-null     str
 4   Profit           50 non-null     float64
dtypes: float64(4), str(1)
memory usage: 2.1 KB
   R&D Spend  Administration  Marketing Spend     State     Profit
0   23249.01        80940.81        256596.41  Colorado   48310.33
1    9502.16        76624.10        285795.66  Colorado   38771.23
2   94239.48       129294.80        250853.37  Colorado   91486.44
3  157306.30       120329.07        301450.10    Oregon  131344.54
4   84391.55       119377.80        405881.35   Arizona  108054.56
    R&D Spend  Administration  Marketing Spend     State    Profit
45   18362.19       103447.26          1318.94    Oregon  17606.44
46  104174.66        58217.82         98358.55  Colorado  68235.41
47   60755.62        84399.10        471454.34    Oregon  94892.35
48   42630.81       120465.75        367102.69    Oregon  72697.49
49   31200.53       130144.39        308203.98    Oregon  65172.93
   emp id     name   salary
0       1    MIRA K    4200
1       2    ARUN P    5600
2       3   LEENA R    7350
3       4     DEV S    4900
4       5  NIKHIL T    8100
   emp id     name   salary
2       3   LEENA R    7350
3       4     DEV S    4900
4       5  NIKHIL T    8100
5       6    TARA M    3800
6       7    OMAR V    6700
<class 'pandas.DataFrame'>
RangeIndex: 7 entries, 0 to 6
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   emp id  7 non-null      int64
 1   name    7 non-null      str
 2   salary  7 non-null      int64
dtypes: int64(2), str(1)
memory usage: 300.0 bytes
0    4200
1    5600
2    7350
3    4900
4    8100
5    3800
6    6700
Name: salary, dtype: int64
<class 'pandas.Series'>
Mean: 5807.142857142857
Median: 5600.0
Mode:
 0    3800
1    4200
2    4900
3    5600
4    6700
5    7350
6    8100
Name: salary, dtype: int64
```

```
Variance: 2652023.8095238092
Standard deviation: 1628.5035491284043
         emp id       salary
count  7.000000     7.000000
mean   4.000000  5807.142857
std    2.160247  1628.503549
min    1.000000  3800.000000
25%    2.500000  4550.000000
50%    4.000000  5600.000000
75%    5.500000  7025.000000
max    7.000000  8100.000000
          emp id   name        salary
count   7.000000       7     7.000000
unique       NaN       7          NaN
top          NaN  MIRA K          NaN
freq         NaN       1          NaN
mean    4.000000     NaN  5807.142857
std     2.160247     NaN  1628.503549
min     1.000000     NaN  3800.000000
25%     2.500000     NaN  4550.000000
50%     4.000000     NaN  5600.000000
75%     5.500000     NaN  7025.000000
max     7.000000     NaN  8100.000000
  emp id     name  salary
0      1    MIRA K   4200
1      2    ARUN P   5600
2      3   LEENA R   7350
3      4     DEV S   4900
4      5  NIKHIL T   8100
5      6    TARA M   3800
6      7    OMAR V   6700
```

Experiment-12

Code and complete execution output

Code Cell 1

```
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

# Generate a different random sample from the manual's example
array = np.random.randint(1, 100, 16)
print(array)
print(array.mean())
print(np.percentile(array, 25))
print(np.percentile(array, 50))
print(np.percentile(array, 75))
print(np.percentile(array, 100))

def outDetection(array):
    sorted(array)
    Q1, Q3 = np.percentile(array, [25, 75])
    IQR = Q3 - Q1
    lr = Q1 - (1.5 * IQR)
    ur = Q3 + (1.5 * IQR)
    return lr, ur

lr, ur = outDetection(array)
print(lr, ur)
sns.displot(array)
sns.distplot(array)
plt.show()
new_array = array[(array > lr) & (array < ur)]
print(new_array)
sns.displot(new_array)
plt.show()
lr1, ur1 = outDetection(new_array)
print(lr1, ur1)
final_array = new_array[(new_array > lr1) & (new_array < ur1)]
print(final_array)
sns.distplot(final_array)
plt.show()
```

Output

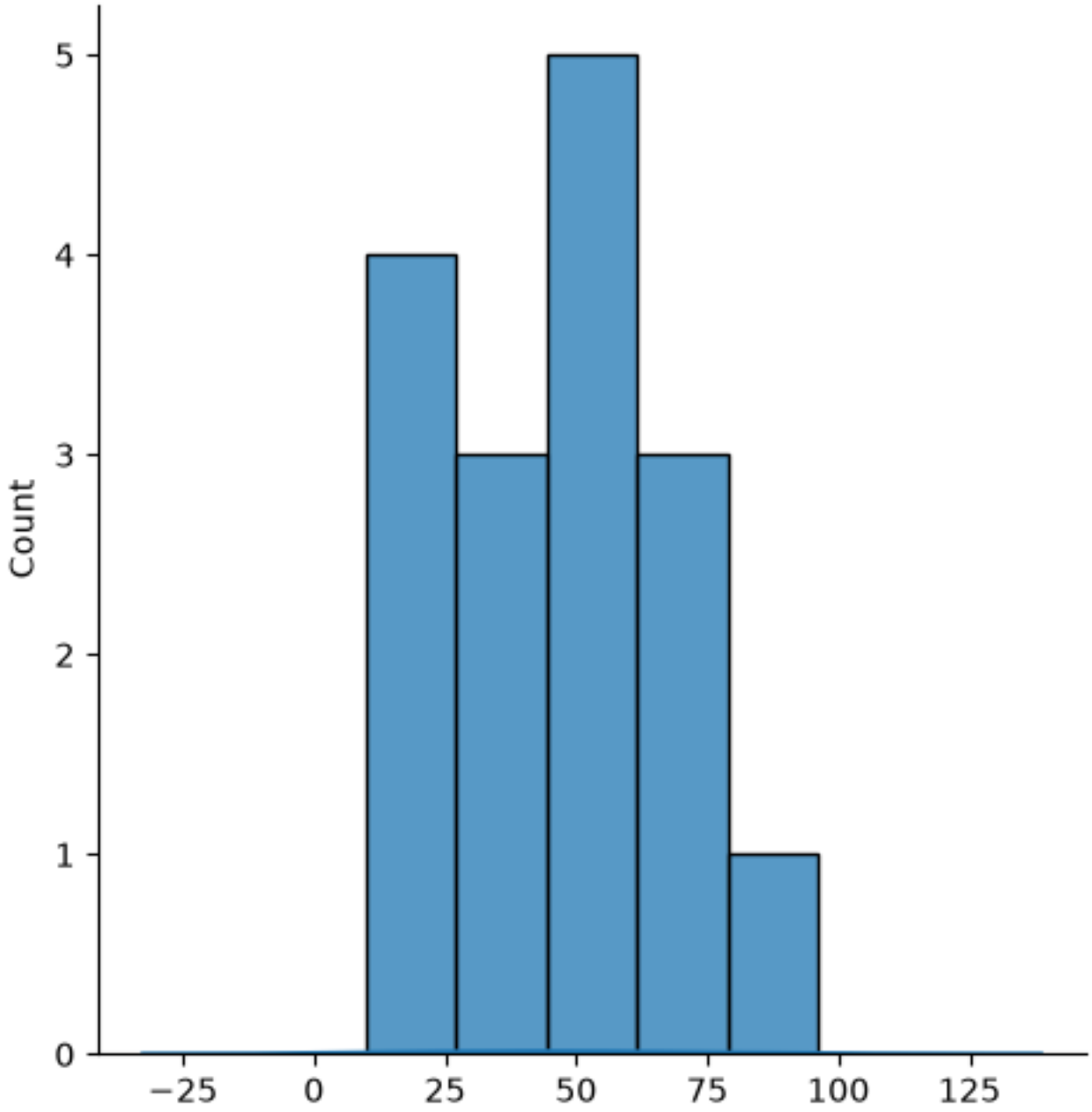
```
[38 10 33 61 46 22 35 96 66 77 46 49 17 75 54 13]
46.125
30.25
46.0
62.25
96.0
-17.75 110.25
C:\Users\Myself\AppData\Local\Temp\ipykernel_36852\1262253782.py:25: UserWarning:

`distplot` is a deprecated function and will be removed in seaborn v0.14.0.

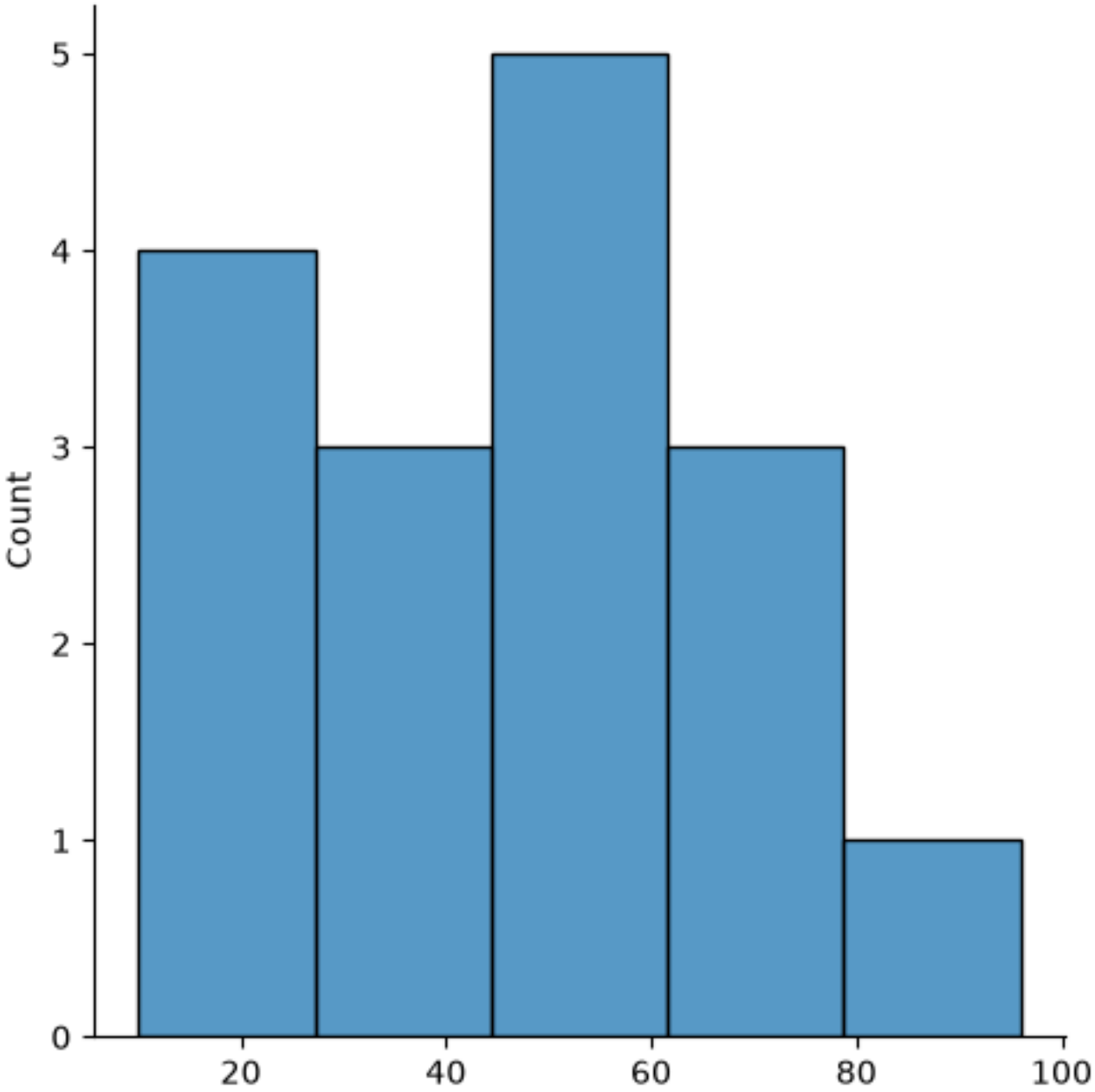
Please adapt your code to use either `displot` (a figure-level function with
similar flexibility) or `histplot` (an axes-level function for histograms).

For a guide to updating your code to use the new functions, please see
https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751

  sns.distplot(array)
```




```
[38 10 33 61 46 22 35 96 66 77 46 49 17 75 54 13]
```



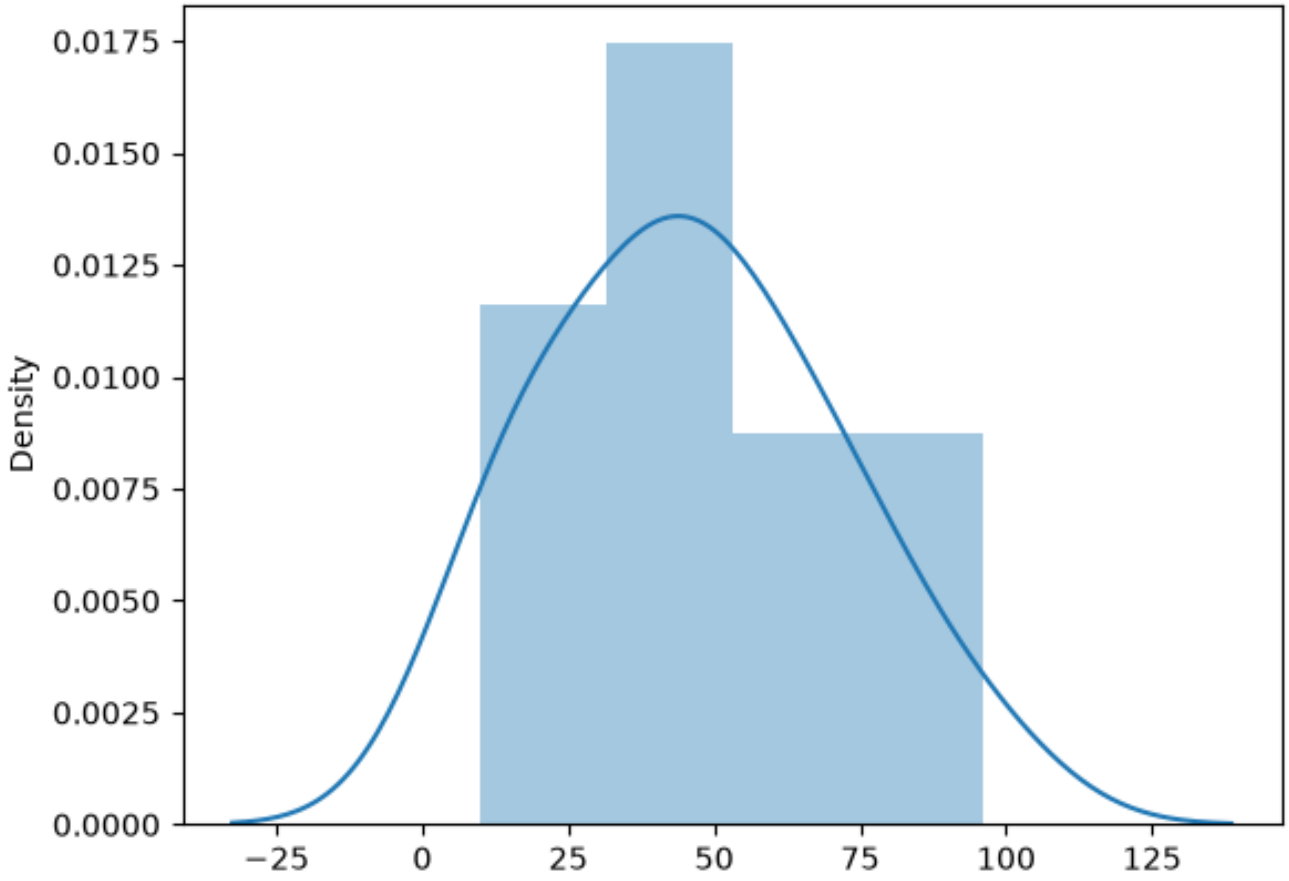

```
-17.75 110.25
[38 10 33 61 46 22 35 96 66 77 46 49 17 75 54 13]
C:\Users\Myself\AppData\Local\Temp\ipykernel_36852\1262253782.py:35: UserWarning:

`distplot` is a deprecated function and will be removed in seaborn v0.14.0.

Please adapt your code to use either `displot` (a figure-level function with
similar flexibility) or `histplot` (an axes-level function for histograms).

For a guide to updating your code to use the new functions, please see
https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751

  sns.distplot(final_array)
```


Density
0.0175
0.0150
0.0125
0.0100
0.0075
0.0050
0.0025
0.0000
−25
0
25
50
75
100
125

Experiment-13

Code and complete execution output

Code Cell 1

```
import numpy as np
import pandas as pd
from pathlib import Path

# Load the alternate hotel dataset
df = pd.read_csv(Path.cwd() / "data" / "hotel_dataset.csv")
print(df)

# Identify duplicate records and inspect the dataset
print(df.duplicated())
df.info()

# Remove duplicate records and reset the index
df.drop_duplicates(inplace=True)
print("Rows after removing duplicates:", len(df))
index = np.array(list(range(0, len(df))))
df.set_index(index, inplace=True)
print(index)
print(df)

# Remove the duplicate Age_Group column
if "Age_Group.1" in df.columns:
    df.drop(["Age_Group.1"], axis=1, inplace=True)
print(df)

# Mark invalid numeric values as missing
df.loc[df.CustomerID < 0, "CustomerID"] = np.nan
df.loc[df.Bill < 0, "Bill"] = np.nan
df.loc[df.EstimatedSalary < 0, "EstimatedSalary"] = np.nan
df.loc[(df["NoOfPax"] < 1) | (df["NoOfPax"] > 20), "NoOfPax"] = np.nan
print(df)

# Standardize inconsistent category labels
print("Age groups:", df.Age_Group.unique())
print("Hotels before correction:", df.Hotel.unique())
print("Food preferences before correction:", df.FoodPreference.unique())
df["Hotel"] = df["Hotel"].replace({"Ceder House": "Cedar House"})
df["FoodPreference"] = df["FoodPreference"].replace({
    "Plant based": "Plant-based", "plant-based": "Plant-based",
    "Vegetarian": "Plant-based", "omnivore": "Omnivore"
})
print("Hotels after correction:", df.Hotel.unique())
print("Food preferences after correction:", df.FoodPreference.unique())

# Impute numerical columns with mean or median, as appropriate
df["EstimatedSalary"] = df["EstimatedSalary"].fillna(round(df["EstimatedSalary"].mean()))
df["NoOfPax"] = df["NoOfPax"].fillna(round(df["NoOfPax"].median()))
df["Rating(1-5)"] = df["Rating(1-5)"].fillna(round(df["Rating(1-5)"].median()))
df["Bill"] = df["Bill"].fillna(round(df["Bill"].mean()))
print(df)
```

Output

```
    CustomerID Age_Group  Rating(1-5)        Hotel FoodPreference    Bill  \
0          101     18-24            4  Cedar House    Plant-based  1420.0
1          102     25-34            5   Pine Lodge       Omnivore  2380.0
2          103     35-44            6   Metro Stay    Plant based  1880.0
3          104     18-24           -1   Pine Lodge    plant-based  1560.0
4          105       45+            3  Cedar House     Vegetarian  1120.0
5          106       45+            3  Ceder House       omnivore  2050.0
6          107     35-44            4   Metro Stay     Vegetarian  1675.0
7          108     18-24            7   Pine Lodge    Plant-based  2990.0
8          109     25-34            2  Cedar House       Omnivore  3240.0
9          109     25-34            2  Cedar House       Omnivore  3240.0
10         110     35-44            5   Metro Stay    plant-based     NaN

    NoOfPax  EstimatedSalary Age_Group.1
0         2            52000       18-24
1         3            73500       25-34
2         2            61000       35-44
3         2            48000       18-24
4         2            83000         45+
5         2            91500         45+
6        -1            69000       35-44
7        -4           405000       18-24
8         3           -12000       25-34
9         3           -12000       25-34
10        4            97700       35-44
0     False
1     False
2     False
3     False
4     False
5     False
6     False
7     False
8     False
9      True
10    False
dtype: bool
<class 'pandas.DataFrame'>
RangeIndex: 11 entries, 0 to 10
Data columns (total 9 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   CustomerID       11 non-null     int64
 1   Age_Group        11 non-null     str
 2   Rating(1-5)      11 non-null     int64
 3   Hotel            11 non-null     str
 4   FoodPreference   11 non-null     str
 5   Bill             10 non-null     float64
 6   NoOfPax          11 non-null     int64
 7   EstimatedSalary  11 non-null     int64
 8   Age_Group.1      11 non-null     str
dtypes: float64(1), int64(4), str(4)
memory usage: 924.0 bytes
Rows after removing duplicates: 10
[0 1 2 3 4 5 6 7 8 9]
   CustomerID Age_Group  Rating(1-5)        Hotel FoodPreference    Bill  \
0         101     18-24            4  Cedar House    Plant-based  1420.0
1         102     25-34            5   Pine Lodge       Omnivore  2380.0
2         103     35-44            6   Metro Stay    Plant based  1880.0
3         104     18-24           -1   Pine Lodge    plant-based  1560.0
4         105       45+            3  Cedar House     Vegetarian  1120.0
5         106       45+            3  Ceder House       omnivore  2050.0
6         107     35-44            4   Metro Stay     Vegetarian  1675.0
7         108     18-24            7   Pine Lodge    Plant-based  2990.0
8         109     25-34            2  Cedar House       Omnivore  3240.0
9         110     35-44            5   Metro Stay    plant-based     NaN

   NoOfPax  EstimatedSalary Age_Group.1
0        2            52000       18-24
1        3            73500       25-34
2        2            61000       35-44
3        2            48000       18-24
4        2            83000         45+
5        2            91500         45+
6       -1            69000       35-44
7       -4           405000       18-24
8        3           -12000       25-34
9        4            97700       35-44
   CustomerID Age_Group  Rating(1-5)        Hotel FoodPreference    Bill  \
0         101     18-24            4  Cedar House    Plant-based  1420.0
1         102     25-34            5   Pine Lodge       Omnivore  2380.0
2         103     35-44            6   Metro Stay    Plant based  1880.0
```

```
3         104     18-24           -1   Pine Lodge    plant-based  1560.0
4         105       45+            3  Cedar House     Vegetarian  1120.0
5         106       45+            3  Ceder House       omnivore  2050.0
6         107     35-44            4   Metro Stay     Vegetarian  1675.0
7         108     18-24            7   Pine Lodge    Plant-based  2990.0
8         109     25-34            2  Cedar House       Omnivore  3240.0
9         110     35-44            5   Metro Stay    plant-based     NaN

   NoOfPax  EstimatedSalary
0        2            52000
1        3            73500
2        2            61000
3        2            48000
4        2            83000
5        2            91500
6       -1            69000
7       -4           405000
8        3           -12000
9        4            97700
   CustomerID Age_Group  Rating(1-5)        Hotel FoodPreference    Bill  \
0       101.0     18-24            4  Cedar House    Plant-based  1420.0
1       102.0     25-34            5   Pine Lodge       Omnivore  2380.0
2       103.0     35-44            6   Metro Stay    Plant based  1880.0
3       104.0     18-24           -1   Pine Lodge    plant-based  1560.0
4       105.0       45+            3  Cedar House     Vegetarian  1120.0
5       106.0       45+            3  Ceder House       omnivore  2050.0
6       107.0     35-44            4   Metro Stay     Vegetarian  1675.0
7       108.0     18-24            7   Pine Lodge    Plant-based  2990.0
8       109.0     25-34            2  Cedar House       Omnivore  3240.0
9       110.0     35-44            5   Metro Stay    plant-based     NaN

   NoOfPax  EstimatedSalary
0      2.0          52000.0
1      3.0          73500.0
2      2.0          61000.0
3      2.0          48000.0
4      2.0          83000.0
5      2.0          91500.0
6      NaN          69000.0
7      NaN         405000.0
8      3.0              NaN
9      4.0          97700.0
Age groups: <StringArray>
['18-24', '25-34', '35-44', '45+']
Length: 4, dtype: str
Hotels before correction: <StringArray>
['Cedar House', 'Pine Lodge', 'Metro Stay', 'Ceder House']
Length: 4, dtype: str
Food preferences before correction: <StringArray>
['Plant-based',    'Omnivore', 'Plant based', 'plant-based',  'Vegetarian',
    'omnivore']
Length: 6, dtype: str
Hotels after correction: <StringArray>
['Cedar House', 'Pine Lodge', 'Metro Stay']
Length: 3, dtype: str
Food preferences after correction: <StringArray>
['Plant-based', 'Omnivore']
Length: 2, dtype: str
   CustomerID Age_Group  Rating(1-5)        Hotel FoodPreference    Bill  \
0       101.0     18-24            4  Cedar House    Plant-based  1420.0
1       102.0     25-34            5   Pine Lodge       Omnivore  2380.0
2       103.0     35-44            6   Metro Stay    Plant-based  1880.0
3       104.0     18-24           -1   Pine Lodge    Plant-based  1560.0
4       105.0       45+            3  Cedar House    Plant-based  1120.0
5       106.0       45+            3  Cedar House       Omnivore  2050.0
6       107.0     35-44            4   Metro Stay    Plant-based  1675.0
7       108.0     18-24            7   Pine Lodge    Plant-based  2990.0
8       109.0     25-34            2  Cedar House       Omnivore  3240.0
9       110.0     35-44            5   Metro Stay    Plant-based  2035.0

   NoOfPax  EstimatedSalary
0      2.0          52000.0
1      3.0          73500.0
2      2.0          61000.0
3      2.0          48000.0
4      2.0          83000.0
5      2.0          91500.0
6      2.0          69000.0
7      2.0         405000.0
8      3.0         108967.0
9      4.0          97700.0
C:\Users\Myself\AppData\Local\Temp\ipykernel_3520\23492568.py:14: Pandas4Warning: The inplace keyword in
    DataFrame.drop_duplicates is deprecated and will be removed in a future version (PDEP-8).
See https://pandas.pydata.org/docs/dev/whatsnew/v3.1.0.html#deprecation-inplace for more details.
```

```
  df.drop_duplicates(inplace=True)
C:\Users\Myself\AppData\Local\Temp\ipykernel_3520\23492568.py:17: Pandas4Warning: The inplace keyword in
    DataFrame.set_index is deprecated and will be removed in a future version (PDEP-8).
See https://pandas.pydata.org/docs/dev/whatsnew/v3.1.0.html#deprecation-inplace for more details.
  df.set_index(index, inplace=True)
C:\Users\Myself\AppData\Local\Temp\ipykernel_3520\23492568.py:23: Pandas4Warning: The inplace keyword in
    DataFrame.drop is deprecated and will be removed in a future version (PDEP-8).
See https://pandas.pydata.org/docs/dev/whatsnew/v3.1.0.html#deprecation-inplace for more details.
  df.drop(["Age_Group.1"], axis=1, inplace=True)
```

Experiment-14

Code and complete execution output

Code Cell 1

```
import numpy as np
import pandas as pd
from pathlib import Path
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder, StandardScaler, MinMaxScaler

# Load the alternate preprocessing dataset
df = pd.read_csv(Path.cwd() / "data" / "pre-process_datasample.csv")
print(df)
print(df.head())
df["Country"] = df["Country"].fillna(df["Country"].mode()[0])
features = df.iloc[:, :-1].values
label = df.iloc[:, -1].values
print("Labels:\n", label)

age_imputer = SimpleImputer(strategy="mean", missing_values=np.nan)
salary_imputer = SimpleImputer(strategy="mean", missing_values=np.nan)
features[:, [1]] = age_imputer.fit_transform(features[:, [1]])
features[:, [2]] = salary_imputer.fit_transform(features[:, [2]])
print("Imputed features:\n", features)

encoder = OneHotEncoder(sparse_output=False)
Country = encoder.fit_transform(features[:, [0]])
print("One-hot encoded countries:\n", Country)
final_set = np.concatenate((Country, features[:, [1, 2]]), axis=1)
print("Combined features:\n", final_set)

standard_scaler = StandardScaler()
standard_scaler.fit(final_set)
feat_standard_scaler = standard_scaler.transform(final_set)
print("Standard-scaled features:\n", feat_standard_scaler)

minmax_scaler = MinMaxScaler(feature_range=(0, 1))
minmax_scaler.fit(final_set)
feat_minmax_scaler = minmax_scaler.transform(final_set)
print("Min-max-scaled features:\n", feat_minmax_scaler)
```

Output

```
  Country   Age   Salary Purchased
0   Japan  29.0  51000.0       Yes
1   Chile  41.0  68000.0        No
2  Norway  36.0      NaN       Yes
3   Chile   NaN  59000.0        No
4   Japan  52.0  93000.0       Yes
5  Norway  33.0  62000.0        No
6   Japan  47.0  84000.0       Yes
7   Chile  25.0  47000.0        No
8     NaN  39.0  73000.0       Yes
9   Japan  44.0  79000.0        No
  Country   Age   Salary Purchased
0   Japan  29.0  51000.0       Yes
1   Chile  41.0  68000.0        No
2  Norway  36.0      NaN       Yes
3   Chile   NaN  59000.0        No
4   Japan  52.0  93000.0       Yes
Labels:
 <StringArray>
['Yes', 'No', 'Yes', 'No', 'Yes', 'No', 'Yes', 'No', 'Yes', 'No']
Length: 10, dtype: str
Imputed features:
 [['Japan' 29.0 51000.0]
 ['Chile' 41.0 68000.0]
 ['Norway' 36.0 68444.44444444444]
 ['Chile' 38.44444444444444 59000.0]
 ['Japan' 52.0 93000.0]
 ['Norway' 33.0 62000.0]
 ['Japan' 47.0 84000.0]
 ['Chile' 25.0 47000.0]
 ['Japan' 39.0 73000.0]
 ['Japan' 44.0 79000.0]]
One-hot encoded countries:
 [[0. 1. 0.]
 [1. 0. 0.]
 [0. 0. 1.]
 [1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]
 [0. 1. 0.]
 [1. 0. 0.]
 [0. 1. 0.]
 [0. 1. 0.]]
Combined features:
 [[0.0 1.0 0.0 29.0 51000.0]
 [1.0 0.0 0.0 41.0 68000.0]
 [0.0 0.0 1.0 36.0 68444.44444444444]
 [1.0 0.0 0.0 38.44444444444444 59000.0]
 [0.0 1.0 0.0 52.0 93000.0]
 [0.0 0.0 1.0 33.0 62000.0]
 [0.0 1.0 0.0 47.0 84000.0]
 [1.0 0.0 0.0 25.0 47000.0]
 [0.0 1.0 0.0 39.0 73000.0]
 [0.0 1.0 0.0 44.0 79000.0]]
Standard-scaled features:
 [[-6.54653671e-01  1.00000000e+00 -5.00000000e-01 -1.21904681e+00
  -1.27490682e+00]
 [ 1.52752523e+00 -1.00000000e+00 -5.00000000e-01  3.29859724e-01
  -3.24817025e-02]
 [-6.54653671e-01 -1.00000000e+00  2.00000000e+00 -3.15517997e-01
  -1.06350971e-15]
 [ 1.52752523e+00 -1.00000000e+00 -5.00000000e-01  0.00000000e+00
  -6.90236178e-01]
 [-6.54653671e-01  1.00000000e+00 -5.00000000e-01  1.74969071e+00
   1.79461406e+00]
 [-6.54653671e-01 -1.00000000e+00  2.00000000e+00 -7.02744630e-01
  -4.70984686e-01]
 [-6.54653671e-01  1.00000000e+00 -5.00000000e-01  1.10431299e+00
   1.13685959e+00]
 [ 1.52752523e+00 -1.00000000e+00 -5.00000000e-01 -1.73534898e+00
  -1.56724214e+00]
 [-6.54653671e-01  1.00000000e+00 -5.00000000e-01  7.17086357e-02
   3.32937450e-01]
 [-6.54653671e-01  1.00000000e+00 -5.00000000e-01  7.17086357e-01
   7.71440434e-01]]
Min-max-scaled features:
 [[0.         1.         0.         0.14814815 0.08695652]
 [1.         0.         0.         0.59259259 0.45652174]
 [0.         0.         1.         0.40740741 0.46618357]
 [1.         0.         0.         0.49794239 0.26086957]
 [0.         1.         0.         1.         1.        ]
 [0.         0.         1.         0.2962963  0.32608696]
```

```
 [0.         1.         0.         0.81481481 0.80434783]
 [1.         0.         0.         0.         0.        ]
 [0.         1.         0.         0.51851852 0.56521739]
 [0.         1.         0.         0.7037037  0.69565217]]
```

Experiment-15

Code and complete execution output

Code Cell 1

```
import numpy as np
import pandas as pd
from pathlib import Path

# Load alternate data and inspect it
df = pd.read_csv(Path.cwd() / "data" / "pre-process_datasample.csv")
print(df)
df.info()

# Fill missing categorical and numeric values with appropriate statistics
df["Country"] = df["Country"].fillna(df["Country"].mode()[0])
df["Age"] = df["Age"].fillna(df["Age"].median())
df["Salary"] = df["Salary"].fillna(round(df["Salary"].mean()))
print(df)

# One-hot encode Country and combine with remaining columns
updated_dataset = pd.concat(
    [pd.get_dummies(df["Country"]), df.iloc[:, [1, 2, 3]]], axis=1
)
print(updated_dataset)
updated_dataset["Purchased"] = updated_dataset["Purchased"].replace({"No": 0, "Yes": 1})
print(updated_dataset)
```

Output

```
  Country   Age   Salary Purchased
0   Japan  29.0  51000.0       Yes
1   Chile  41.0  68000.0        No
2  Norway  36.0      NaN       Yes
3   Chile   NaN  59000.0        No
4   Japan  52.0  93000.0       Yes
5  Norway  33.0  62000.0        No
6   Japan  47.0  84000.0       Yes
7   Chile  25.0  47000.0        No
8     NaN  39.0  73000.0       Yes
9   Japan  44.0  79000.0        No
<class 'pandas.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 4 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   Country    9 non-null      str
 1   Age        9 non-null      float64
 2   Salary     9 non-null      float64
 3   Purchased  10 non-null     str
dtypes: float64(2), str(2)
memory usage: 452.0 bytes
  Country   Age   Salary Purchased
0   Japan  29.0  51000.0       Yes
1   Chile  41.0  68000.0        No
2  Norway  36.0  68444.0       Yes
3   Chile  39.0  59000.0        No
4   Japan  52.0  93000.0       Yes
5  Norway  33.0  62000.0        No
6   Japan  47.0  84000.0       Yes
7   Chile  25.0  47000.0        No
8   Japan  39.0  73000.0       Yes
9   Japan  44.0  79000.0        No
   Chile  Japan  Norway   Age   Salary Purchased
0  False   True   False  29.0  51000.0       Yes
1   True  False   False  41.0  68000.0        No
2  False  False    True  36.0  68444.0       Yes
3   True  False   False  39.0  59000.0        No
4  False   True   False  52.0  93000.0       Yes
5  False  False    True  33.0  62000.0        No
6  False   True   False  47.0  84000.0       Yes
7   True  False   False  25.0  47000.0        No
8  False   True   False  39.0  73000.0       Yes
9  False   True   False  44.0  79000.0        No
   Chile  Japan  Norway   Age   Salary Purchased
0  False   True   False  29.0  51000.0         1
1   True  False   False  41.0  68000.0         0
2  False  False    True  36.0  68444.0         1
3   True  False   False  39.0  59000.0         0
4  False   True   False  52.0  93000.0         1
5  False  False    True  33.0  62000.0         0
6  False   True   False  47.0  84000.0         1
7   True  False   False  25.0  47000.0         0
8  False   True   False  39.0  73000.0         1
9  False   True   False  44.0  79000.0         0
```

Experiment-16

Code and complete execution output

Code Cell 1

```
import seaborn as sns
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from pathlib import Path

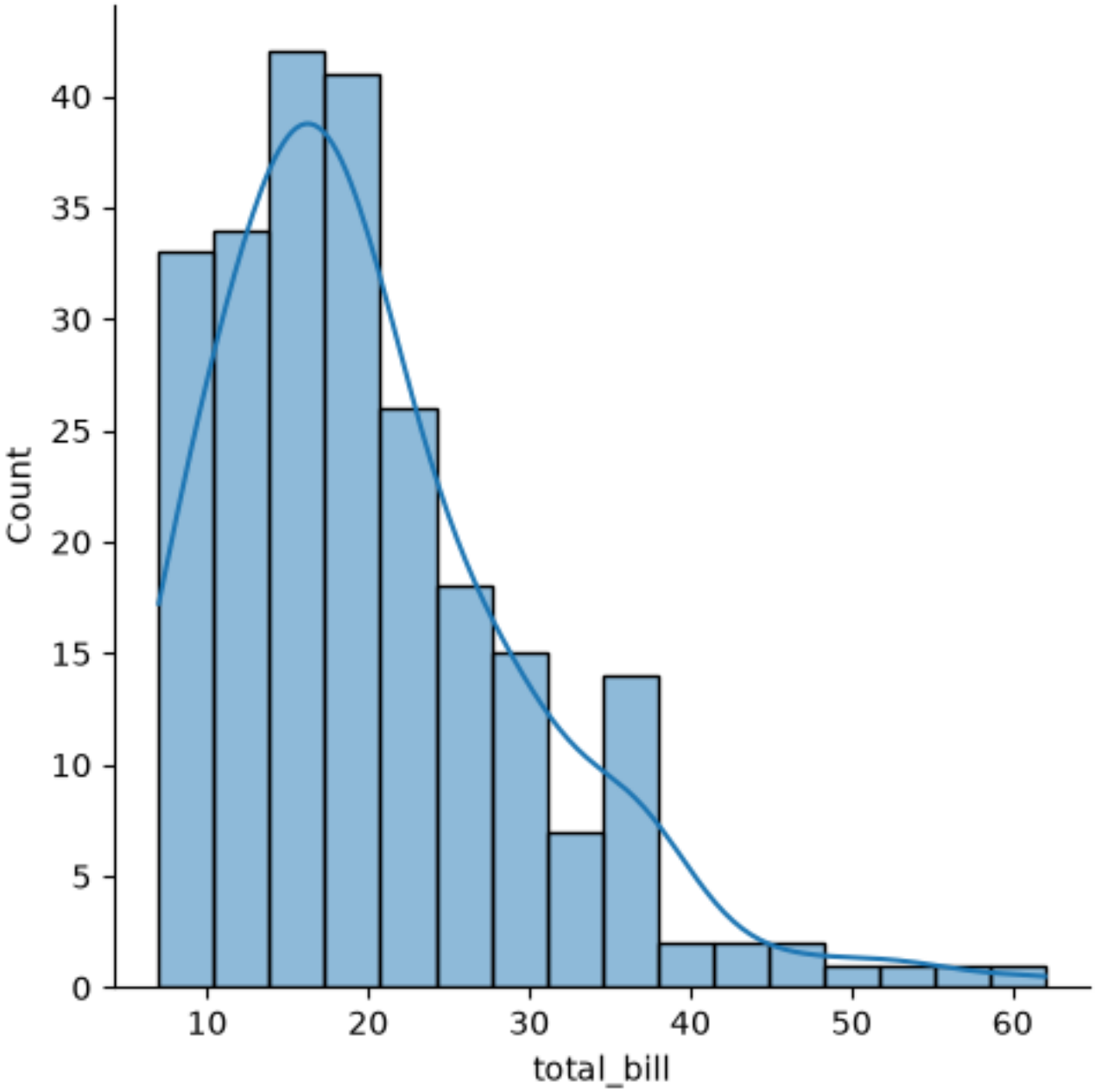
# Alternate local data matching the manual's tips dataset columns
tips = pd.read_csv(Path.cwd() / "data" / "restaurant_tips.csv")
print(tips.head())

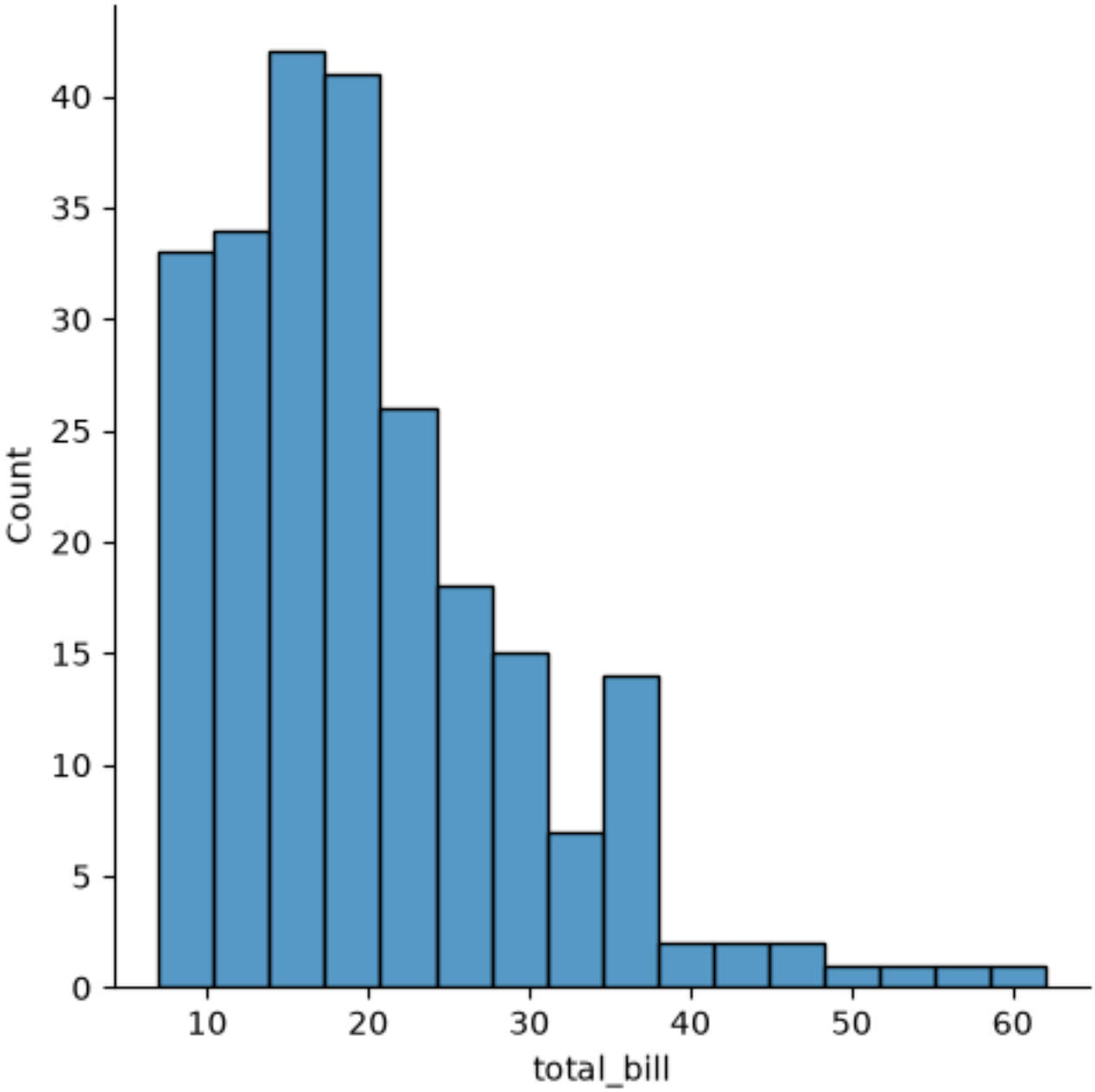
sns.displot(x=tips.total_bill, kde=True)
plt.show()
sns.displot(x=tips.total_bill, kde=False)
plt.show()
sns.jointplot(x=tips.tip, y=tips.total_bill)
plt.show()
sns.jointplot(x=tips.tip, y=tips.total_bill, kind="reg")
plt.show()
sns.jointplot(x=tips.tip, y=tips.total_bill, kind="hex")
plt.show()
sns.pairplot(tips)
plt.show()
print(tips.time.value_counts())
sns.pairplot(tips, hue="time")
plt.show()
sns.pairplot(tips, hue="day")
plt.show()
sns.heatmap(tips.corr(numeric_only=True), annot=True)
plt.show()

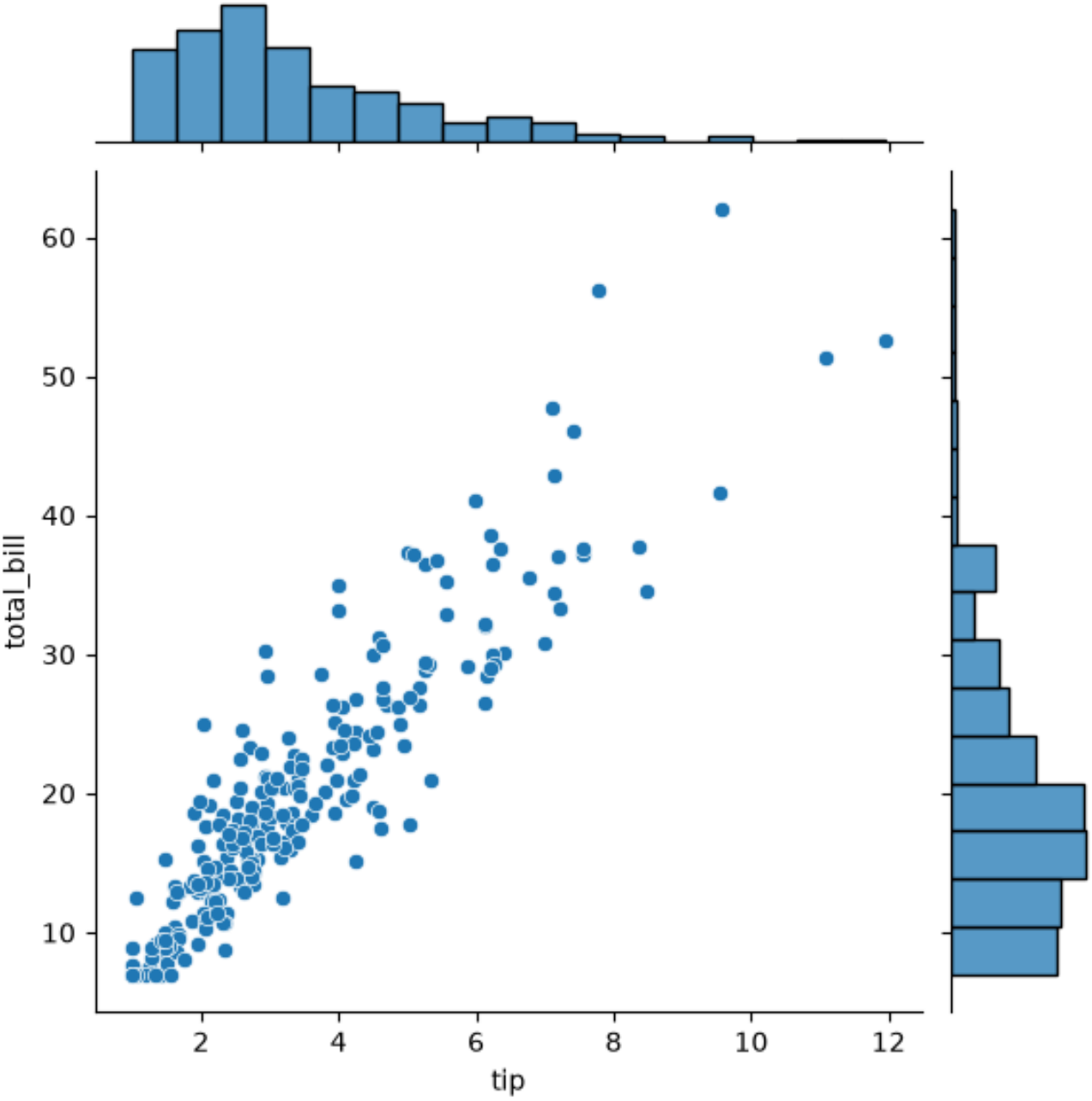
for column in ["total_bill", "tip"]:
    sns.boxplot(y=tips[column])
    plt.show()
sns.countplot(x=tips.day)
plt.show()
sns.countplot(x=tips.sex)
plt.show()
tips.sex.value_counts().plot(kind="pie", autopct="%1.1f%%")
plt.show()
tips.sex.value_counts().plot(kind="bar")
plt.show()
sns.countplot(x=tips.loc[tips.time == "Dinner", "day"])
plt.show()
```

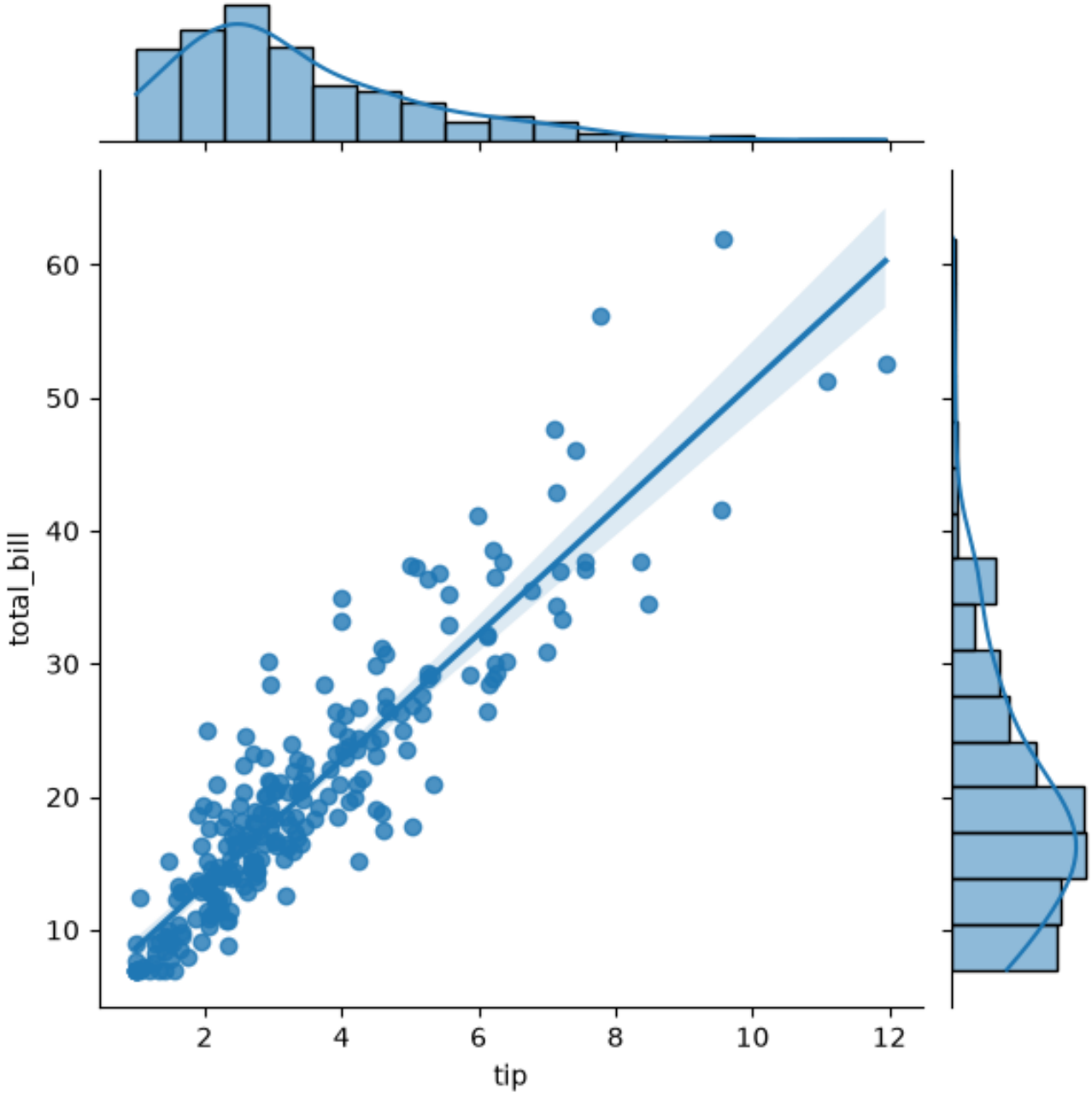
Output

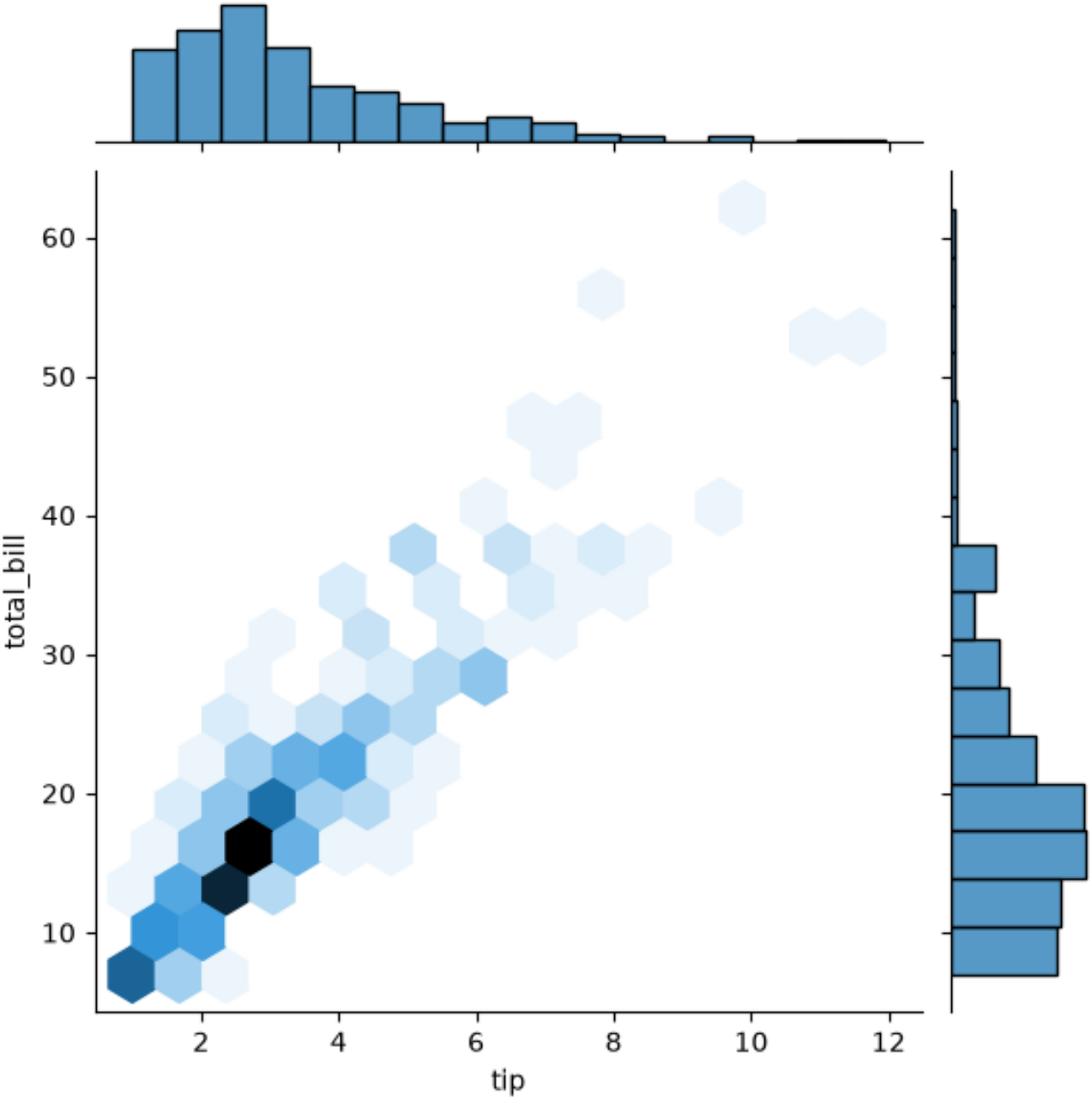
```
   total_bill   tip     sex smoker   day    time  size
0        7.19  1.12    Male     No   Sat  Dinner     1
1       18.23  2.54    Male    Yes   Sat  Dinner     2
2       15.31  2.83  Female     No  Thur  Dinner     2
3       34.90  4.00    Male     No   Sat  Dinner     1
4        7.00  1.00  Female    Yes   Sat  Dinner     4
```

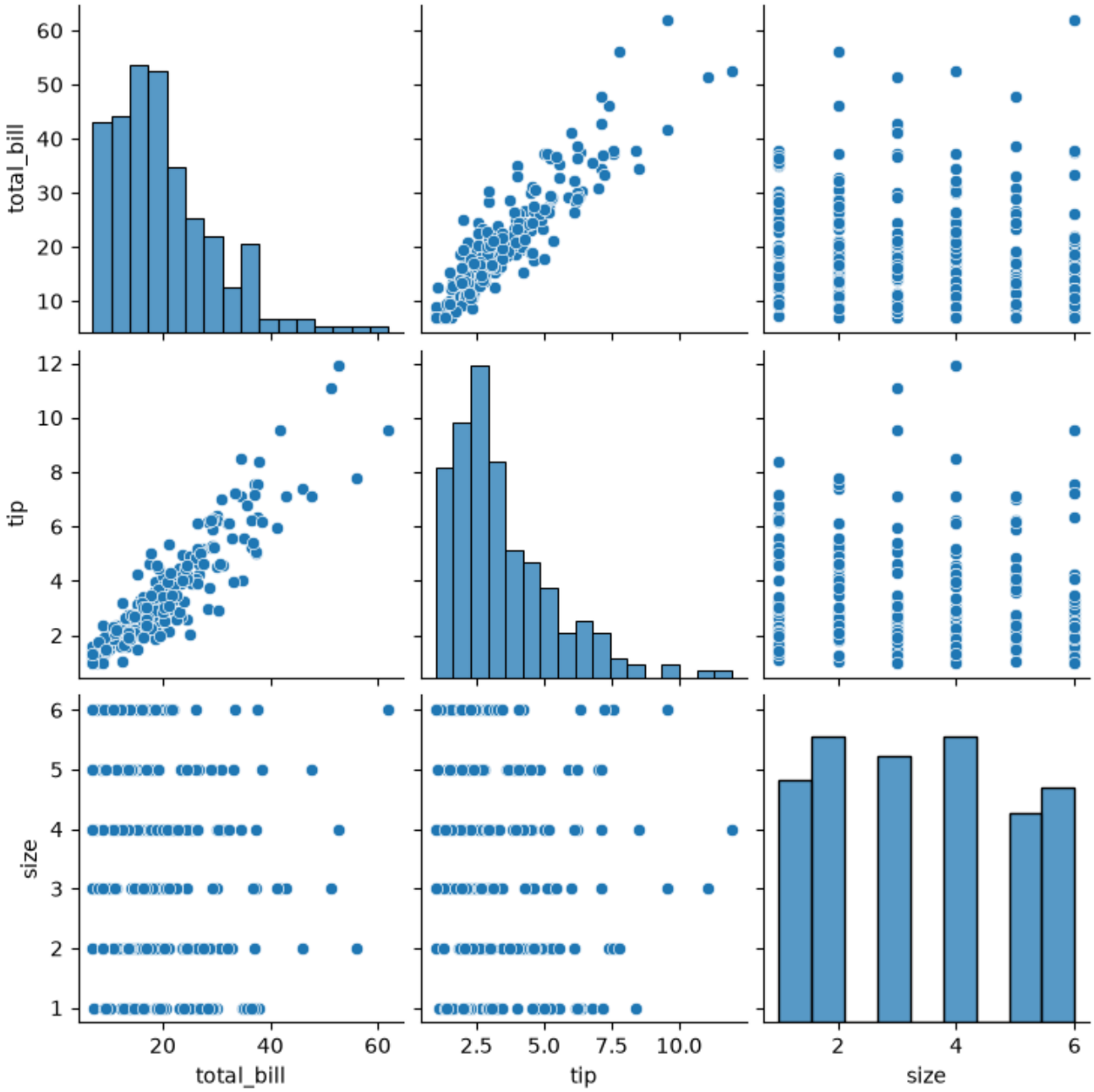

Count
0
5
10
15
20
25
30
35
40
10
20
30
40
50
60
total_bill


Count
0
5
10
15
20
25
30
35
40
10
20
30
40
50
60
total_bill

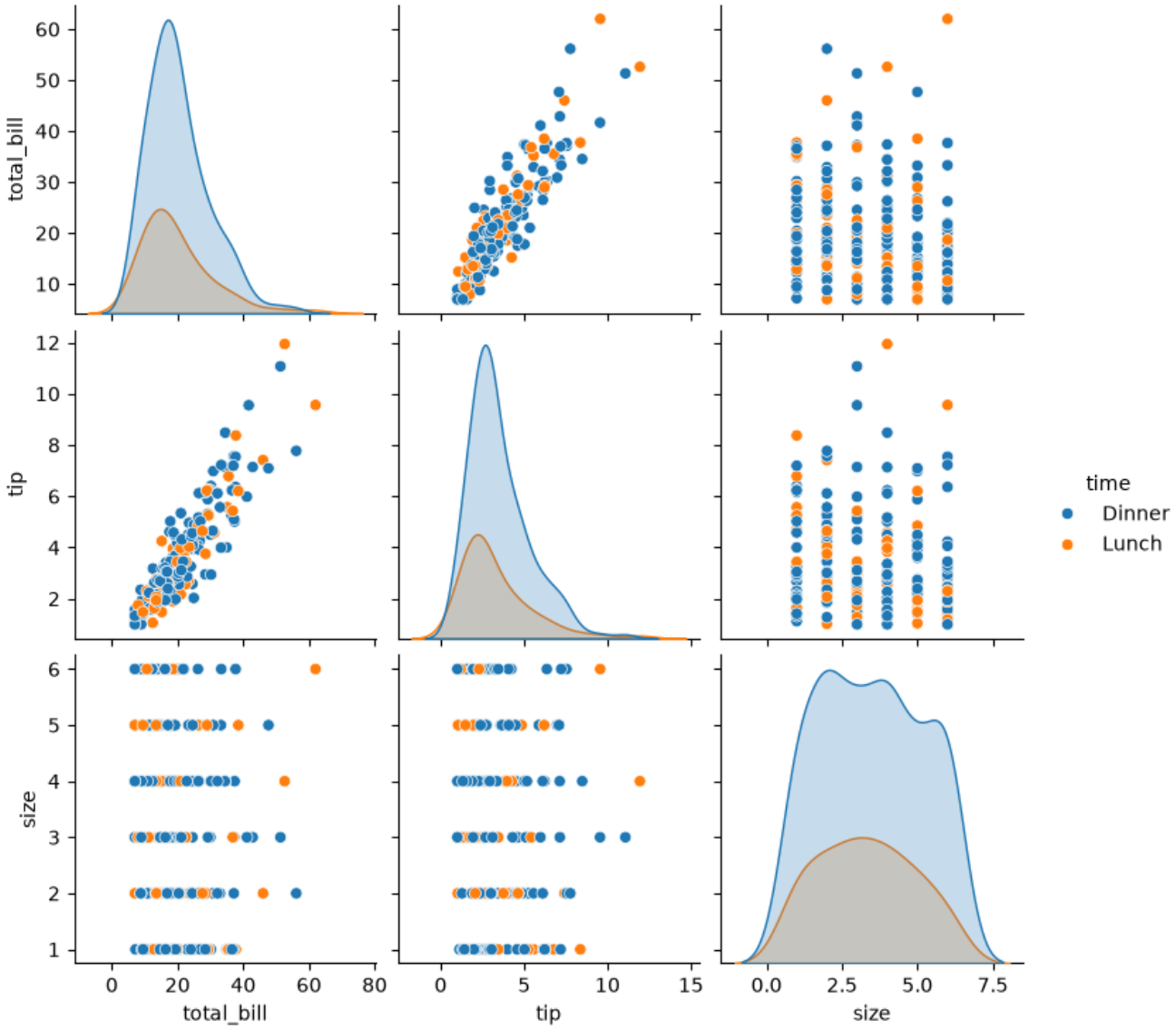

total_bill
tip
60
50
40
30
20
10
2
4
6
8
10
12

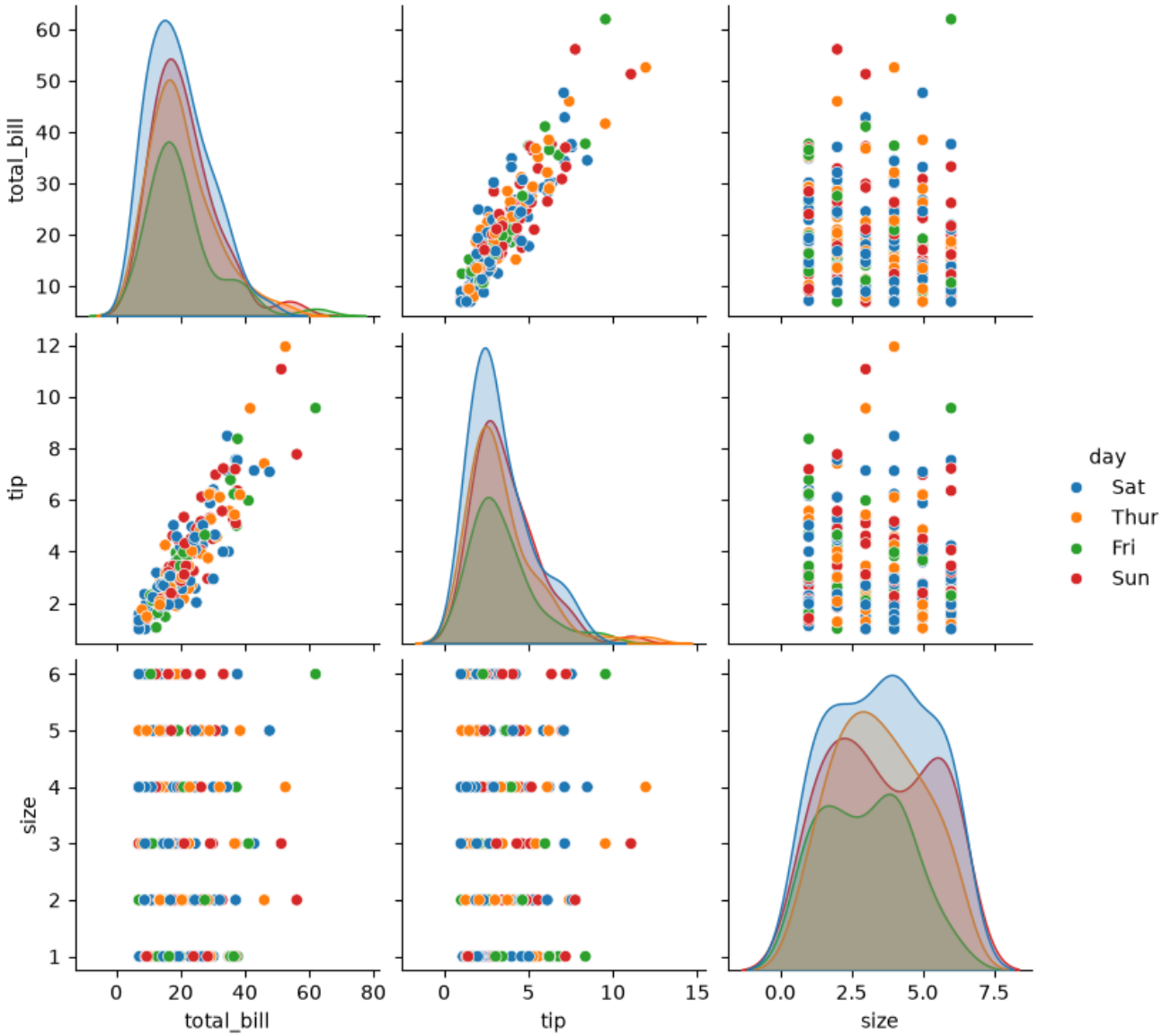

total_bill
tip
60
50
40
30
20
10
2
4
6
8
10
12

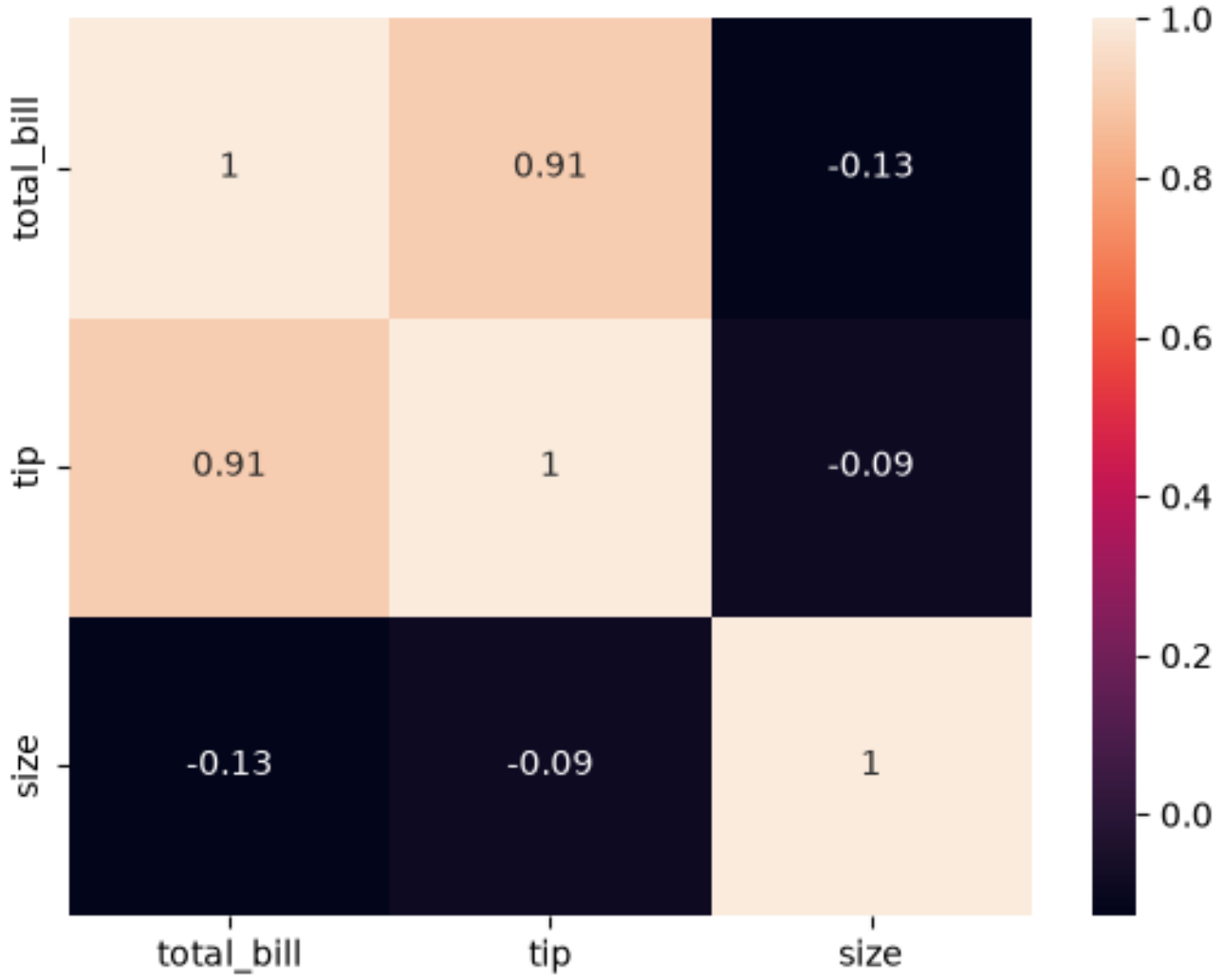

60
50
40
30
20
10
total_bill
2
4
6
8
10
12
tip

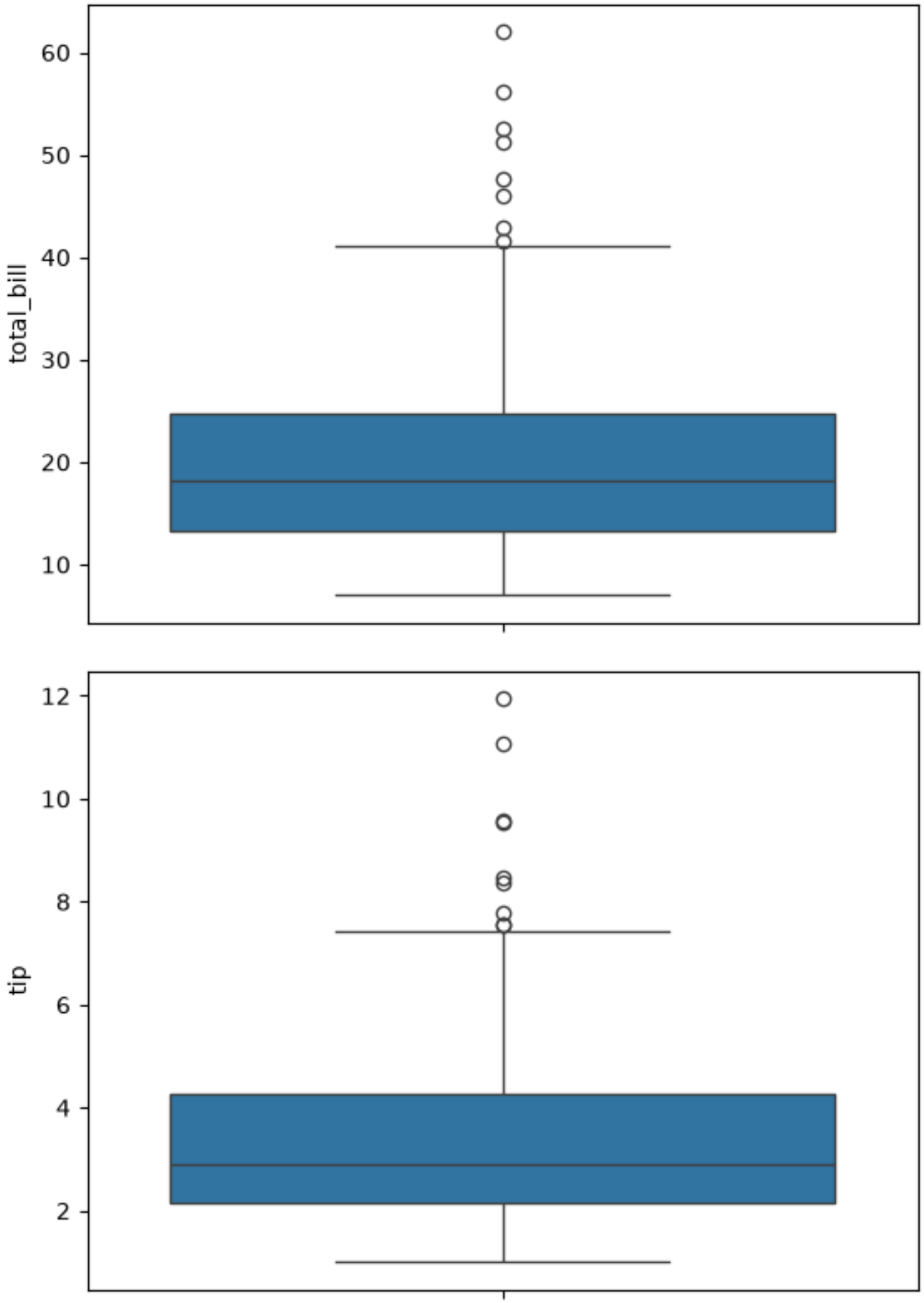


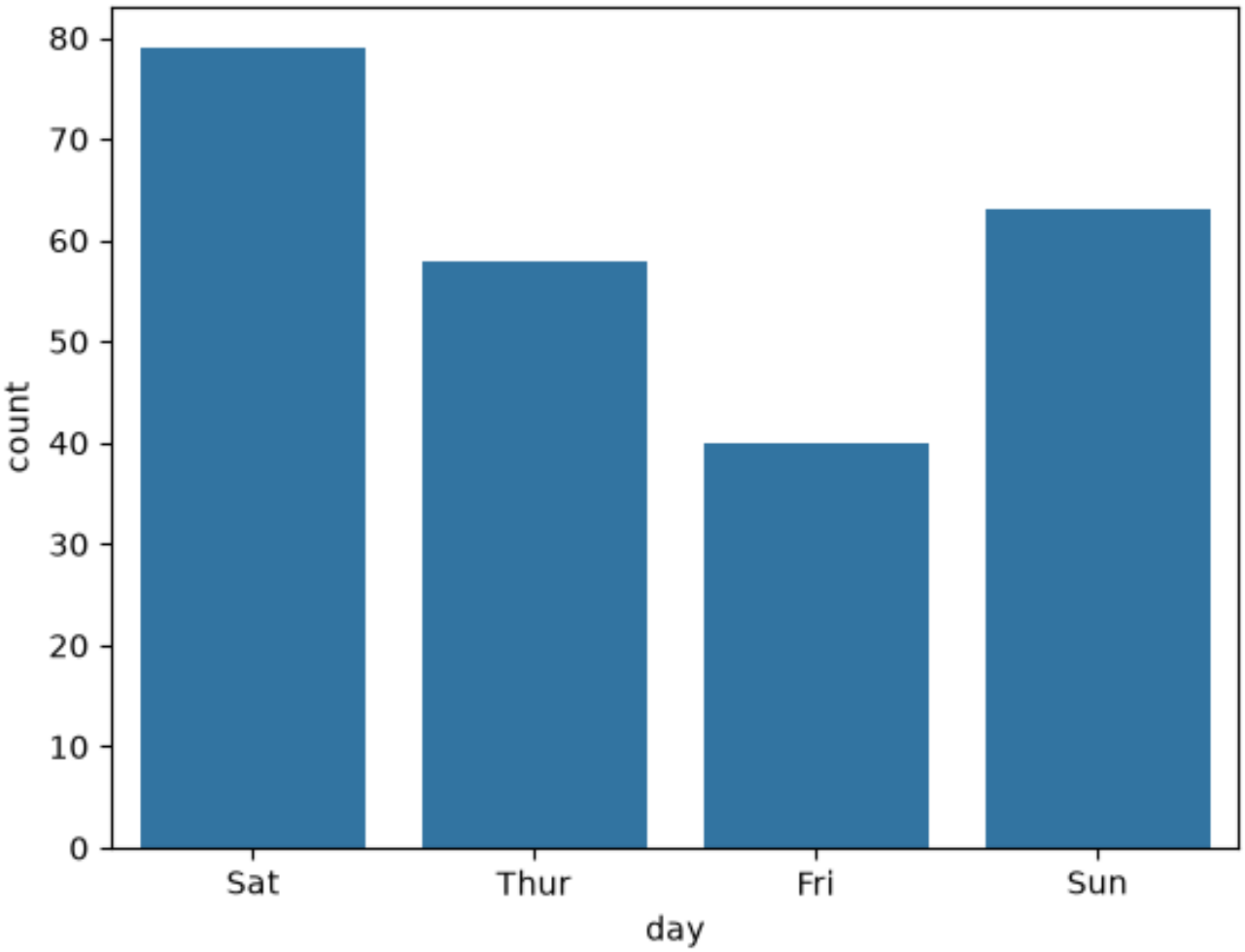

```
time
Dinner    173
Lunch      67
Name: count, dtype: int64
```

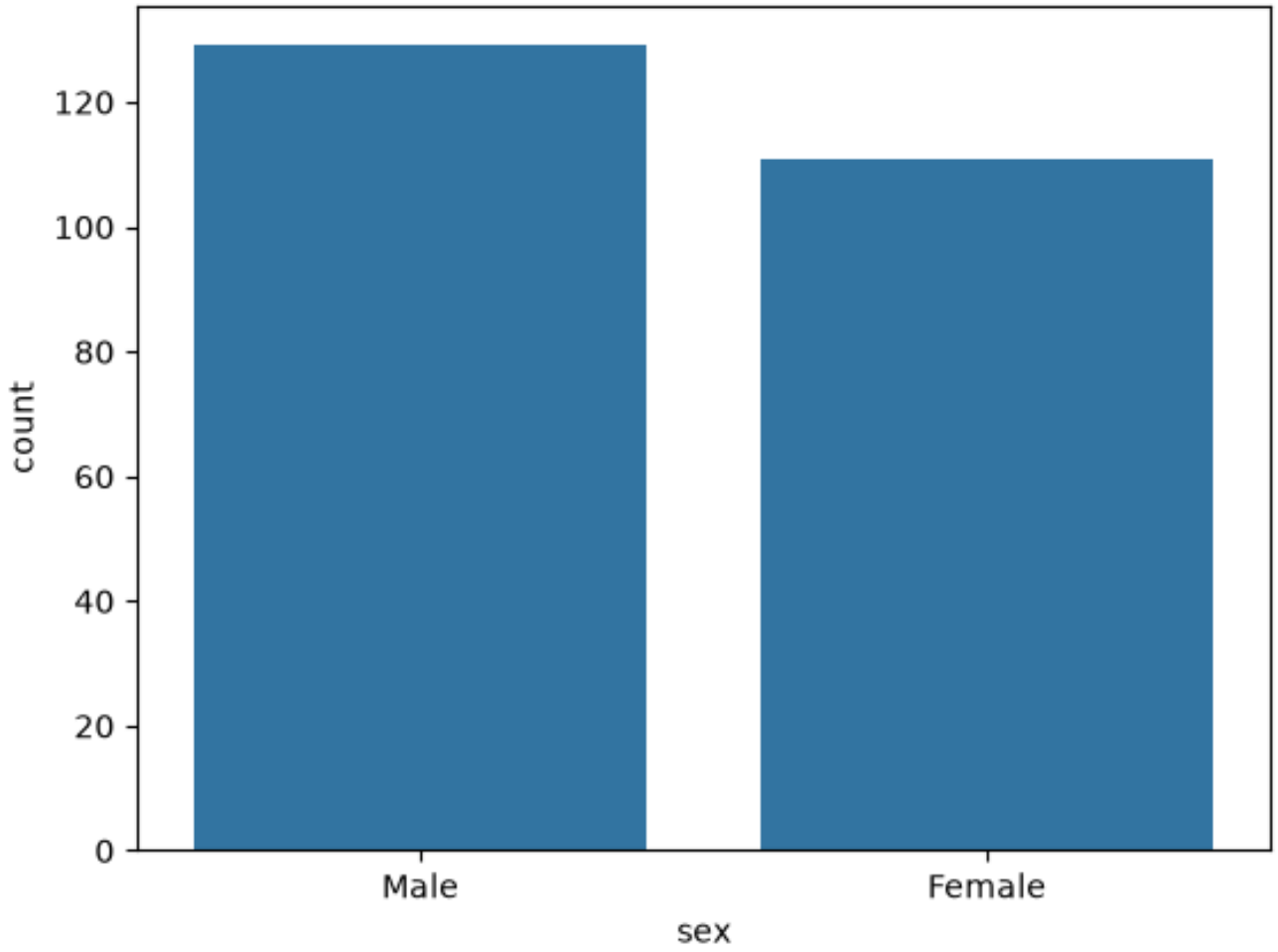

total_bill
tip
size
time
Dinner
Lunch
0
20
40
60
80
0
5
10
15
0.0
2.5
5.0
7.5
10
20
30
40
50
60
2
4
6
8
10
12
1
2
3
4
5
6

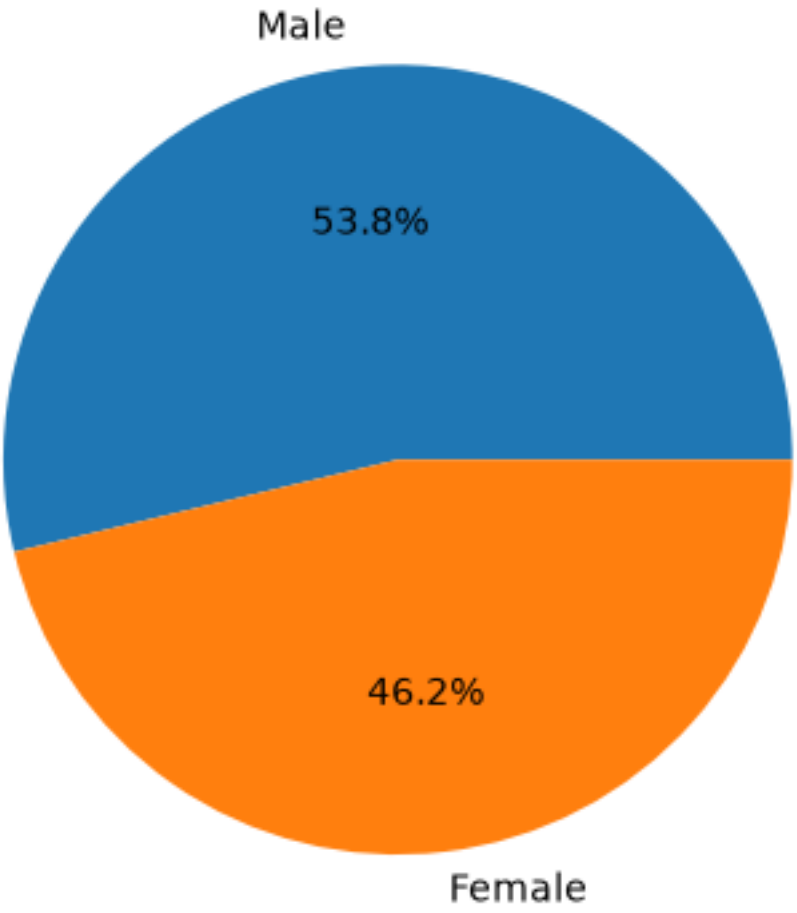

total_bill
tip
size
day
Sat
Thur
Fri
Sun

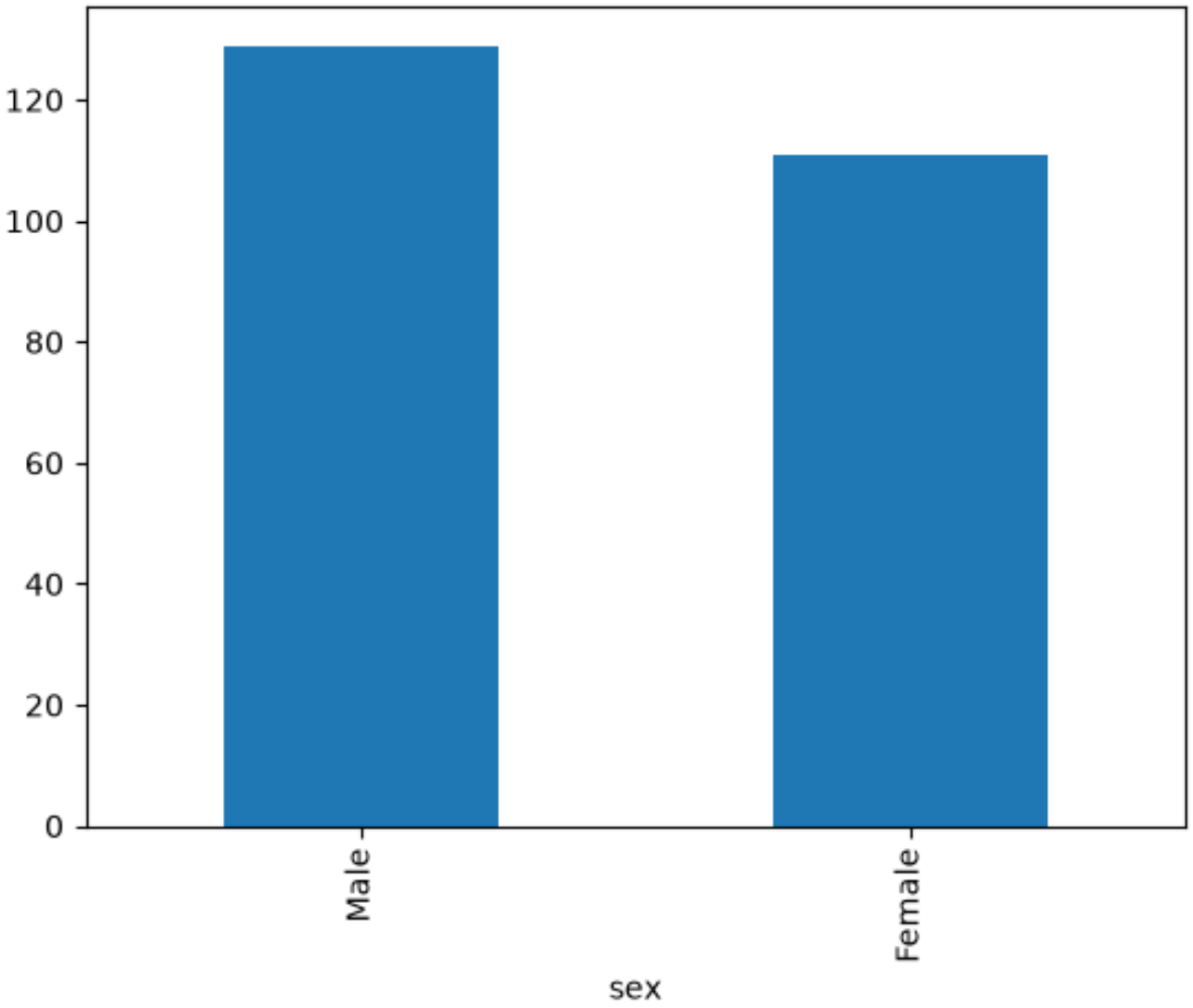

total_bill
tip
size
1
0.91
-0.13
0.91
1
-0.09
-0.13
-0.09
1
total_bill
tip
size
1.0
0.8
0.6
0.4
0.2
0.0

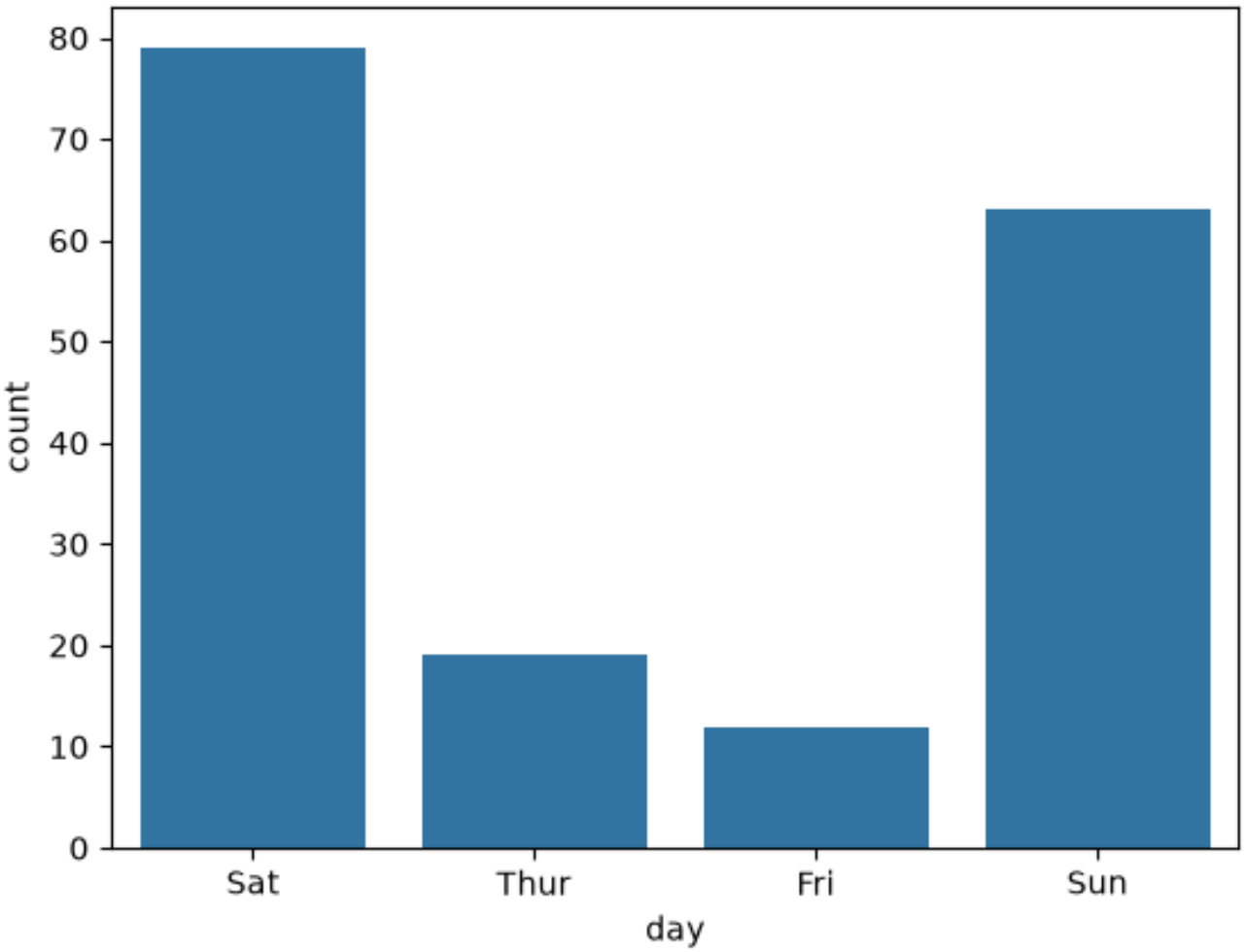

total_bill
60
50
40
30
20
10
tip
12
10
8
6
4
2


80
70
60
50
40
30
20
10
0
count
Sat
Thur
Fri
Sun
day


120
100
80
60
40
20
0
count
Male
Female
sex


Male
53.8%
46.2%
Female


120
100
80
60
40
20
0
Male
Female
sex


80
70
60
50
40
30
20
10
0
count
Sat
Thur
Fri
Sun
day

Experiment-17

Code and complete execution output

Code Cell 1

```
import numpy as np
import pandas as pd
import pickle
from pathlib import Path
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression

# Load the alternate salary dataset
df = pd.read_csv(Path.cwd() / "data" / "Salary_data.csv")
print(df)
df.info()
df.dropna(inplace=True)
df.info()
print(df.describe())

features = df.iloc[:, [0]].values
label = df.iloc[:, [1]].values
x_train, x_test, y_train, y_test = train_test_split(
    features, label, test_size=0.2, random_state=42
)
model = LinearRegression()
model.fit(x_train, y_train)
print("Training score:", model.score(x_train, y_train))
print("Test score:", model.score(x_test, y_test))
print("Coefficient:", model.coef_)
print("Intercept:", model.intercept_)

model_path = Path.cwd() / "data" / "SalaryPred.model"
pickle.dump(model, open(model_path, "wb"))
model = pickle.load(open(model_path, "rb"))
yr_of_exp = float(input("Enter Years of Experience: "))
yr_of_exp_NP = np.array([[yr_of_exp]])
Salary = model.predict(yr_of_exp_NP)
print("Estimated Salary for {} years of experience is {}".format(yr_of_exp, Salary))
```

Output

```
    YearsExperience  Salary
0               0.8   23996
1               1.4   30366
2               1.9   38171
3               2.3   35636
4               2.9   46093
5               3.2   44853
6               3.8   48006
7               4.1   60551
8               4.7   67254
9               5.0   67598
10              5.5   66710
11              5.9   64920
12              6.2   73657
13              6.8   77573
14              7.1   76628
15              7.6   71470
16              8.0   84537
17              8.3   76014
18              8.9   89843
19              9.2   88042
20              9.6  101582
21             10.1   94441
22             10.5   97217
23             11.0  104880
24             11.4  101315
25             12.0  104917
26             12.6  117506
27             13.1  113129
28             13.7  124354
29             14.2  116558
<class 'pandas.DataFrame'>
RangeIndex: 30 entries, 0 to 29
Data columns (total 2 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   YearsExperience  30 non-null     float64
 1   Salary           30 non-null     int64
dtypes: float64(1), int64(1)
memory usage: 612.0 bytes
<class 'pandas.DataFrame'>
RangeIndex: 30 entries, 0 to 29
Data columns (total 2 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   YearsExperience  30 non-null     float64
 1   Salary           30 non-null     int64
dtypes: float64(1), int64(1)
memory usage: 612.0 bytes
       YearsExperience         Salary
count        30.000000      30.000000
mean          7.393333   76927.233333
std           3.905693   27756.927547
min           0.800000   23996.000000
25%           4.250000   61643.250000
50%           7.350000   76321.000000
75%          10.400000  100290.500000
max          14.200000  124354.000000
Training score: 0.9773647143868749
Test score: 0.8629366169214914
Coefficient: [[7151.45536084]]
Intercept: [24029.43885]
Estimated Salary for 4.0 years of experience is [[52635.26029335]]
C:\Users\Myself\AppData\Local\Temp\ipykernel_16880\2351607553.py:13: Pandas4Warning: The inplace keyword in
    DataFrame.dropna is deprecated and will be removed in a future version (PDEP-8).
See https://pandas.pydata.org/docs/dev/whatsnew/v3.1.0.html#deprecation-inplace for more details.
  df.dropna(inplace=True)
```

Experiment-18

Code and complete execution output

Code Cell 1

```
import numpy as np
import pandas as pd
from pathlib import Path
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import confusion_matrix, classification_report

# Load an alternate Iris-shaped dataset
df = pd.read_csv(Path.cwd() / "data" / "iris_classification.csv")
df.info()
print(df.variety.value_counts())
print(df.head())
features = df.iloc[:, :-1].values
label = df.iloc[:, 4].values
xtrain, xtest, ytrain, ytest = train_test_split(
    features, label, test_size=0.2, random_state=42
)
model_KNN = KNeighborsClassifier(n_neighbors=5)
model_KNN.fit(xtrain, ytrain)
print("Training accuracy:", model_KNN.score(xtrain, ytrain))
print("Test accuracy:", model_KNN.score(xtest, ytest))
print("Confusion matrix:\n", confusion_matrix(label, model_KNN.predict(features)))
print(classification_report(label, model_KNN.predict(features)))
```

Output

```
<class 'pandas.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   sepal.length  150 non-null    float64
 1   sepal.width   150 non-null    float64
 2   petal.length  150 non-null    float64
 3   petal.width   150 non-null    float64
 4   variety       150 non-null    str
dtypes: float64(4), str(1)
memory usage: 6.0 KB
variety
Iris-virginica     50
Iris-versicolor    50
Iris-setosa        50
Name: count, dtype: int64
   sepal.length  sepal.width  petal.length  petal.width         variety
0           6.4          3.0           5.0          1.7   Iris-virginica
1           6.0          2.8           4.3          1.7  Iris-versicolor
2           5.4          2.9           1.7          0.4      Iris-setosa
3           5.3          2.7           4.3          1.5  Iris-versicolor
4           5.3          2.7           3.8          1.2  Iris-versicolor
Training accuracy: 1.0
Test accuracy: 1.0
Confusion matrix:
 [[50  0  0]
 [ 0 50  0]
 [ 0  0 50]]
                precision    recall  f1-score   support

    Iris-setosa       1.00      1.00      1.00        50
Iris-versicolor       1.00      1.00      1.00        50
 Iris-virginica       1.00      1.00      1.00        50

       accuracy                           1.00       150
      macro avg       1.00      1.00      1.00       150
   weighted avg       1.00      1.00      1.00       150
```

Experiment-19

Code and complete execution output

Code Cell 1

```
import numpy as np
import pandas as pd
from pathlib import Path
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report

# Load an alternate social-ad dataset
df = pd.read_csv(Path.cwd() / "data" / "Social_Network_Ads.csv")
print(df)
print(df.head())
features = df.iloc[:, [2, 3]].values
label = df.iloc[:, 4].values
print(features)
print(label)

# Compare train and test scores across split seeds, as in the sample workflow
for i in range(1, 401):
    x_train, x_test, y_train, y_test = train_test_split(
        features, label, test_size=0.2, random_state=i
    )
    model = LogisticRegression()
    model.fit(x_train, y_train)
    train_score = model.score(x_train, y_train)
    test_score = model.score(x_test, y_test)
    if test_score > train_score:
        print("Test {} Train{} Random State {}".format(test_score, train_score, i))

x_train, x_test, y_train, y_test = train_test_split(
    features, label, test_size=0.2, random_state=42
)
finalModel = LogisticRegression()
finalModel.fit(x_train, y_train)
print(finalModel.score(x_train, y_train))
print(finalModel.score(x_test, y_test))
print(classification_report(label, finalModel.predict(features)))
```

Output

```
      User ID  Gender  Age  EstimatedSalary  Purchased
0    21000001  Female   19           140608          0
1    21000002  Female   43           123719          1
2    21000003  Female   19           128950          1
3    21000004    Male   32           101924          1
4    21000005    Male   41            84458          1
..        ...     ...  ...              ...        ...
395  21000396  Female   42            83446          1
396  21000397  Female   34            39525          0
397  21000398    Male   59           139152          1
398  21000399  Female   39            63955          1
399  21000400    Male   55           149223          1

[400 rows x 5 columns]
    User ID  Gender  Age  EstimatedSalary  Purchased
0  21000001  Female   19           140608          0
1  21000002  Female   43           123719          1
2  21000003  Female   19           128950          1
3  21000004    Male   32           101924          1
4  21000005    Male   41            84458          1
[[    19 140608]
 [    43 123719]
 [    19 128950]
 [    32 101924]
 [    41  84458]
 [    30 101590]
 [    57 161879]
 [    24  82427]
 [    46 141752]
 [    58 154922]
 [    59 122487]
 [    53 128447]
 [    45  54873]
 [    47 136723]
 [    48 105729]
 [    36  73989]
 [    22 135315]
 [    43 119821]
 [    21 126102]
 [    22  96718]
 [    24 144831]
 [    60  90034]
 [    42 138291]
 [    42 133771]
 [    40 155882]
 [    56 149916]
 [    45 110138]
 [    48 112967]
 [    43  39582]
 [    44  35181]
 [    58 152273]
 [    57 117969]
 [    50 109226]
 [    27 121031]
 [    28  39998]
 [    52 151488]
 [    21 131137]
 [    55 129503]
 [    19  53215]
 [    51  74233]
 [    39 100185]
 [    19 157073]
 [    32  41559]
 [    34  83203]
 [    23  58031]
 [    55  61703]
 [    49  67812]
 [    30  27793]
 [    20  86399]
 [    19 132352]
 [    42  34749]
 [    40 142677]
 [    38  86691]
 [    31  42341]
 [    27  52381]
 [    36  42369]
 [    59 140028]
 [    48  68833]
 [    59 103113]
 [    44 102223]
 [    22 114022]
 [    46 116779]
```

```
[    21  31181]
[    59  40687]
[    42  87090]
[    28  93072]
[    59  88302]
[    37  72261]
[    44 151023]
[    36  28646]
[    46  50184]
[    43  65093]
[    45  69566]
[    48  33313]
[    46  43554]
[    20 150607]
[    25 107594]
[    50 142603]
[    28 143171]
[    48  64952]
[    39  33101]
[    19  60125]
[    42 115015]
[    18  89895]
[    56  67244]
[    26  67347]
[    18 133127]
[    54  26668]
[    35 133390]
[    50 135971]
[    27  42140]
[    58  58973]
[    49 133747]
[    42 158253]
[    47 139946]
[    40 145762]
[    35 130816]
[    40  40177]
[    55 163729]
[    36  61173]
[    58  81833]
[    41 158898]
[    42  26605]
[    29 158558]
[    35  37691]
[    45 127683]
[    41 108032]
[    47 137370]
[    24  50657]
[    59  76041]
[    47 127562]
[    48 138992]
[    60  51653]
[    49  48942]
[    49 154344]
[    55  51487]
[    42  49827]
[    21  38769]
[    27 149470]
[    59  74782]
[    26  91965]
[    33 121039]
[    48 127071]
[    37  52471]
[    56 153975]
[    44 131853]
[    50 105964]
[    52  75678]
[    58  18544]
[    53 148975]
[    48 102665]
[    50 119580]
[    32  75775]
[    25  91589]
[    50 148975]
[    33 128425]
[    49  52786]
[    53 155695]
[    53 139736]
[    43 148114]
[    58 156455]
[    41  87980]
[    43 140272]
[    40  40653]
[    38  18510]
[    45 137003]
```

```
[    51  29108]
[    49 122025]
[    39 131300]
[    55  70481]
[    27 117670]
[    41  31473]
[    42 123710]
[    56  35315]
[    47 112971]
[    34  99176]
[    26 134764]
[    44  88750]
[    29 137742]
[    32  82371]
[    42 115473]
[    33  55815]
[    25  47279]
[    55 131216]
[    48  87161]
[    18 127645]
[    28  85914]
[    20 136523]
[    23 162899]
[    58  31930]
[    41  38774]
[    59  92763]
[    57  78618]
[    54 114983]
[    39 128666]
[    22  24859]
[    23  93736]
[    53  18020]
[    59  33845]
[    19  33796]
[    29 121442]
[    53 159807]
[    30  25932]
[    46  24228]
[    33  66698]
[    30 128654]
[    51  59445]
[    45  29710]
[    24  59301]
[    35  59314]
[    51  28784]
[    57  97737]
[    28 132804]
[    26 151654]
[    28 140153]
[    22  56495]
[    24  58548]
[    48  80531]
[    28  93220]
[    23 124107]
[    55  59294]
[    19 121628]
[    26  19482]
[    40  76314]
[    30  77483]
[    53  49801]
[    31  22294]
[    31  63141]
[    18 120702]
[    23  48614]
[    26 119016]
[    38  97413]
[    28  98553]
[    42  35374]
[    45 112541]
[    54  38918]
[    28 109831]
[    21 132344]
[    23  38115]
[    28  38217]
[    42 131234]
[    44 138330]
[    20  39200]
[    59 161609]
[    47  66641]
[    57  43759]
[    19  60242]
[    55  70388]
[    18  48565]
[    52  91453]
```

```
[    47 147165]
[    26 143790]
[    53 124554]
[    43 145072]
[    56  75446]
[    24 150168]
[    38  35554]
[    38 138461]
[    37  51540]
[    26 151985]
[    46 161093]
[    60  30511]
[    38 132926]
[    47 137414]
[    25 121923]
[    27 149523]
[    26 135471]
[    28 127265]
[    41  53853]
[    18  86780]
[    54 129364]
[    29  53419]
[    43 156214]
[    58 156552]
[    59  97194]
[    56  50465]
[    35 137397]
[    25 146642]
[    41 130960]
[    59  71968]
[    25 121350]
[    39  74276]
[    24 148330]
[    60 109949]
[    47  70815]
[    24 114909]
[    38  30856]
[    34 112679]
[    32  27834]
[    60 125360]
[    54 106862]
[    36  86042]
[    54  96816]
[    20  82527]
[    28 105096]
[    30  86239]
[    56  80366]
[    52 142869]
[    40  58107]
[    56 136236]
[    20 137030]
[    20 109617]
[    54 138784]
[    48 161027]
[    59 144490]
[    42  43863]
[    42 144276]
[    26  33286]
[    46 115047]
[    27  36569]
[    19 144207]
[    28  60791]
[    39  25977]
[    44  96734]
[    50  33636]
[    30  29633]
[    28  98889]
[    27 126763]
[    21 149132]
[    60  47081]
[    43  69486]
[    42 100753]
[    59  63614]
[    20  52817]
[    19 153401]
[    49  91953]
[    44  87615]
[    49  87214]
[    48  58856]
[    44 145870]
[    25 154274]
[    23  35018]
[    33  74981]
[    30  45163]
```

```
[    49  43907]
[    39  72079]
[    52 155814]
[    51 147471]
[    31  72892]
[    55  48293]
[    23  95929]
[    41  31275]
[    21  72979]
[    58 162166]
[    59 159985]
[    57 120941]
[    35 164159]
[    26 147341]
[    27  36595]
[    19  51488]
[    58 134037]
[    35  51794]
[    55  79953]
[    52  77126]
[    46 127306]
[    21  86308]
[    26  80862]
[    20 100977]
[    59  80123]
[    37 120241]
[    36 154273]
[    39  70245]
[    23 161450]
[    20  96928]
[    41  59042]
[    55 132081]
[    24  80946]
[    45 117576]
[    42  67154]
[    39 150108]
[    47 138695]
[    28  20142]
[    19 105852]
[    19  94139]
[    46  25334]
[    24 122628]
[    40 126677]
[    18 130482]
[    41 126607]
[    27  38821]
[    49  29978]
[    24 124247]
[    36  75499]
[    49 145015]
[    57 114051]
[    36  26942]
[    41 152860]
[    45 118541]
[    46  57501]
[    30 133853]
[    38  66082]
[    31  52361]
[    27 155538]
[    45 132265]
[    34  78488]
[    21 148415]
[    43  99923]
[    27  63714]
[    30 126491]
[    25 150254]
[    58  72148]
[    42  75789]
[    30  90969]
[    52 138425]
[    32 161432]
[    33  89496]
[    36  18942]
[    24  19268]
[    46 128986]
[    29 101323]
[    29  46822]
[    31  99419]
[    60  21146]
[    27 115449]
[    56  71922]
[    42  83446]
[    34  39525]
[    59 139152]
```

```
 [    39  63955]
 [    55 149223]]
[0 1 1 1 1 0 1 0 1 1 1 1 0 1 1 1 0 1 0 0 1 0 1 1 1 1 1 1 1 1 1 1 1 0 1 0
 1 0 1 0 1 1 0 0 1 1 0 0 0 0 1 1 0 0 1 1 1 1 1 1 1 0 1 0 0 1 1 1 0 1 1 1 0
 0 1 1 1 1 0 1 0 1 1 1 0 0 0 1 1 0 1 1 1 1 1 0 0 1 0 1 1 1 1 1 1 1 1 0 1 1
 1 1 1 1 1 0 0 1 1 1 0 1 1 1 1 1 1 0 1 1 1 0 0 1 1 0 1 1 1 1 1 1 0 0 1 1 1
 1 1 1 0 1 1 1 1 1 1 1 1 1 1 0 1 1 1 0 1 0 1 1 0 1 1 1 0 0 1 1 0 1 1 0 0 0
 1 1 0 0 0 1 1 0 1 1 0 0 1 0 1 0 0 0 1 1 0 0 0 1 0 0 1 0 1 1 1 0 1 0 0 1 1
 0 1 1 0 0 1 0 1 1 0 1 1 1 1 0 1 0 1 1 1 1 1 1 1 1 0 0 1 0 1 1 0 1 1 1 1
 1 0 1 1 1 1 1 1 0 0 0 1 1 1 1 1 0 0 0 1 1 1 1 1 1 1 1 1 0 1 0 1 0 1 0 1 0
 1 1 0 1 1 1 1 1 0 1 1 1 0 0 1 1 0 0 0 1 1 1 1 0 1 0 1 0 1 1 1 1 1 1 0 0 1 0 1
 1 1 0 0 0 1 1 1 1 0 0 0 1 0 1 1 1 1 0 0 0 1 1 1 1 1 1 0 0 0 0 1 1 0 1 1 1 1
 1 0 1 1 1 1 1 0 1 1 1 1 1 1 1 1 0 0 1 1 0 0 1 1 1 1 1 0 1 1 1]
Test 0.85 Train0.8125 Random State 2
Test 0.825 Train0.81875 Random State 4
Test 0.8625 Train0.815625 Random State 6
Test 0.8625 Train0.81875 Random State 8
Test 0.85 Train0.828125 Random State 10
Test 0.85 Train0.815625 Random State 11
Test 0.8375 Train0.821875 Random State 12
Test 0.85 Train0.825 Random State 13
Test 0.8625 Train0.8125 Random State 14
Test 0.8625 Train0.809375 Random State 16
Test 0.825 Train0.815625 Random State 21
Test 0.8625 Train0.81875 Random State 22
Test 0.8375 Train0.81875 Random State 24
Test 0.8375 Train0.83125 Random State 25
Test 0.8375 Train0.8125 Random State 26
Test 0.8375 Train0.8125 Random State 27
Test 0.85 Train0.821875 Random State 28
Test 0.875 Train0.809375 Random State 31
Test 0.8375 Train0.83125 Random State 32
Test 0.85 Train0.81875 Random State 36
Test 0.8625 Train0.809375 Random State 37
Test 0.875 Train0.80625 Random State 38
Test 0.825 Train0.821875 Random State 43
Test 0.8625 Train0.815625 Random State 46
Test 0.8375 Train0.821875 Random State 47
Test 0.9 Train0.809375 Random State 48
Test 0.9 Train0.80625 Random State 52
Test 0.8875 Train0.809375 Random State 55
Test 0.8375 Train0.825 Random State 60
Test 0.85 Train0.81875 Random State 62
Test 0.8375 Train0.821875 Random State 63
Test 0.875 Train0.803125 Random State 64
Test 0.825 Train0.81875 Random State 65
Test 0.8375 Train0.828125 Random State 66
Test 0.85 Train0.81875 Random State 67
Test 0.825 Train0.815625 Random State 68
Test 0.85 Train0.815625 Random State 69
Test 0.85 Train0.821875 Random State 71
Test 0.85 Train0.815625 Random State 77
Test 0.8625 Train0.815625 Random State 79
Test 0.85 Train0.834375 Random State 80
Test 0.8375 Train0.809375 Random State 84
Test 0.8375 Train0.8125 Random State 85
Test 0.9 Train0.803125 Random State 86
Test 0.825 Train0.821875 Random State 87
Test 0.8625 Train0.815625 Random State 88
Test 0.8375 Train0.809375 Random State 89
Test 0.9 Train0.803125 Random State 90
Test 0.8625 Train0.815625 Random State 91
Test 0.85 Train0.8125 Random State 93
Test 0.825 Train0.815625 Random State 95
Test 0.9125 Train0.8 Random State 98
Test 0.85 Train0.8125 Random State 99
Test 0.8375 Train0.821875 Random State 101
Test 0.8625 Train0.80625 Random State 104
Test 0.875 Train0.80625 Random State 106
Test 0.875 Train0.8125 Random State 110
Test 0.8875 Train0.80625 Random State 112
Test 0.8625 Train0.83125 Random State 113
Test 0.85 Train0.815625 Random State 115
Test 0.85 Train0.828125 Random State 118
Test 0.85 Train0.81875 Random State 119
Test 0.8375 Train0.8125 Random State 123
Test 0.8375 Train0.821875 Random State 124
Test 0.85 Train0.83125 Random State 125
Test 0.825 Train0.821875 Random State 127
Test 0.8375 Train0.825 Random State 128
Test 0.8625 Train0.8125 Random State 129
Test 0.8625 Train0.8125 Random State 131
Test 0.825 Train0.821875 Random State 132
Test 0.875 Train0.815625 Random State 133
```

Test 0.8375 Train0.8125 Random State 141
Test 0.8375 Train0.81875 Random State 142
Test 0.85 Train0.81875 Random State 143
Test 0.85 Train0.821875 Random State 144
Test 0.8375 Train0.81875 Random State 145
Test 0.825 Train0.815625 Random State 146
Test 0.875 Train0.8125 Random State 148
Test 0.8625 Train0.828125 Random State 151
Test 0.825 Train0.81875 Random State 154
Test 0.8375 Train0.809375 Random State 155
Test 0.85 Train0.815625 Random State 156
Test 0.8625 Train0.825 Random State 163
Test 0.8875 Train0.809375 Random State 164
Test 0.8625 Train0.809375 Random State 165
Test 0.9 Train0.821875 Random State 167
Test 0.85 Train0.815625 Random State 169
Test 0.8375 Train0.828125 Random State 170
Test 0.8375 Train0.815625 Random State 177
Test 0.8375 Train0.821875 Random State 178
Test 0.8375 Train0.81875 Random State 181
Test 0.825 Train0.81875 Random State 182
Test 0.825 Train0.81875 Random State 186
Test 0.8375 Train0.828125 Random State 187
Test 0.85 Train0.81875 Random State 189
Test 0.8625 Train0.8125 Random State 190
Test 0.85 Train0.8125 Random State 192
Test 0.825 Train0.821875 Random State 193
Test 0.825 Train0.815625 Random State 195
Test 0.85 Train0.825 Random State 196
Test 0.825 Train0.81875 Random State 198
Test 0.875 Train0.803125 Random State 199
Test 0.875 Train0.809375 Random State 200
Test 0.8375 Train0.815625 Random State 203
Test 0.85 Train0.81875 Random State 208
Test 0.8625 Train0.815625 Random State 210
Test 0.875 Train0.8125 Random State 211
Test 0.85 Train0.81875 Random State 212
Test 0.8625 Train0.8125 Random State 213
Test 0.85 Train0.825 Random State 215
Test 0.8875 Train0.80625 Random State 217
Test 0.8375 Train0.821875 Random State 218
Test 0.8625 Train0.8125 Random State 223
Test 0.85 Train0.83125 Random State 224
Test 0.8875 Train0.815625 Random State 225
Test 0.875 Train0.8125 Random State 228
Test 0.825 Train0.821875 Random State 229
Test 0.8375 Train0.821875 Random State 231
Test 0.8375 Train0.821875 Random State 232
Test 0.8375 Train0.828125 Random State 233
Test 0.825 Train0.821875 Random State 235
Test 0.85 Train0.821875 Random State 237
Test 0.95 Train0.790625 Random State 242
Test 0.925 Train0.8 Random State 245
Test 0.9 Train0.803125 Random State 246
Test 0.8375 Train0.815625 Random State 248
Test 0.875 Train0.8125 Random State 249
Test 0.85 Train0.815625 Random State 250
Test 0.875 Train0.809375 Random State 251
Test 0.8875 Train0.803125 Random State 252
Test 0.85 Train0.821875 Random State 257
Test 0.8625 Train0.821875 Random State 259
Test 0.8875 Train0.81875 Random State 260
Test 0.85 Train0.834375 Random State 261
Test 0.85 Train0.815625 Random State 262
Test 0.8375 Train0.81875 Random State 266
Test 0.8375 Train0.825 Random State 268
Test 0.8625 Train0.828125 Random State 270
Test 0.85 Train0.821875 Random State 272
Test 0.85 Train0.821875 Random State 273
Test 0.8625 Train0.815625 Random State 274
Test 0.85 Train0.81875 Random State 275
Test 0.8375 Train0.821875 Random State 276
Test 0.8375 Train0.83125 Random State 279
Test 0.825 Train0.821875 Random State 282
Test 0.85 Train0.81875 Random State 283
Test 0.8875 Train0.80625 Random State 284
Test 0.825 Train0.8125 Random State 285
Test 0.8375 Train0.825 Random State 287
Test 0.875 Train0.809375 Random State 288
Test 0.8875 Train0.80625 Random State 293
Test 0.825 Train0.81875 Random State 299
Test 0.85 Train0.815625 Random State 300
Test 0.8375 Train0.821875 Random State 302
Test 0.8375 Train0.809375 Random State 304

```
Test 0.85 Train0.821875 Random State 305
Test 0.8375 Train0.81875 Random State 307
Test 0.8625 Train0.80625 Random State 308
Test 0.8375 Train0.815625 Random State 309
Test 0.8875 Train0.809375 Random State 310
Test 0.85 Train0.81875 Random State 311
Test 0.85 Train0.81875 Random State 313
Test 0.875 Train0.8125 Random State 314
Test 0.8875 Train0.803125 Random State 315
Test 0.85 Train0.8125 Random State 317
Test 0.8375 Train0.83125 Random State 323
Test 0.85 Train0.815625 Random State 324
Test 0.9125 Train0.803125 Random State 325
Test 0.825 Train0.8125 Random State 326
Test 0.8625 Train0.825 Random State 327
Test 0.825 Train0.821875 Random State 329
Test 0.8375 Train0.821875 Random State 330
Test 0.8875 Train0.815625 Random State 335
Test 0.875 Train0.809375 Random State 336
Test 0.8375 Train0.81875 Random State 337
Test 0.85 Train0.809375 Random State 342
Test 0.85 Train0.809375 Random State 344
Test 0.875 Train0.809375 Random State 345
Test 0.8375 Train0.81875 Random State 347
Test 0.825 Train0.821875 Random State 348
Test 0.8375 Train0.821875 Random State 349
Test 0.8375 Train0.81875 Random State 359
Test 0.875 Train0.8125 Random State 361
Test 0.8625 Train0.8125 Random State 368
Test 0.8375 Train0.81875 Random State 369
Test 0.875 Train0.815625 Random State 372
Test 0.8375 Train0.825 Random State 374
Test 0.8375 Train0.81875 Random State 376
Test 0.825 Train0.81875 Random State 377
Test 0.85 Train0.81875 Random State 380
Test 0.8375 Train0.821875 Random State 383
Test 0.825 Train0.821875 Random State 384
Test 0.8625 Train0.8125 Random State 386
Test 0.8375 Train0.81875 Random State 390
Test 0.825 Train0.821875 Random State 391
Test 0.85 Train0.8125 Random State 394
Test 0.8375 Train0.8125 Random State 396
Test 0.8875 Train0.80625 Random State 398
Test 0.8375 Train0.825 Random State 399
Test 0.9 Train0.81875 Random State 400
0.821875
0.7875
              precision    recall  f1-score   support

           0       0.73      0.71      0.72       134
           1       0.86      0.87      0.86       266

    accuracy                           0.81       400
   macro avg       0.79      0.79      0.79       400
weighted avg       0.81      0.81      0.81       400
```

Experiment-20

Code and complete execution output

Code Cell 1

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from pathlib import Path
from sklearn.cluster import KMeans

# Load alternate Mall Customers-shaped data
df = pd.read_csv(Path.cwd() / "data" / "Mall_Customers.csv")
df.info()
print(df.head())
sns.pairplot(df)
plt.show()

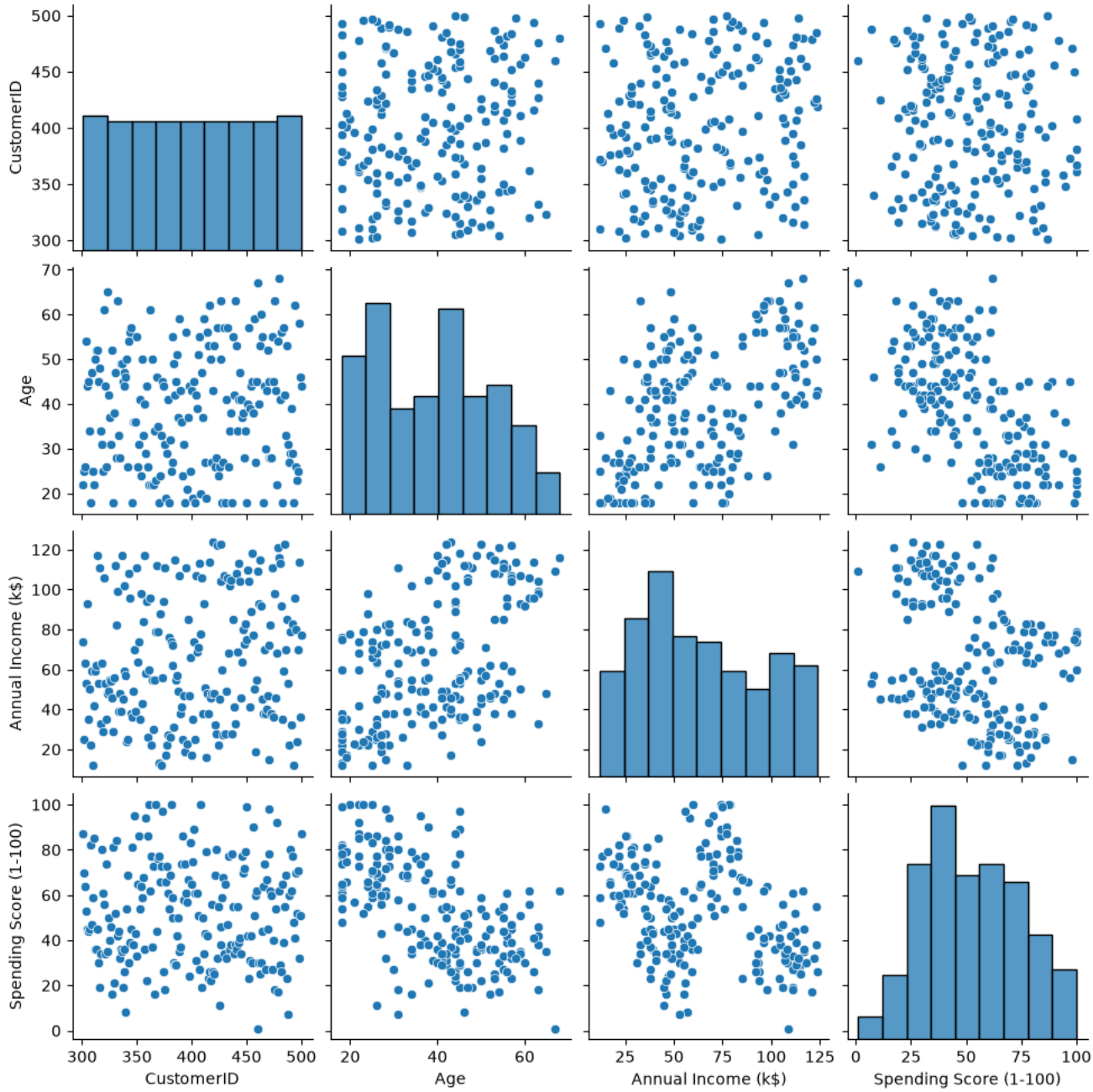
features = df.iloc[:, [3, 4]].values
model = KMeans(n_clusters=5)
model.fit(features)
Final = df.iloc[:, [3, 4]].copy()
Final["label"] = model.predict(features)
print(Final.head())

sns.set_style("whitegrid")
sns.FacetGrid(Final, hue="label", height=8).map(
    plt.scatter, "Annual Income (k$)", "Spending Score (1-100)"
).add_legend()
plt.show()

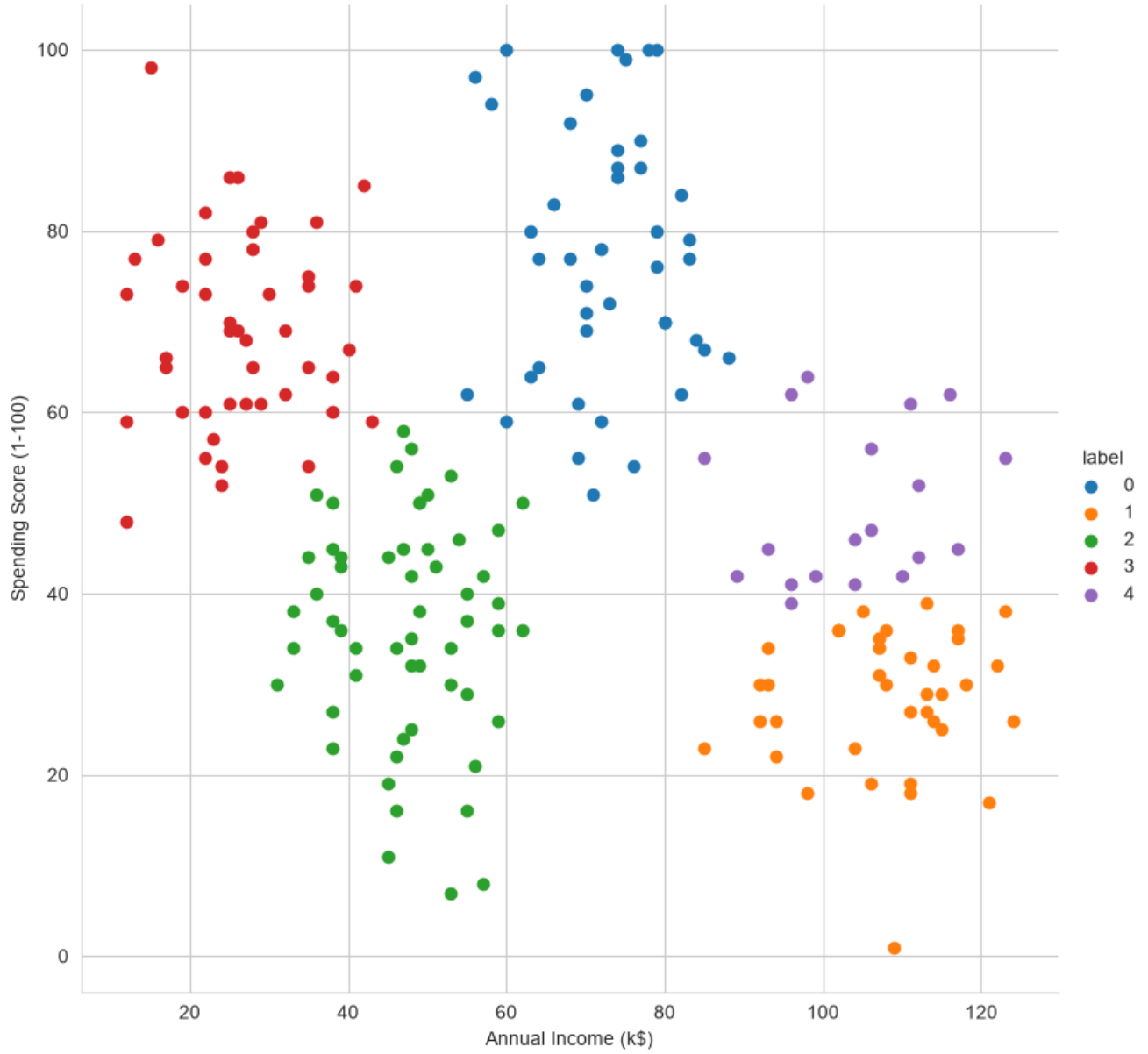
features_el = df.iloc[:, [2, 3, 4]].values
wcss = []
for i in range(1, 10):
    model = KMeans(n_clusters=i)
    model.fit(features_el)
    wcss.append(model.inertia_)
plt.plot(range(1, 10), wcss)
plt.xlabel("Number of clusters")
plt.ylabel("WCSS")
plt.title("Elbow method")
plt.show()
```

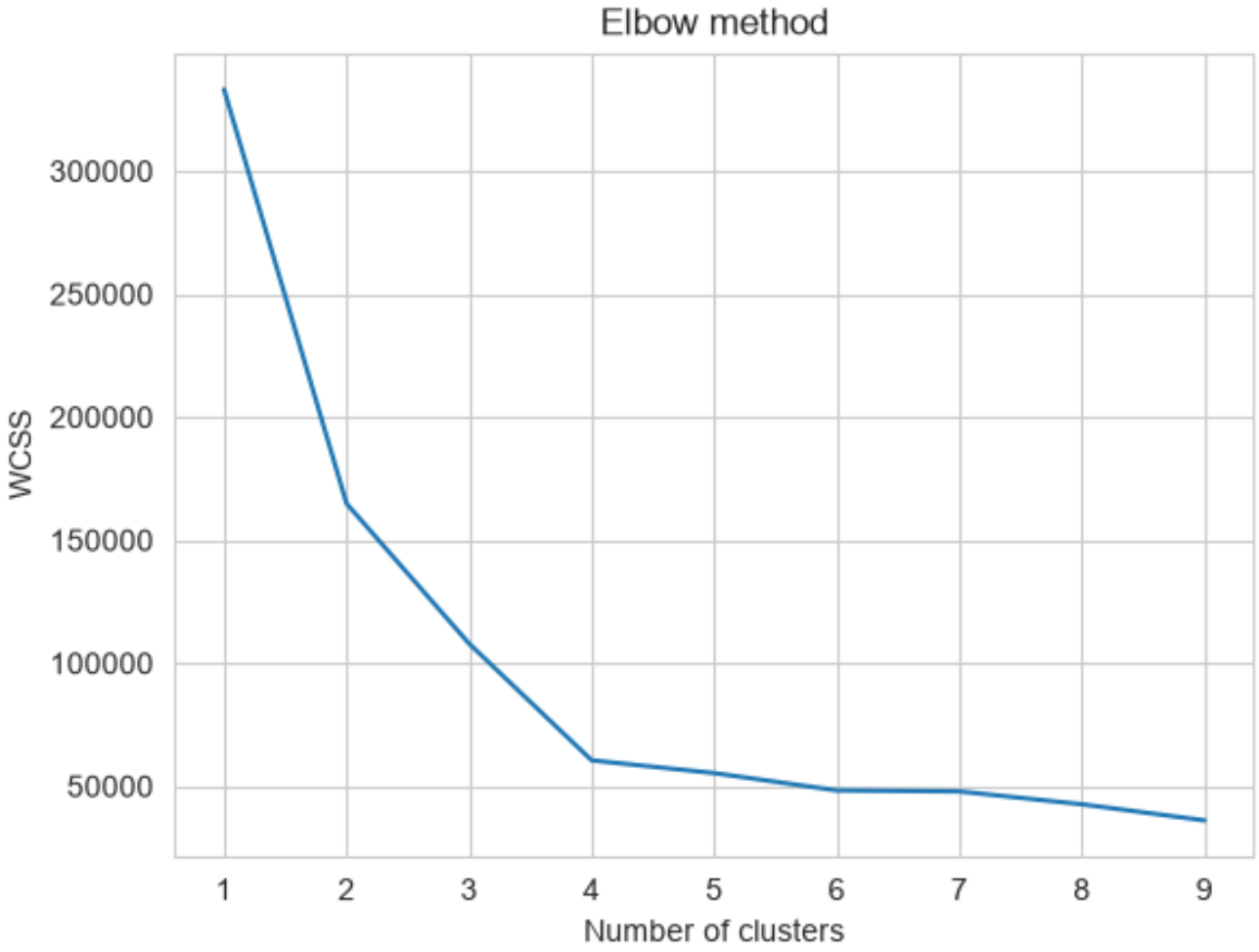
Output

```
<class 'pandas.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 5 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   CustomerID              200 non-null    int64
 1   Gender                  200 non-null    str
 2   Age                     200 non-null    int64
 3   Annual Income (k$)      200 non-null    int64
 4   Spending Score (1-100)  200 non-null    int64
dtypes: int64(4), str(1)
memory usage: 7.9 KB
   CustomerID  Gender  Age  Annual Income (k$)  Spending Score (1-100)
0         301    Male   22                  74                      87
1         302    Male   25                  25                      70
2         303    Male   26                  63                      64
3         304    Male   54                  53                      53
4         305  Female   44                  93                      45
```



	Annual Income (k$)	Spending Score (1-100)	label
0	74	87	0
1	25	70	3
2	63	64	0
3	53	53	2
4	93	45	4


Spending Score (1-100)
Annual Income (k$)
label
0
1
2
3
4


Elbow method
WCSS
300000
250000
200000
150000
100000
50000
1
2
3
4
5
6
7
8
9
Number of clusters